

VECTOR

Reports from the APL world:

- Kx Systems in London 50
- Forests in Finland IX 52
- Devon McCormick on Bayes 65
- Paul Cockshott on Vector Pascal 91
- Richard Smith on Rowan.net 101
- Dan Baronet on regular expressions 126



*The Journal of the
British APL Association*

A specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.21 No.2 Spring 2005

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2004-2005

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

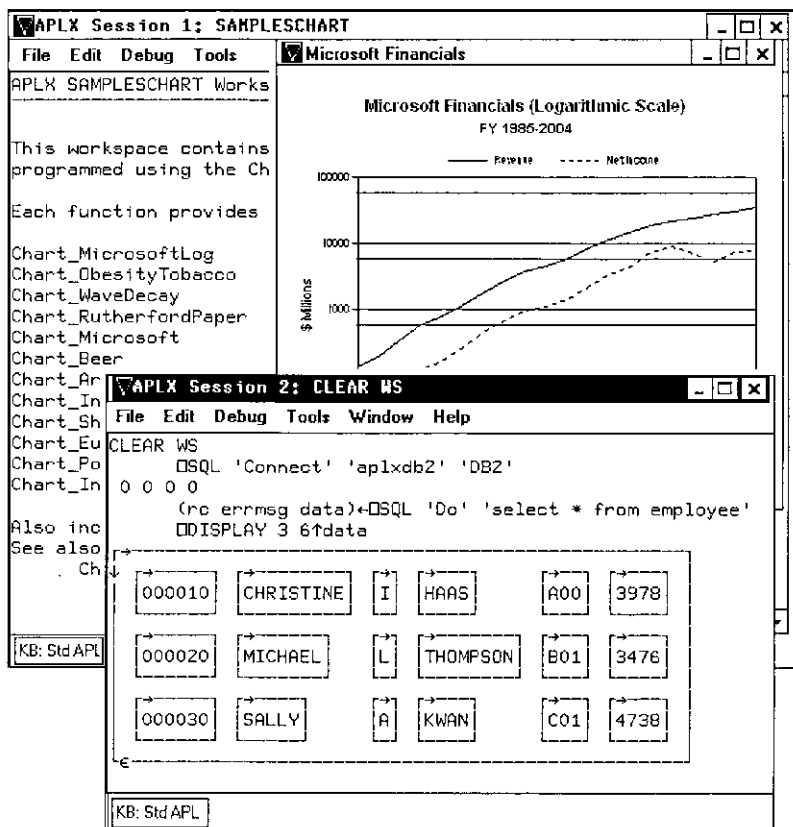
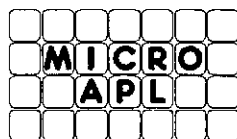
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Contents

		Page
Editorial:	Stephen Taylor	3
APL NEWS		
Quick Reference Diary		5
British APL Association News		
Notice of Annual General Meeting		6
APL Product Guide	compiled by Gill Smith	12
The Education Vector		
Zark Newsletter Extracts	edited by Jonathan Barman	27
Crossword and Puzzle		35
J-ottings 42: I (continue to) Spy with very little J ...	Norman Thomson	38
Review		
The Solid Form of Language (Robert Bringhurst)	Stephen Taylor	44
RECENT MEETINGS		
KX Systems Users' Meeting	Stephen Taylor	50
Forests in Finland IX	Adrian Smith	52
GENERAL ARTICLES		
Bayesian Financial Dynamic Linear Modelling	Devon McCormick	65
How Wrong was I?	Sylvia Camacho	78
Efficient Compilation of Array Expressions	Paul Cockshott	91
Rowan.net: An APL-like Interpreter for .Net	Richard Smith	101
TECHNICAL SECTION		
Hackers' Corner: Hack My Exe	Michael Baas	112
Technical Correspondence	Nakano, et al	116
Arithmetic Boundary Checking	Steve Mansour	119
Tools 4: Regular Expressions	Dan Baronet	126
APL to Unicode Transcoding in Java	Mark Wickens	137
Index to Advertisers		143

APLX



Version 3: Built-in charting. Built-in database access. Image manipulation. Browser. HTTP and E-Mail Clients.

The full-featured APL for Windows, MacOS and Linux.
Download a demo from <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Other trademarks acknowledged.

Editorial: Very Well, Alone Then

by Stephen Taylor (sjt@lambenttechnology.com)

Our Ken Iverson commemorative issue is still not ready to print and we've fallen behind our publication schedule. So here in 21.2 and shortly after in 21.3 are the issues we'd been planning to follow.

This last year has seen the near demise of Quote-Quad and the official APL conferences. ACM SIGAPL, if not actually dead, has been on life support for a while now. A rescue team has been recruited to refloat it, and we wish them well.

Arthur Whitney was in London in February for what has become Kx Systems' annual meeting with its users here. (See my note on p.50 of a talk by one of them.) The day after his conference, he met some of the BAA committee. This was the first time we'd met him since Iverson's death. "So," he said, counting faces, "it's just us now."

And here it's spring. The future of Dyalog Ltd has been secured for a while under the management of Morten Kromberg and Gitte Christensen. (See Sustaining Members' news.) The London community of Q programmers continues to grow. This year's conference suggests they might be ready to come out of hiding, and we plan to be carrying Q articles in most issues of Volume 22.

In my own work with Optima Systems we've been developing tactical systems for a large UK pension provider. Over the last year a handful of us have produced recurring savings in our client's operations of over £750,000 a year. Our client's IT division is said to be the second largest in the country, and has for years been deprecating our technology as 'non-strategic'. But as business conditions get tougher, our results are winning us an audience of managers who wonder if we might not have something to teach them about developing software.

Of course it's an old story: a handful of APL programmers produce results the 'official' developers can't match. In four decades we've heard this story so many times. And we know how it ends: the productive mavericks get sidelined, isolated and finally squeezed out. The tortoise beats Achilles by cutting his tendon.

Perhaps not this time. The results are in on Software Engineering by now. No one seriously believes great advances in productivity remain to be squeezed out of formal methods. Agile Development and Extreme Programming have done

serious damage to the doctrine that there is only one good way to produce software. The listening among business managers is changing.

This will be too late to inspire many of our readers, who have watched the decline in APL's usage and reputation in the last decades of their careers. But there's money to be made; it's only a question of who will do it.

Doing their bit in this issue are Stephen Mansour, checking where points fall between boundaries; Sylvia Camacho following up from Vector 18.2 on Bell's Inequality; Paul Cockshott on compiling array expressions; and Devon McCormick follows up on the Monty Hall problem from Vector 21.1. Richard Smith writes from Cambridge University, where he has constructed Rowan, an APL-like interpreter, out of .NET parts. Those undergraduates and their spare time!

We have the fourth and last instalment of Dan Baronet's series on tools. This is on Regular Expressions, a powerful tool fundamental to programming in Perl, and widely used by (e.g.) Ruby and PHP programmers. Even if you don't care to master this tool, study it long enough to point rudely to it the next time you hear the crack about APL code looking like noise.

Mark Wickens uses Java to translate source code between APLs from different vendors — keep grinning, J and Q programmers. Michael Baas likes distributing his Dyalog APL workspaces as EXE files and shares tricks for polishing the result.

More — very soon — in 21.3.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

**Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.**

Please note that Vol.1 No.2 is now out of stock.

Quick Reference Diary

Date	Venue	Event
May 20	RSS, London	British APL Association AGM and Dyalog workshop
May 23-24	Düsseldorf, Germany	APL Germany Technical Meeting
Oct 19-21 2005	Denmark	Dyalog 2005 Conference
Nov 6-10 2005	Florida	APL2000 User's Meeting. Naples Beach Hotel, Florida.

Dates for Future Issues of VECTOR

	Vol.21 No.3	Vol.21 No.4*	Vol.22 No.1
Copy date	in press	3rd June	1st July
Ad booking	in press	10th June	8th July
Ad Copy	in press	17th June	15th July
Distribution	April 2005	July 2005	August 2005

* This is the Ken Iverson special issue, and the "normal" 22:1 will be produced at the same time.

British APL Association AGM 2005
to be held at
the Royal Statistical Society
starting at 2 pm on Friday 20th May 2005

Agenda

1. Minutes of AGM 2004 (as published in Vector)
2. Report from the Chairman (Adrian Smith)
3. Report from the Treasurer (Nicholas Small)
(including report from membership secretary)
4. Committee for 2005-2006
Nominations by 30 April, please. The Committee welcomes people who would like to join the committee; details from the secretary or any member of the committee.
5. Appointment of Auditor
6. Questions
7. Any other business

Anthony Camacho, Hon Sec

Following the AGM there will be a Dyalog masterclass on object oriented programming starting at approximately 2.30pm.

News from Sustaining Members

Dyalog Ltd

New Management at Dyalog Ltd.

On April 7th 2005, a consortium consisting of *APL Italiana SpA*, *Insight Systems ApS* and *Simcorp A/S*, purchased 100% of the shares of Dyalog Ltd. On that date, *Gitte Christensen* and *Morten Kromberg*, currently Managing Director and Technical Director of Insight Systems ApS (respectively), assumed the same roles at Dyalog.

The direct reason for the reorganisation of the management team at Dyalog APL is that Peter Donnelly, Managing Directory of Dyalog, was keen to retire. Having identified that the team at Insight Systems ApS had the ideal combination of business and technical skills to take the APL business forward, APL Italiana SpA (of Milan, Italy) and SimCorp A/S (Copenhagen, Denmark) were identified as ideal financial backers, because they have a firm understanding of the value of APL and long term commitment to the technology at the highest level of management. The two companies both develop and market asset management solutions, and are thus competitors, but are keen to participate in this joint venture because both companies firmly believe that all customers, including themselves, are best served by a strong and independent Dyalog Ltd.

We believe that most readers of Vector are familiar with the qualities of Dyalog APL and the remarkable qualifications of the development team who are behind the product. However, we recognise that some of you may be less familiar with Gitte and Morten and the team at Insight.

The management team at Insight started their careers at I.P.Sharp Associates and have been working with APL for more than 25 years. During that time, the team has worked on just about every APL system and platform in commercial use since 1980, and primarily used Dyalog APL for the last ten years. Insight Systems, formed in 1990, are recognised as specialists in interfacing APL to components developed in other tools, in the tuning of large APL applications, and in conversions between different APL systems. The company has a broad software experience, ranging from Graphical (CAD) application development in C, MVS systems programming, APL system extensions in the form of Auxiliary Processors, DLLs and Unix add-ins developed in C, Web Applications using Visual Studio.NET, Client/Server technology as well as general software Analysis, Design and Teaching skills. Insight Systems have been purveyors of

portable SQL interfaces to a large part of the APL community for more than a decade, and have provided Dyalog with an ODBC interface for the last 13 years.

In the late 1990's, Insight Systems ApS was part of Adaytum Software Inc. The global development team which built Adaytum Planning was recruited and managed from Denmark by Gitte and Morten. The adventure ultimately resulted in the sale of Adaytum Inc. to Cognos in 2002. It is the proceeds from Insights participation in Adaytum which has now made it financially possible for Insight Systems ApS to perform a "management buy in" to Dyalog with the help of two of Dyalog's customers.

"We are both happy to be able to have this opportunity to contribute to the future of APL and we see our new roles as a logical conclusion of many years of professional dedication to APL" says Morten Kromberg.

"We believe that the combination of Dyalog's core technology with the application-building skills and sales and marketing know-how at Insight Systems will make it possible not only to provide an even higher level of service to existing customers, but also to substantially grow the company and the APL market in the 21st century. The most significant benefit to existing customers is that this will enable a significantly higher level of investment in APL technology and services. We firmly believe that APL is a development methodology which is still in its infancy, with significant growth potential."

Commenting on the deal, Peter Donnelly said:

"This deal has not come about by accident. Two years ago, John Scholes and I recognised the need to find the right people to take over the Dyalog business to ensure its future growth and prosperity when we eventually retired. We decided that Morten Kromberg and Gitte Christensen had exactly the right combination of APL technical skills, enthusiasm, and commercial experience that we were looking for. Early in 2004, with the full agreement and support of our staff, we approached Morten and Gitte. They responded enthusiastically, and the process began.

Both John and I are delighted that our plans have come to fruition and we confidently hand over the business to the new management. We know that Dyalog is in good hands and will continue to serve our customers well."

John Scholes will continue to work for Dyalog in his capacity as language designer. The interpreter development team will continue to be based in the UK. All current employees of the company, including Geoff Streeter and John Daintree, continue in their current rôles, and it is our plan to add new members to the team within the next year or two.

Administration will be handled from Denmark, where Helene Boesen will be responsible for Accounting / Sales / Shipping / Conferences and Events.

We would like to take this opportunity to invite you to the Dyalog 2005 conference, to be held in Denmark from October 19th to 21st. Our intention is to alternate between hosting the conference in European countries in odd-numbered years, and North America in even-numbered years – in co-operation with other vendors of Array Languages if practical. We sincerely hope that you will attend our conferences regularly, participate in sharing your solutions and problems, and in building a thriving community.

For more information, please visit the Dyalog web site at <http://www.dyalog.com>.

MicroAPL Ltd April 2005

MicroAPL is pleased to announce the release of Version 3 of APLX, available for Windows, MacOS and Linux.

The new version is designed to extend the APLX philosophy of providing the easiest-to-use, most productive APL development environment on all the major desktop platforms. Of particular interest will be the new built-in charting, networking, image-manipulation, and database access facilities.

Key features of the upgrade include:

- A built-in, very powerful, but easy-to-use facility for producing business, statistical, and scientific graphs. At its simplest, you can use a new 'right-click' menu item to bring up an instant chart of an APL variable. The new `CHART` system functions allows the APL programmer to produce quick charts with almost no programming, and the `WI` Chart object provides a full-featured way of placing graphs in your APLX applications. The types of chart which can be displayed include Line, Area, Scatter, Bar, Stacked Bar, Horizontal Bar, Horizontal Stacked Bar, High-Low-Open-Close, and Candlestick graphs.
- The new system function `SQL` provides a built-in, extensible, cross-platform interface to standard relational databases such as Oracle, SQL Server, IBM DB2, MySQL, or PostgreSQL, using Structured Query Language (SQL). It can also be used (via ODBC) to access data from spreadsheets, Microsoft Access, db2 files, and many other data sources. You can use it to retrieve data from relational databases as nested arrays in the workspace, or to update a database directly from within APL.

- A new `WBI` Web-browser object allows you to incorporate web pages in your APLX application.
- New `WBI` networking and internet access facilities. The `SendMail` and `GetMail` objects allow you to send and receive e-mail under program control, and the `HTTPClient` object fetches pages and other files from web sites into APL variables. There is also a new `Socket` object for low-level networking.
- A new `Image` object. This is an interface to `ImageMagick`, allowing you to do image conversion to/from a wide range of graphics formats, as well as a huge range of image manipulation operations. You can use the `Draw` method (or a `Chart` object) to create the image to be manipulated or saved, or you can read it from file or from the clipboard.
- Other `WBI` enhancements include a `ToolButton` control, for buttons with images and for toolbars, a new `tooltip` property for hover-help, and new methods for coordinate conversion.
- New system functions `WTR` (translate text to/from APLX internal representation), `WSYMB`, `WSSIZE`, `WLIB`

In addition, the release incorporates a number of bug-fixes, and a major speed-up for the Linux version of APLX.

As with previous versions of APLX, all these facilities have been implemented so that, as far as possible, they work in the same way on all platforms, but take advantage of the strengths of particular systems. For example, the new Web Browser object is based on Internet Explorer under Windows, and Safari on MacOS, but presents the same programming interface in APL.

Time-limited but fully-functional demonstration copies of APLX Version 3 can be downloaded free of charge from our website <http://www.microapl.co.uk/apl>

Causeway Graphical Systems Ltd

Turning *RainPro* into *SharpPlot* has taken almost exactly 18 months, during which time we have had very little time or energy to do anything else. By choosing to keep development in APL, but ship as fully-managed (compiled) C# we took a considerable risk – as far as I know there have been no commercially successful compiler projects since the early days of STSC back in 1979. However, all's well that ends well and www.sharpplot.com is out there for all to see. We now have a graphics engine that is fast, lightweight and fits all the requirements of a top-

quality .Net component. Great for Causeway, but why should the APL world care?

It occurs to me that we may not be alone in wanting to ship APL-authored components to .Net customers! I also remember the time when APLers cared enough about speed to buy the STSC compiler (not cheap in those days) just to get a few bottleneck functions coded down into assembler. Maybe there is someone else out there who could use this stuff? What could we offer to the APL world?

- a lightweight library of APL primitives, hand-coded in C# and callable from any .Net language (such as C# or VB.Net). These implement the ISO-standard for first-generation APL and as much 'nested' stuff as we have needed so far. This is well-tested, well-documented and very reliable. Guide-price £1,000 per developer with site-wide deployment £5,000. Royalty-free runtimes, as always.
- a bunch of functions (implemented in both APL2000 and Dyalog APL) which take your APL source-code and generate one or more C# source-files which can be compiled into a DLL or a stand-alone application. This code is constantly evolving (see any recent backchat on the DFns newgroup), falls over in a huge heap at the slightest provocation, and issues amusing error messages like "FUNCTION ABUSE" at things it doesn't like. Seriously, it is not all that bad, but no way is it a product we could sell.
- a lot of expertise. Between us, we reckon we could look at an APL application and within a day, give a reliable estimate on what would need to be done to get it through the compiler hoops. We targeted C#, but there would be no problem in creating source in VB.net, Java, or even ANSI C if those were the language standards in your organisation.

Which leaves me thinking that the best way forward is probably a short seminar (say 2 days) which is both a technology preview and a workshop. My aim would be to assemble enough people to give the converter a serious workout, nail the more egregious bugs by teatime on the last day, and hand over a time-limited version to all the participants.

This way, you get to see the tool in action before you try it for yourself. I appreciate that this is not Causeway's usual approach, but I am really scared of the support cost of simply handing out trial copies. Did anyone watch *Raiders of the Lost Ark* (for the fourteenth time) the other night? Just think of #.cc as a very dangerous box, and you will see why I want to make sure you know enough of the right words before you lift the lid!

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

A full audit of the guide will be carried out before the next Volume of Vector is produced, so please be ready to respond to our email and to confirm or update your entry. All unconfirmed entries will be archived and removed.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and tools	poa	Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-874, Part Number 45P7514
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
Oasis	Dyalog APL	poa	Dyalog Ltd
	APL+PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Dyalog's best, leading edge APL interpreter
RE Time Tracker Oy	APL+PC (APL+PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL+PLUS II)		
	APL+Win (APL+PLUS III), APL+Link		
	APL+UNIX		
	APL+PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL+PLUS products	poa	All APL+PLUS products including upgrades and educational.
	Dyalog and JSoftware products	poa	
	USA		
	Dyalog and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software\Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials, APL Conference Software, Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick_hotl@email.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.

CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K/CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, iC/E and IC/1 users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Entropy Software	Personal Information Management	poa	Entropy specialises in Personal Information Management Systems (PIMS) which run on a PC. Most systems are inadequate in some way. This is an attempt to fulfil all the needs of George MacLeod who has been an unhappy user of a variety of systems for many years. The product is being developed in modules (Calendar, Address List, To Do List, etc) which will be available at no charge. When all the modules are complete they will be integrated together. There will be a charge for the integrated product.
		poa	
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark

All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!

JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and DyalogW.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAAPL Pro	poa	ODBC interface for Dyalog APLW
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APLW
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APLW
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APLW
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.

	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book. (ISBN 0-9519067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc. \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, algorithms, tree processing and design programs for craft work.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL*Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consulting and maintenance for APL applications.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming: APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ian Clark	Consultancy	poa	Interfacing APL, VB, C/C++, ActiveX/COM. Screen design and documentation. National language porting.
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.

Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivatives plc	Consultancy	poa	Financial trading software in Q, K and kdb+
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APLW.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambert Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.

Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	poa	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT! customisation and issues concerning the internet and/or web design.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Graeme Robertson	Consultancy & Training	250-500	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Skelton Consulting	Consultancy	poa	K/Q Consultancy
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
		MACfns	MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and

Weighthead Systems Consultancy		poa	datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
Stephen Wynn	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
		poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
Atomic Vector	RELCELL	poa	Details: Report Writer, Data Mining, Database Visualization, and Export to Excel from relational databases. Offered in English, Spanish, Italian, Portuguese, French and German. With this product, you don't have to know English in order to use SQL. Point and click with easy to use Wizards.
	APL Games Volume 1	\$15	Details: Volume 1 - A collection of four APL games on cd, saved from past yearly issues of the APL Planet. Great way to demonstrate the capabilities of APL, and to entice the younger generations into APL. Written entirely in Dyalog APL.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Optima	IBM Compatible	poa	Complete business solutions including hardware, software and support plus additional network installation and maintenance if required.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.
		\$69.95 (edu)	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63489. Email: Dieter_Lattermann@t-online.de
APL Software Services	Dick Holt	31570 Heather Field Lane, Earlysville VA 22936 USA Tel: +1 (434) 973-4255; Fax: +1 (703) 528-7617; Email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AtomicVector Productions Inc.	John Mangas	413 Comanche Trail, Lawrenceville, GA 30044, USA. Tel: +1 (770) 972-3755 Email: E-mail: mailto:seapldoc@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rooiberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375; Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@blueyonder.co.uk
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 18510 USA. Tel: +1 570-963-2036
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email rolf@cinerea.se
Ian Clark	Ian Clark	1205 Deer Creek Drive, Plainsboro, NJ 08536, USA. Tel: +1 609 716 8832 Email: earthspot2000@hotmail.com
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework- it.com

ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyalog Ltd.	Peter Donnelly	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar del Segura, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod2003@yahoo.co.uk
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel: 028 3025 2242 Fax 028 3025 2060 Email: enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	3 S. Tulenim Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 98th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Juichiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel: +81(045)563-1411 Fax: +81(045)566-1617 E-mail: takeuchi@ae.keio.ac.jp
J Austria	Joachim Hoffmann	Hohenilz Nord 104, 8182 Puch bei Weiz, Austria. Tel: +43 (0)699 191 42 512 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSOFTWARE INC.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470- 7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi

Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambert Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lamberttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts. EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.78 Fax: +33.1.46.04.60.23 Email: eric@lescassee.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)7802 430 202 Email: alastair.kinloch@btinternet.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Rolier Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualedi Inc.	Nicole Schless, Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Resources & Results	Frank Rhodes	342 Glen Oaks Blvd. Dallas, TX 75232, USA. Tel: +1 (817) 501-9825 Email: frank21@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxyproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapidoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Skelton Consulting GmbH	Charles Skelton	Weinklinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com

SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5051, Obninsk-5, Kaluga Region 249020, Russia
Russian Chapter of SIGAPL		Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA
		Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131
		Fax: +1 (860) 868-8970 Email: rex@rexswain.com
SwedAPL	Christer Ulthielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServer: 100341.404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland
		Email: sl@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA
		Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada
		Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK.
		Tel: +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK.
		Tel: 01273-327238 Email: centre@boltblue.com
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Forum	www.aplforum.com
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net
AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/

First Derivatives plc	www.firstderivatives.com
Godin London Inc	www.godin.com/
Houben	http://home.wanadoo.nl/houb/IDA.htm
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.soliton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Zark Newsletter Extracts

Utility Corner:

Permutations and Anagrams

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's *Limbering Up* column, you were asked to define a monadic utility function `PERMUTE`, whose argument is a positive integer `N` and whose result is an `N`-column, `(!N)`-row matrix, in which each row is a different permutation of `1N`.

```

      PERMUTE 3
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

```

Your second task was to write a monadic function `ANAGRAM`, whose argument is a character vector, and whose result is a character matrix in which each row is a different arrangement of the argument's letters. (`ANAGRAM` calls `PERMUTE`.)

```

      ANAGRAM 'SMSI'
IMSS
ISMS
ISSM
MISS
MSIS
MSSI
SIMS
SISM
SMIS
SMIS
SSIM
SSMI

```

The `ANAGRAM` function is fairly straightforward:

```

▽ R←ANAGRAM WORD
[1]  R Returns all different combinations
[2]  R of the characters in vector WORD.

```

```

[3]  A Needs PERMUTE
[4]  A
[5]  A Sort chars to check for duplicates
[6]  A and to have result sorted if none:
[7]  WORD←WORD[⊞AV↓WORD]
[8]  A All permutations:
[9]  R←WORD[PERMUTE⊞WORD]
[10] A Exit if no duplicate chars
[11] →(⊞/WORD≠1⊞WORD)/0
[12] R←R[⊞AV↓R;] A Sort permutations
[13] A Eliminate duplicates
[14] R←(⊞1↓1,⊞/R≠1⊞R)⊞R

```

▽

There were two common mistakes made within ANAGRAM functions submitted. The first was to forget to eliminate the duplicate permutations caused by redundant letters. For example, the permutations 'SMSI' [2 4 1 3] and 'SMSI' [2 4 3 1] both return the string MISS.

The second mistake was to assume you can avoid sorting the character matrix (to catch duplicates) by first sorting the character string. The assumption seems reasonable but isn't. For example, the sorted permutations 'IMSS' [1 3 2 4], 'IMSS' [1 3 4 2] and 'IMSS' [1 4 2 3] return the unsorted strings ISMS, ISSM, ISMS.

The PERMUTE functions submitted seemed to fall into three categories: those that used recursion (PERMUTE calls itself), those that used looping, and those that boldly solved the problem without recursion or looping.

Let's address the bold solutions first. George Brigham's solution appeared in the May 1995 issue of the Big APL Newsletter (published by the New York chapter of SIGAPL: NY/SIGAPL, P.O. Box 2697, New York, NY 10163-2697).

```

▽ R←PERMUTE N
[1]  A Returns an N-column matrix, in
[2]  A which each row is a different
[3]  A permutation of 1N. Has !N rows.
[4]  A
[5]  R←(N/N)τ(1N*N)-⊞IO
[6]  R←⊞IO⊞⊞(⊞/⊞/[1+⊞IO]((1N)-⊞IO)⊞.=R)/R

```

▽

In this solution, encode (τ) is used to generate all possible combinations of the numbers in 1N (in origin 0), even duplicates. For example, for N=3,

```

0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 2 2 2 2 2 2 2
0 0 0 1 1 1 2 2 2 0 0 0 1 1 1 2 2 2 0 0 1 1 1 2 2
0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2

```

Those columns that contain exactly one each of the numbers 0 1 2 are selected, the result is transposed, and the numbers are adjusted back to origin 1 if needed.

In the June 1995 issue of Big APL, J. Philip Benkard presented a variation on the theme:

```

▽ R←PERMUTE N
[1]  A Returns an N-column matrix, in
[2]  A which each row is a different
[3]  A permutation of ιN. Has !N rows.
[4]  A
[5]  R←(N/N)⌈(ιN*N)-⌊IO
[6]  R←⌊IO+⊖((N+.*(ιN)-⌊IO)=N+.*R)/R
▽

```

The numbers in the matrix are applied as exponents of N, and are added across the rows. The result for each column, say $(3*0)+(3*0)+(3*1)$, is then compared to the sum on N raised to each of the powers in ιN (in origin 0), say $(3*0)+(3*1)+(3*2)$. Only those columns with exactly one each of the numbers 0 1 2 will match the total and be selected.

In the category of recursive solutions is this function, the heart of which came from John Sullivan:

```

▽ R←X PERMUTE N;Y
[1]  A Returns an N-column matrix, in
[2]  A which each row is a different
[3]  A permutation of ιN. Has !N rows.
[4]  A Left argument is used when called
[5]  A recursively (if N≥4).
[6]  A
[7]  A For N∈0 1 2 3, results hard-coded:
[8]  →(N=0 1 2 3)/L0,L1,L2,L3
[9]  Y←ιN
[10] R←⊖, [⌊IO]/Y, "((<Y)~"Y)PERMUTE" N-1
[11] →L4
[12] L0:R←1 0⍴1 0
[13] →L4
[14] L1:R←1 1⍴1
[15] →L4
[16] L2:R←2 2⍴⌊IO+0 1 1 0
[17] →L4
[18] L3:R←6 3⍴⌊IO+0 1 2 0 2 1 1 0 2 1 2 0 2 0 1 2 1 0
[19] A

```

```

[20]  A Exit if no left argument
[21]  L4:→(0=⊂NC'X')/0
[22]  R←X[R] A Map result into left arg
      ▽

```

Because recursive solutions tend to call themselves at one less level of complexity (in this instance via PERMUTE N-1), any method that speeds up the lower levels will speed up the overall recursive process. In this case, PERMUTE checks whether its right argument is 0, 1, 2, or 3, and returns the permutation matrix directly without recursing further (lines [12], [14], [16], [18]).

The core algorithm of PERMUTE (line [10]) recognizes the fact that each higher level permutation is a function of the permutation below it. For example, here's the result of PERMUTE 4 (truncated, and with extra spacing):

```

1   2 3 4
1   2 4 3
1   3 2 4
1   3 4 2          1,(2 3 4)[PERMUTE 3]
1   4 2 3
1   4 3 2

2   1 3 4
2   1 4 3
2   3 1 4
2   3 4 1          2,(1 3 4)[PERMUTE 3]
2   4 1 3
2   4 3 1
    (etc.)

```

The actual indexing does not occur on line [10]. Instead, on that line, PERMUTE is given an optional left argument (e.g. 1 3 4) that is indexed on line [22].

John's PERMUTE function requires an APL implementation that supports nested arrays. The next recursive function, from Ram Biyani, takes a similar tack, but avoids the use of nested arrays.

```

      ▽ R←PERMUTE N:A;I;J;M;P;S;Y
[1]  A Returns an N-column matrix, in
[2]  A which each row is a different
[3]  A permutation of ιN. Has !N rows.
[4]  A Recursive if N≥4
[5]  A
[6]  A For N∈0 1 2 3, results hard-coded:
[7]  →(N=0 1 2 3)/L0,L1,L2,L3
[8]  M←1↑P J←PERMUTE P←N-1 A Do a prev one
[9]  S←N×N
[10] Y←ιN

```

```

[11] A Rows of tN each with one different
[12] A element moved to the front of row:
[13] A←Y,(N,P)p(Sp0,Np1)/SpY
[14] A Pluck col-wise with permutations:
[15] R←((N×M),N)pA[;⌊IO,1+J]
[16] →0
[17] L0:R←1 0p10
[18] →0
[19] L1:R←1 1p11
[20] →0
[21] L2:R←2 2p⌊IO+0 1 1 0
[22] →0
[23] L3:R←6 3p⌊IO+0 1 2 0 2 1 1 0 2 1 2 0 2 0 1 2 1 0

```

▽

One immediate advantage of this function over the previous one is that it makes just one recursive call at each level. For example, when doing PERMUTE 4, the execution of PERMUTE 3 occurs just once (on line [8]). In the previous function, PERMUTE 3 is called four times (via the each " operator).

The core algorithm of this function is located on lines [13] and [15]. The expression on line [13] returns a matrix in this form (for PERMUTE 4, shown with extra spacing):

		A	
1	2	3	4
2	1	3	4
3	1	2	4
4	1	2	3

To get the required permutations, you need to compute all permutations of the last three columns of each row. For example:

			1	2 3 4
			1	2 4 3
1	2 3 4	→	1	3 2 4
			1	3 4 2
			1	4 2 3
			1	4 3 2

This is done for all rows at once via column-indexing on line [15].

In the category of looping solutions is the following submission, which was developed as a group effort by members of the 'comp.lang.apl' group on the Internet. Among the contributors were Jim Weigang, Roger Hui, and others.

```

    ▽ R←PERMUTE N;A;I
[1]  A Returns an N-column matrix, in
[2]  A which each row is a different
[3]  A permutation of 1..N. Has !N rows.
[4]  A
[5]  R←1 0pI←0
[6]  L1:→(N<I+I+1)/L2
[7]  A←+λ(1I),[!IO]1,(I-1 1)pI↑-1
[8]  R←A[;!IO,1+R]
[9]  →L1
[10] L2:R←((!N),N)pR
    ▽

```

The heart of this algorithm is on lines [7] and [8], and is in fact the same algorithm used in the previous PERMUTE function. The matrix A computed on line [7] (say for $I=4$) is the same as the matrix A computed on line [13] of the previous function (for $N=4$).

One subtle difference between these two functions is the timing of the reshape operation on the result of the column-indexing. Reshaping is needed to convert from shape $(N, (!N-1), N)$ to shape $((!N), N)$. In the former function, reshaping is done immediately after indexing so the shape of the recursive call will be correct. In the latter function, reshaping is postponed until the end of the loop (on line [10]). By the time the reshape takes place, the shape of R is $(N, (N-1), (N-2), (N-3), \dots, 1, 1, N)$.

We did some timings to see which of these PERMUTE functions is fastest. Here are the results:

	<u>N=4</u>	<u>N=5</u>	<u>N=6</u>	<u>N=7</u>
No Loop 1	6.77	90.00	1601.82	Huge
No Loop 2	7.64	98.96	1648.18	Huge
Recurse 1	2.64	14.77	93.82	686.36
Recurse 2	1.00	2.27	6.05	31.18
Looping	1.73	2.64	6.05	30.55

The times have been scaled so they are all relative to the fastest time. For example, Recurse 1 takes 2.64 times as long to run for $N=4$ as does Recurse 2 for $N=4$; and Looping takes 30.55 times as long to run for $N=7$ as Recurse 2 does for $N=4$.

The two non-looping functions are painfully slow. Boldness doesn't necessarily translated to quickness. Too many comparisons and calculations are being made, especially as N gets larger.

The first recursive function is much better than the non-looping functions but gets unwieldy as N get large because of its need to call itself !N-1 times (though one-sixth [!3] of these calls are avoided due to the special case logic used when N is 3 or less). The last two functions, for practical purposes, are equally efficient. That's no surprise since they use the same approach. Once again, we see that recursing is just looping in sheep's clothing.

Limbering Up: Market Days

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

When working with investment data, a special calendar must be used - the calendar that has one day per "Market Day," i.e. per day that the securities market is open for trading. The market is not open at weekends nor on certain holidays. For certain analyses, it is necessary to move forward or backward a certain number of market days. How is this done?

Suppose the "Market Day" calendar is expressed in two arrays, *c iy* and *c id*:

```
      ciy
1994 1995
```

```
      pcid
2 12 31
```

```
      cid
WWDDDDWDDDDWDDDDWDDDDWDDDDWDD
DDDDWDDDDWDDDDWDDDDWDDDDWDDXXX
DDDDWDDDDWDDDDWDDDDWDDDDWDDDD
HWDDDDWDDDDWDDDDWDDDDWDDDDWDDX
WDDDDWDDDDWDDDDWDDDDWDDDDWDDH
DDDDWDDDDWDDDDWDDDDWDDDDWDDDDX
DWWHDDDDWDDDDWDDDDWDDDDWDDDDW
DDDDWDDDDWDDDDWDDDDWDDDDWDDDD
DDWWHDDDDWDDDDWDDDDWDDDDWDDDDX
WWDDDDWDDDDWDDDDWDDDDWDDDDWDD
DDDDWDDDDWDDDDWDDDDWDDDDWDDDDX
DDWWDDDDWDDDDWDDDDWDDDDWDDDDW
```

```

WHDDDDWDDDDWDDDDWDDDDWDD
DDDWDDDDWDDDDWDDDDWDDDDWDDXXX
DDDWDDDDWDDDDWDDDDWDDDDWDDDD
WWDDDDWDDDDWDDDDWDDDDWDDDDWDX
DDDDWDDDDWDDDDWDDDDWDDDDWDDDD
DDWDDDDWDDDDWDDDDWDDDDWDDDDDX
WWDHDDWDDDDWDDDDWDDDDWDDDDWDD
DDDDWDDDDWDDDDWDDDDWDDDDWDDDD
DWWHDDDDWDDDDWDDDDWDDDDWDDDDWDX
WDDDDWDDDDWDDDDWDDDDWDDDDWDD
DDDWDDDDWDDDDWDDDDWDDDDWDDDDX
DWDDDDWDDDDWDDDDWDDDDWDDDDWDD

```

`ciy` is a vector of the years in the calendar. `cid` is a 3-dimensional character array with one plane per year in `ciy`, 12 rows (one per month), and 31 columns (one per day). Thus, the ninth row of the second plane corresponds to September 1995:

```
DWWHDDDDWDDDDWDDDDWDDDDWDDDDWDX
```

The `Ds` correspond to market days, the `Ws` to weekend days, the `Hs` to holidays, and the `Xs` to non-existent days. For example, September 4 is a holiday (Labor Day); September 2 and 3 are weekend days; and September 31 does not exist.

Your task is to write a monadic function `DAYSPLUS` whose left argument is an array of (market day) dates and whose right argument is a corresponding array of numbers of days to add. The result is an array of the (market day) dates that come from the date "addition." For example,

```

19950906 DAYSPLUS 2 0 -1 -3
19950908 19950906 19950905 19950831

```

The arguments should be scalar conformable, you know, vector and vector returns vector, matrix and scalar returns matrix, and so on. Assume globals `ciy` and `cid`.

Send your solution to:

Vector Production, Brook House, Gilling East, York YO62 4JJ, UK
email: apl385@compuserve.com

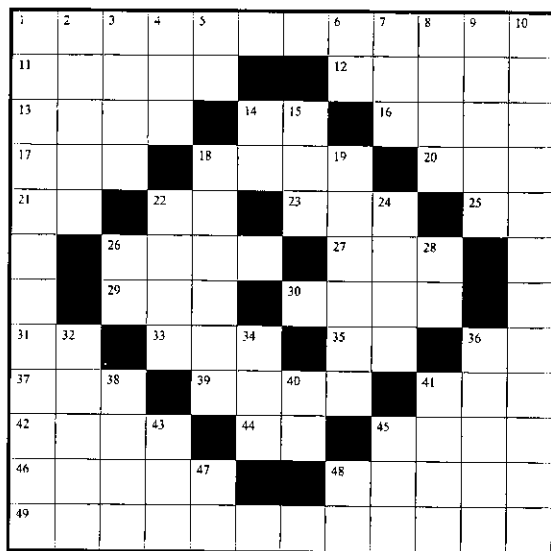
The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

21.1 Solution, Puzzle, and New Crossword

Solution to Crossword in 21.1

-	B	+	F	?	V		ρ	D	S	~	V
E	V	\	U	M		B	ρ	E	†	K	E
f	/	R	M		W	$\wedge$	V	N	D	$\equiv$	C
D	E	C	I	M	A	L	P	O	I	N	T
S	-	J	G	B		T		M		I	O
	1	$\in$	A	>	D		$\times$	I	N		R
4		E	T	C		+	/	N	$\in$	V	
L	Y		O		o		,	A	J	,	U
A	N	D	R	E	D	U	C	T	I	O	N
B	$\times$	*	M	F	$\times$	L		O	M	N	I
I	B	K	A	$\div$	H		-	R	†	ι	O
F	O	U	N	D		D	E	S	I	G	N

New Crossword



Across

1. How to turn PL or AL into APL
11. One random number from $\{-N\}, \{-2 \times N\}, \{-3 \times N\}, \dots, \{-L \times N\}$ to EV, without multiplying
12. $+/(R=R)/CP$ for vector R, scalar CP
13. Round elements of vector B down to nearest integer; rotate left N places
14. $Z > 0$ for non-negative Z
16. $(L=KO) \uparrow 1$ for vector L, scalar KO
17. $M - E \times \{M \div E\}$
18. Cumulative sum of the absolute values of the numbers in the vector Y
20. $D[\uparrow 3]$
21. 'ROAD' ~ 'AEIOU'
22. Any
23. $+/WpQ$ for scalars W and Q
25. $3 + 2\phi$ 'QUEUE'
26. Round to the nearest integer
27. $)_$ is the same as $)RESET$
29. $\sim K \in 2$
30. $(1n B) - (1n T)$
31. v
33. $V \uparrow U ? U$ where $V \leq U$

35. $(\ast 1) \ast N$
37. $)_$ to see what's suspended
39. Circumference of N circles, diameter D
41. $\vee / , \vee$
42. The larger of U or minus N
44. $\square_$ sets significant display digits
45. BI formatted to F[2] decimal places in a width of F[1] characters
46. Indices of negatives in vector G, where $C \leftarrow G < 0$
48. $\uparrow / A, T$ for scalar A, vector T
49. Branch of math with which o deals

Down

1. The only dyadic operator in std. APL
2. $((D \times D) \times 0.5) [\phi \uparrow \rho D] \times N$
3. $S + (1 | M) - M$
4. E distinct random numbers from $\uparrow 8$
5. Use $\square_$ to regenerate random numbers
6. $\square_$ tells whether a fn or var
7. World's most concise notation
8. O, (O+O), (O+O+O) (note: the letter O)
9. Pad vector Q to length K and reshape to matrix with shape R
10. APL: A _____ Language
14. Running product
15. Remainder from $W \div Z$
18. Sum of 52 distinct random numbers from $\uparrow N$
19. Raise SB to the power D; multiply by Y
22. $(MAT ____) \neq MAT$ to return all rows of MAT except those matching vector V
24. $(\div N) \times QI$
26. $K - 1 | K$
28. $\square_$ can alter the result of $A > B$
32. $' \uparrow \rho \Psi R$
34. $\ast (\oplus U) + \oplus P$
36. $(\div B) \geq ((B \times R) \sim 0) \uparrow S$ for bit vector B, nonzero vector R
38. $(D-1), (D-2), \dots, (D-I)$
40. P times pi
41. Round T up to nearest integer and format to one decimal place
43. $\div N \uparrow G$ for positive scalars N and G
45. $(\uparrow 0) \rho (F < E) \phi F, E$ for scalars F and E
47. $\rightarrow : ___$ to
48. 'PAM' ~ 'AP' [$\square IO + TIME > 1200$] for $TIME \leftarrow 100 \uparrow 2 \uparrow 3 \uparrow \square TS$ yesterday morning

J-ottings 42: I (continue to) Spy with very little J ...

by Norman Thomson

Way back in *J-ottings 25*, (the I-spy book of J) I described the basis of the cryptography which underlies the security of many of the millions of web transactions which take place every minute out in cyberspace. Such cryptographic systems based on substitution codes depend on using public and private keys. Although an identical technique is used for sending and receiving messages, it is nevertheless safe to broadcast one's public key to allow any sender to mail you encrypted messages, since only you possess the private key which is essential to decrypt them. This property makes such codes non-reciprocal. Reciprocal coding techniques are more primitive, and trade off the simplicity of doing the same thing to encrypt and decrypt against the requirement that the single underlying key must be kept secret between all communicating parties.

One of the best known of these techniques uses the so-called Vigenère tables after a sixteenth century French diplomat of that name. The idea is that the communicating parties share a table of alphabetic characters which resolve a pair of characters into a single character. 26 by 26 blocks take quite a lot of space, so I shall demonstrate the principle with the six-character alphabet 'ABCDES'. To make a Vigenère table, choose a word such as 'BED' which leads to a *table keyword* 'BEDACS' in which the characters not in the keyword are appended in alphabetic order. The table keyword then serves as row and column headers for the Vigenère table. To make the body of the table, start with the table keyword reversed and the remaining rows progressively shifted to the left :

	B	E	D	A	C	S
B	S	C	A	D	E	B
E	C	A	D	E	B	S
D	A	D	E	B	S	C
A	D	E	B	S	C	A
C	E	B	S	C	A	D
S	B	S	C	A	D	E

Both the encrypt and the decrypt of the letter pair EC is then B. No doubt you can already glimpse how well J is adapted to producing such tables!

Messages are converted into character pairs by the parties agreeing a *message keyword* (this is quite separate from the table keyword) which is written by

expanding it as often as is necessary beneath the message, so that if the message is "Abe's dad's a cad", and the keyword is "SAD" write

```
ABESDADSACAD
SADSADSADSAD
```

Using the columns as indexes to the table gives the coded message

```
ADDEBBECABDSE
```

Now repeat this procedure for the coded message

```
ADDEBBECABDSE
SADSADSADSAD
```

... and back comes the original message :

```
ABESDADSACAD
```

Now repeat the procedure, only this time using J in order to make notes which capture the thinking underlying the various steps. First a utility verb `maketab` performs the progressive row left shifts :

```
makevtab=.|. "0 1~i.@# NB. table of progressive shifts
(makevtab i.3);makevtab 'pqrs'
+-----+-----+
|0 1 2|pqrs|
|1 2 0|qrsp|
|2 0 1|rspq|
|      |spqr|
+-----+-----+
```

For the Vigenère table which uses 'SAD' as the table keyword define

```
tkey=: 'sadbce'
]vtab=:maketab |.tkey NB. Vig. table as fn. of tkey
ecbdas
cbdase
bdasec
dasecb
asecbd
secbda
```

With the same message as before, and ignoring blanks for simplicity, obtain the block of character pairs

```

msg=. 'abesdadsacad'
msg,: 'sad' ($~#)msg
abesdadsacad
sadsadsadsad

```

and then use the table keyword to convert this into indices into vtab :

```

]a=.tkey i.msg,: 'sad' ($~#)msg
1 3 5 0 2 1 2 0 1 4 1 2
0 1 2 0 1 2 0 1 2 0 1 2

```

For convenience, use append items (,) rather than laminate (,:) so that the indices are the primary lists (rows) and the code letters can be extracted by using box at rank 1 :

```

]a=.tkey i.msg,: 'sad' ($~#)msg
]encmsg=.(<"1 a){vtab NB. encrypted msg
caceddbcdaba

```

To decrypt, apply the same procedure to the encrypted message :

```

a=.tkey i.encmsg,.(#encmsg)$'sad'
(<"1 a){vtab
abesdadsacad

```

Now bring it all together in a single verb, for which there seems to be little point in struggling to achieve an ingenious tacit definition :

```

encrypt=.dyad : '(<"1 tkey i.y.,.(#y.)$x.){vtab'
'sad' encrypt msg NB.syntax is: mkey encrypt message
caceddbcdaba

```

and to decrypt use the same algorithm :

```

'sad' encrypt 'caceddbcdaba'
abesdadsacad
or equivalently :

```

```

'sad' encrypt^:2 msg
abesdadsacad

```

As confirmation, the above result should follow using any message keyword whose letters belong to the alphabet 'abcdes' :

```
'beads' encrypt ^:2 msg
abesdadsacad
'scab' encrypt ^:2 msg
abesdadsacad
```

Sorting the rows and columns into alphabetic order is only a consideration if the table is to be displayed for human as opposed to computer use. With a six-letter alphabet, even this is hardly worth while, but with a full alphabet of 26 letters it would certainly be desirable. Alphabetic row ordering using the table keyword is achieved by

```
order=.{~/:}&tkey
```

and column ordering by doing the same to the transpose, so that the form of vtab in which the row and column headings are both 'abcdes' is given by

```
vgrade=.order&|:@:order
]ordvtab=.vgrade vtab
basdec
aecsbd
scbeda
dseacb
ebdcas
cdbse
```

An amended encryption algorithm now uses ordvtab to achieve the same end result:

```
alph='abcdes'
aencrypt=.dyad : '(<"1 alph i.y.,.(#y.)$x.){ordvtab'
'sad' aencrypt msg
caceddbcdaba
'sad' aencrypt ^:2 msg
abesdadsacad
```

Of course any anagram of the alphabet will do for the table keyword, and a convenient way of mapping integers into keywords is to make tkey a derivative of alph by using the "ordered anagram" verb A. :

```
]tkey=.299 A. alph
cdbsea
```

vtab must be reassigned, and encrypt re-executed:

```
vtab=.makevtab |.tkey
encrypt=.dyad : '(<"1 tkey i.y.,.(#y.)$x.){vtab'
```

following which

```
'sad' encrypt msg
secaadsabds
'sad' encrypt ^:2 msg
abesdadsacad
```

Nothing new is required algorithmically to move up to a full alphabet, and there is no reason why spaces and other punctuation characters should not be thrown in for good measure. The variables `alph`, `tkey` and `vtab` have to be reassigned, and `encrypt` has to be re-executed:

```
alph=.' ',(97+i.26){a.
vtab=.makevtab |.tkey=.525 A. alph
encrypt=.dyad : '(<"1 tkey i.y.,.(#y.)$x.){vtab'
```

following which here is the encryption and decryption of a short message:

```
'vector' encrypt 'quick a lazy dog'
nvc hcykeondqhu
'vector' encrypt^:2 'quick a lazy dog'
quick a lazy dog
```

A Vigenère cipher can be made more secure by performing a final alphabetic shift using a further key, call it a *shift key*:

```
skey=.3
shift=.dyad : '((#alph)|x.+alph i.y.){alph'
```

The price of this extra security is that the encryption algorithm is no longer reciprocal. However, inverting the shift is simply a matter of applying minus to the shift key, so that the total operation is confirmed by

```
|x=.skey shift 'sad' encrypt 'quick a lazy dog'
txpgqyi miedjajc
'sad' encrypt (-skey)shift x
quick a lazy dog
```

The title of this article promised very little J. To demonstrate just how little, here is a summary in the form of a recipe for Vigenère based codes. First choose a global `tkey`, which might be explicit or generated by anagram number or otherwise, and then construct a global Vigenère table by

```
vtab=:makevtab |.tkey      NB. Vig. table from table keyword
makevtab=|. "0 i~i.@#      NB. make table of right shifts
```

then encrypt and decrypt messages using

```
encrypt=.dyad : '(<"1 tkey i.y.,.(#y.)$x.){vtab'
```

which has the syntax <message keyword> encrypt <message> :

For the further security of a shift key use alph=./:~tkey and

```
shift=.dyad : '((#alph)|x.+alph i.y.){alph'
```

Truly a lot with a little!

Liquid Mathematics (Review)

by Stephen Taylor

The Solid Form of Language by Robert Bringhurst

Does mathematics have a liquid form? This question is prompted by a new book from the poet, linguist and typographer Robert Bringhurst, author of the classic *The Elements Of Typographic Style*.

Writing is the solid form of language, the precipitate. Speech comes out of our mouths, our hands, our eyes in something like a liquid form and then evaporates at once. [...] Yet language can also solidify — into iridescent, sharp, symmetrical crystals, or into structures more like hailstones or shale beds or mud.

Bringhurst's essay, *The Solid Form of Language*, looks at the natural history of human writing systems and their relationships to languages.

Languages and scripts, like plant and animal species, are also subject to change. Their territories grow and shrink and subdivide and fuse, but there are none that are not mortal, none that will not someday be extinct.

Starting from scratch, with no imported models, people have made the shift from oral to literate culture at least three times but perhaps not many more than that. In Mesopotamia about 5,000 years ago, in northern China about 4,500 years ago, and in Guatemala and southern Mexico about 2,000 years ago, humans created a script and a scribal culture, apparently without imported models of any kind.

Writing is an "outgrowth of drawing" that acquires certain formal characteristics. Among them:

A writing system is codified. It consists of a repeating set of symbols sufficient to the language that it serves. twelve Latin letters are enough to write Hawaiian. To write a lengthy Chinese text, thousands of glyphs may be required. But whatever system is used, writers can write *what has never been written before* without inventing further symbols. The system is not entirely closed; new symbols *can* be borrowed or created. The point is that they very rarely have to be.

Æsthetics and technology combine to mould the letter forms.

As a calligraphic tradition develops, the symbols start to talk to one another, nourished by the dialogues of writers with their tools.

ΟΗΙΤΖΑΗΞΑΗΝΟΒΙΑΚΑΗΤΗΑΟΗΙΤΖΑΔΗΟΤΟΤ
 ΕΜΑΟΝΓΑΡΑΜΕΝΤΟΕΟΜΤΟΣΕΜΒΟΙΠΕΦΑΤΙΣΜ
 ΞΟΗΟΗΧΑΔΝΟΗΞΟΗΟΤΖΑΖΑΗΝΑΗΙΤΖΑΗΟΗΑ
 ΤΙΜΕΕΣΤΑΙΑΛΛΟΠΑΡΕΧΕΤΟΕΟΜΤΟΣΕΡΕΤΟΓΕ
 ΙΑΗΑΜΑΤΗΟΤΑΗΙΚΑΗΟΛΟΗΑΖΑΔΑΠΕΑΗΙΟΜ

ΤΑΟΤΟΜΔΕΣΤΙΜΟΕΙΜΤΕΚΑΙΟΝΜΕΚΕΜΕΣΤΙΜΟΒ
 ΜΑΙΟΝΓΑΡΑΜΕΝΤΟΝΕΟΜΤΟΣΕΜΩΙΠΕΦΑΤΙΣΜΕ
 ΜΟΜΕΣΤΙΜΕΝΡΘΞΕΙΣΤΟΜΟΕΙΜΙΟΝΔΕΧΡΟΜΟΣΕ
 ΣΤΙΜΒΕΣΤΑΙΑΛΛΟΠΑΡΕΞΤΟΝΕΟΜΤΟΣΕΡΕΙΤΟΓ
 ΕΜΟΙΡΑΕΡΕΔΒΣΕΜΟΝΛΟΜΑΚΙΜΒΤΟΜΤΕΜΕΜΑΙ

ταῦτον δ' ἐπὶ νοεῖν τε καὶ ἔνεκεν ἔστι νόημα.
 ἔ γάρ ἄνευ τῷ ἐόντι, ἐν ᾧ πεφασμένον ἐστίν,
 εὐρήσεις τὸ νοεῖν· ἔδὲ χρόνον ἔστιν ἢ ἔξαι
 ἄλλο πᾶρεξ τῷ ἐόντος, ἐπεὶ τόδε μοῖρ' ἐπέδησεν
 ἔλлон ἀκίνητόν τ' ἔμεναι.

Three versions of the same Greek text: a passage from the poem of Parmenides, composed about 500 BCE. Engraved in stone in Parmenides' time and place, the text might have looked like version 1: lines alternating direction and breaking arbitrarily; letters changing orientation with the lines; no spaces between the words. At Athens in Plato's time, it would have looked more like version 2. After many further transformations, Greek script had acquired punctuation, diacritics, and a cursive form replete with ligatures and alternates. In Western Europe during the Scottish Enlightenment, the same text would have looked like version 3.

In the example above we can see how the script used for Greek was adapted to changes in the inscribing technology. Like Scandinavian runes, the original glyphs were suitable for carving into wood or stone. The latest form is adapted to the fast and fluent strokes of pens.

Bringhurst's essay introduces us to a wide world of languages and scripts, evolving over thousands of years, shaped by factors both technical and political.

Turkish was written in Arabic script from the twelfth or thirteenth century until 1928, when a government decree forced a shift to the Latin alphabet. Tajik was first written in Arabic script, then for a time in Latin letters, but in the Soviet era, these scripts were replaced by Cyrillic. In 1991, when Tajikistan declared its independence, efforts were underway to return to Arabic script.

The forces of evolution are always at work. Typewriters brought crude and simple printing to every office. Setting this type no longer required a skilled craftsman, only a trained typist. Typewriters changed the economics of producing documents, and habits of writing, spelling and punctuation adapted to the change in the inscribing technology.

The spread of typewriters deprecated the use of characters not found on the QWERTY keyboard. So the use of ligatures in English came to be seen as at first *medieval*, then *mediaeval* and in American English, *medieval*.

A corresponding industrial æsthetic (sorry about that) sought the simplification of forms that produced typefaces such as Helvetica and Gill Sans.

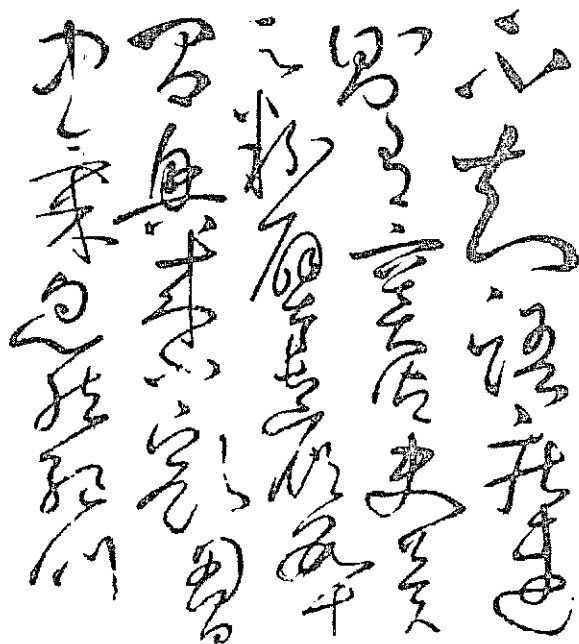
The same industrial civilization gave us Greek, Cyrillic, Hebrew, Devanagari, Japanese and Chinese types with similar characteristics: a global epidemic of *Helvetica*s.

The middle of the century saw the turn of this tide, as the defeat of Fascism ended the Modernist love affair with the iconography of heavy industry. Typographers like Herman Zapf drew faces inspired by Renaissance printers such as Aldo Manuzio. The new humanism saw lighter, more calligraphic forms; the closed forms of Helvetica were supplemented by the open and fluent shapes of Lucida Sans.

A generation ago typing was still a specialist job, albeit lowly. As a programmer, I was assured that managers would never use keyboards. The spread of personal computers has changed all that. No one is now considered competent for office work without basic keyboard skills.

At first the Anglo-Saxon cultural and technological hegemony looked set to condense written expression further around the English language and the QWERTY keyboard; or more precisely, the ASCII character set. For those of us who worked between countries it was possible to regard the alphabetic variations of other European languages and their local keyboards as anomalies to be accommodated for the time being by the Extended Latin ASCII characters, but eventually fated to disappear. Surely one day all Germans would write *Strasse* and not *Straße*?

That view now looks hopelessly naïve. While English remains unassailed as the *lingua franca* of international business, the spread of Net-linked PCs has fuelled demand for access to the new technology through the languages people already speak. Machines remain in the service of humans.



Part of a letter written in *kuancáo* or Wild Grass cursive by the monk Huáisǔ, c. 725-785. [After Chiang Yee 1973.]

Between us, my partner and I write in APL, Danish, English, French, German, HTML, Italian, Japanese, Javascript, PHP and Swahili. Japanese writers use three native scripts (one phonetic, another adapted from Chinese ideographs), Arabic numbers and Latin letters (*rōmaji*). Japanese is possibly the most challenging of our languages to type on a computer keyboard, but it is a thoroughly solved problem.

When I last bought a computer I set aside some time to configure it for writing in Japanese. I need not have troubled. On a machine bought in a non-specialist store in New York, standard configuration options allowed me to select the writing

systems we wanted to use. (These included native American languages I'd scarcely heard of. Bringhurst, who speaks some, must be pleased.) Internationalisation — across languages and writing systems — is now commonplace.

The keyboard mappings shipped with MacOS X include native American languages, but not APL. Presumably our linguistic community is smaller. More importantly, we don't have a consensus on what our keyboard maps should be. But the Unicode positions are fixed for our glyphs, and it was trivial to install Adrian Smith's APL385 Unicode font.

In the narrow world of information technology, English and the ASCII character set rule. From the wider perspective that Bringhurst opens, machines remain in our service, and human preferences are insisting they use our languages and our scripts. Twenty years ago it looked as if the change in inscribing technology, from golfball and daisywheel to CRT, would kill off APL's elegant glyphs. Now we can see that the script can remain — if we want it.

The latest APL, Rowan, has been developed by Richard Smith who is an undergraduate at Cambridge University. He has built it using .NET components and it is "Unicode from the little rubber feet up," as his father Adrian says. I was struck by Richard's use of traditional APL glyphs and asked him about it. He said he liked having single character glyphs instead of reserved words. Well I always did too.

Does mathematics have a liquid form? It is a difficult language to speak. Unlike natural languages, it flourishes mostly in its solid form. (Though I do remember awaking many years ago from a dream in which some repeated event had occurred to me as a repeated 1-drop on a list.) Iverson's notation has evolved rapidly since the publication of *A Programming Language* in 1962. So too has the inscribing technology. Bringhurst opens our eyes to a landscape in which the solid form of mathematics enjoys more freedom than we thought it had.

Robert Bringhurst, *The Solid Form of Language*, Gaspereau Press, Canada. ISBN 1-894031-88-1. 75pp. C\$19.95, US\$16.95.

See <http://www.gaspereau.com/solidform.html>

RECENT MEETINGS

In this issue, we cover the second Kx international users' meeting (London) and the ninth annual forest seminar (Finland). Thanks to Stephen for making sense of the notes on the Kx day, and to Adrian for making the annual pilgrimage to Helsinki to report the forest seminar.

Kx Systems Users' Meeting

notes by Stephen Taylor (sjt@lambenttechnology.com)

Kx Systems came to London in February for their annual user meeting. Arthur Whitney gave a brief demonstration of Q, his new and more accessible 'skin' over kdb+ and the K programming language. He started his demonstration by downloading and installing Q from his server in California. This took less than a second.

Various Kx customers spoke, but the star of the show was Mark Sykes from Deutsche Bank Global Trading in London. Sykes is an arbitrage trader, which is to say he looks for mismatches in financial markets from which he can make a risk-free profit. Or if not risk-free, one in which the odds are heavily in his favour.

This is an obvious field for automation. Arbitrage traders have theories about what opportunities look like. But most 'windows of opportunity' slam fast quickly. Despite a generation of traders raised on Playstations, the real action is in getting computers to watch and trade.

Sykes told us when he first came to arbitrage, traders desks sported thick books on Excel and Visual Basic. Now they are more likely to carry books on J2EE, as traders' home-grown programming becomes more sophisticated. Europe, he thinks, is ahead of America in this, but the gap is narrowing.

He plugged in to his trading systems at DB and showed us live market prices and his positions in 10-year Bundesbank interest-rate futures. He described a well-known arbitrage model "Reversion to Mean": when the price moves a standard deviation from its rolling mean, the probability of it reverting to the mean almost immediately is very high.

A chart of tick (trade-by-trade prices) against the standard deviation from the rolling mean demonstrated the truth of this. Every time the price line crossed the deviation line, it jumped back to the mean very shortly afterwards.

If you acted promptly, you could trade at the deviant price and reverse the trade at a near risk-free profit very shortly afterwards. 'Very shortly' is the key here. From the chart we could see that the window of opportunity typically closed in 600-1200 milliseconds.

Sykes explained the rules of the game to us. Reuters has used 200 milliseconds of your window just getting the price to you. You have to recalculate the rolling

mean and the standard deviation, decide whether to trade or not, compose the trading order and get it on the wire to the exchange. This will succeed, but if you are too slow you risk getting your order filled after the market price has reverted to the mean. Speed is everything. He aims to get everything done in under half a second.

The speed advantage of Q exposes second-order opportunities. Q programs get their orders filled first and so scoop the best out of the opportunity. Behind them come Java programs. By the time Excel models have joined in, the price is swinging the other way, past the rolling mean. Canny Q programs might be out there waiting for a second bite!

APLs have long been favoured for financial markets software. A+ (aplusdev.org) was developed on a Manhattan trading floor. The rich set of mathematical primitives and fast array handling are obvious advantages in this business. But equally important is the low volume of APL source code. In fast-moving business environments, no one writes and 'signs off' program specifications.

Coding here is a design activity, a process of thinking through both the problem and its solution. Low code volume keeps the software soft and pliable, a tool for thought. Nowhere was this more evident than in Mark Sykes' display of his own code.

Though he apologised for failing to achieve the brevity of Whitney's own code, we could see on even a casual inspection a high degree of semantic density. That is, a high proportion of the defined tokens denoted entities in the problem domain, such as prices, dates, trades, rather than artefacts of the programming language, such as labels and counters.

Mark Sykes has no time to brief analysts and programmers as his thinking develops. Instead of discussing his problems and their solutions in ambiguous, untestable English, he writes Q, with help from the professional programmers at First Derivatives.

Once again, in one of the most competitive business environments humans have ever devised, an APL is winning by collapsing software development and execution times. Expect to hear more stories like this.

Finnish Forest Seminar IX in Calliola 17th - 18th March 2005

reported by Adrian Smith (adrian@apl385.com)

Acknowledgements

Thanks again to FinnAPL for setting up a winter seminar in wonderful surroundings. This time the snow was crisp, the air was cold and clear, and I got to go ski-ing on the Baltic.



The content was excellent too. *Morten Kromberg* gave two superb talks on SQAPL and the future of databases with ADO.net. I learned a great deal, much of it of likely commercial value. We were also shown a DVD made from a panel discussion at the 1974 APL conference with Ken, Adin, Larry Breed and others, many with hair and beard in classic 1970s 'programmer' style. *Kimmo Linna* gave an extended example of the use of SQAPL running against MySQL in Finnair, and *Kimmo Kekäläinen* another very thorough application demo, again using database access to get at a huge data-warehouse very efficiently.

My talk was more of a workshop - I thought it would be fun to take a random piece of APL from the last Vector and have a go at compiling it. I think I

succeeded in convincing the audience that I am not trying to 'get out of APL' by generating C# from my old RainPro code. The object is to stay with APL as the development tool, but to ship the results as lean, mean, compiled code. With Dyalog 10.2, even APLers might prefer to call it as a .Net library rather than cluttering their application with my namespaces.

On the second day, *Sami Laitinen* showed us a lot of interesting stuff calling Dyalog from Excel (including the use of RainPro, using some tricks which I hadn't seen before), Morten continued with a review of database performance under Dyalog 10 and .net (there are some major bear traps here which can absolutely kill your speed), then we were treated to a major web-application development by *Dinosoft*, and the meeting closed with a review of *Anssi Seppälä's* work with APL and J in modelling power-supply quality in Finland.

Day-1 - DVD on the Origins of APL (1974 panel)

I think everyone found this fascinating, and not just because of the haircuts on display. It sent me off searching for the System/360 specification (the single most important piece of APL ever written?) and also to re-read "A Programming Language" to get right back to the roots of the language.

The initial choice of a fixed workspace size was to facilitate swopping and storage allocation for a simple and reliable timesharing operation. Someone on the panel commented "Today even, people are still building APL systems using these techniques" - this was back in the early 1970s, remember. And here we are running Dyalog APL (or APL2000) under Windows XP which has excellent memory management, and still we see WS FULL messages. Unpicking old design strategies does take a remarkably long time.

I hope the BAA can make this excellent historical document available on the web in some way. I was rather expecting a tacky video with blurred pictures and fuzzy sound, but not at all. This was done in a proper studio on real film, with a professional chairman who moved the discussion along in just the right way. Highly recommended viewing.

Morten Kromberg on SQAPL Version 5

SQAPL is one of those products that just refuses to die. It was devised in the days when getting at databases was really hard, and for a while "middleware" was a killer concept which insulated applications from the nitty-gritty of data storage. SQAPL dates from this time, and much of the original 1983 code is still around. Time has moved on, and for the world of 'real programmers' Microsoft have more

or less killed off the market by providing ODBC, ADO and now ADO.net which all purport to make the bridge between data-provider and application.

So why is SQAPL still crucial to so many major commercial APL systems? Looking at Morten's examples (some of which follow in the report of his second talk) the answer is a mix of 'ease of use' and speed. I suppose inertia has something to do with it too, but if I had got used to the comfort of SQAPL, I would really hate to learn all the new tricks that you need to make ADO work for you as fast and reliably as SQAPL does.

New features in SQAPL version 5

SQAPL has been modified to work well with ODBC 3.51 and now goes directly to the drivers, improving speed and reliability. Errors are better trapped, and the reliance on INI files or registry settings is removed, making installation trivial.

One of the most appreciated updates is the 'columnwise' option - formerly SQAPL would return the result of a query as a matrix, so 10,000 rows of 2 columns (say 'Name' and 'Age') would make a very large nested structure in memory. Now it simply makes a 2-element vector, each part having a uniform 10,000-element array. The only puzzling thing here is that it still insists on padding strings out to the column width, which seems a huge waste of time and space, as I guess the first thing most APLers would do is strip the unwanted blanks! I think a further option may be added very soon to prevent this.

Anyway, it was running at around 160K records per second on Morten's laptop, which feels fast enough for all practical purposes. Part of the speed-up is due to a bug fix - version 3 accidentally limited maxrows to 50 even if you explicitly set a much bigger number. This fix alone could be worth a factor of 10 on a fast machine with plenty of memory. Here are some more highlights:

Many databases support a 'read-only' connection, which is an excellent precaution for analytical software, and often yields another small speed-up.

Exceptions are now trapped, which stops Dyalog dying with Syserror 999. Morten was very clear that the application should NOT trap around these errors - you should always let the application fail, close the database connection and restart from cold.

SQAPL has also added a new Julian date format "J#2" which faithfully reproduces the Excel bug and regards 29th Feb 1900 as a perfectly reasonable date. This brings the retrieved dates into line with dates read (for example) from Excel worksheets.

The syntax for setting up 'bind variables' has been made more programmer-friendly, as you can now construct a 7-column matrix with all the required information and pass this in a single call.

If you pass a list of commands for multiple execution, you can now stop on the first error, rather than running them all and then checking the flags. But ... MySQL ignores this setting so SQAPL can make use of a fallback setting ('Loop' 0) which stops the driver looping and iterates in SQAPL instead. The extra time taken is trivial.

There were plenty of minor improvements, many are really workarounds for the increasingly popular MySQL database where the ODBC drivers are distinctly flaky. All told, this looks like a very welcome step forward, and from the warm applause at the end, Morten (and of course Bjørn Christensen who has done the hard work in C) had answered a good many prayers from existing users.

Adrian Smith on Updates to *RainPro* and *NewLeaf*



This was shoe-horned into the programme, to save time answering lots of questions in the sauna afterwards.

RainPro has gained horizontal boxplots, spanned X-axis labels and infinitely variable tick-mark lengths. Thresholds can be set so that only ticks above a certain length trigger gridlines. This

should make summary timeseries (Years, Quarters, Months) very easy to program and label. Currently, some of this code is only in the SharpPlot workspace, but it will all be back-fitted to RainPro for Dyalog and APL+Win.

NewLeaf has gained the ability to embed TrueType fonts in PDFs. Of course you may not always want to embed a font like Verdana, just have it show up in the document if it is available on the user's machine and fall back nicely if not. Thanks to Joe Blaze (who chased me around Naples until I ran out of pillars to hide behind) I finally faced up to reading the (intentionally obscure) Adobe manuals and spending a few days growling at Acrobat Reader's complete lack of debug help. Anyway, it all works nicely now (see screenshot above) and if anyone really really wants PostScript fonts as well as TrueType, I know how to do that too.

Adrian Smith on APL to C# Conversion

Why has Causeway invested the best part of 2 man years ignoring everything Dyalog are offering and writing their own toolkit to save *RainPro* as a true .Net component (see www.sharpplot.com for the result)? Basically because we want a product that ticks ALL the right Microsoft boxes, and shipping a charting component with a 2Mb 'unmanaged' APL engine stuck to its rear-end makes no sense at all. Could we have simply re-written it in C# in that time? Maybe, but then it would have become detached from its (much valued) APL user-base and would very likely freeze at that point.

Is this technology likely to be useful to anyone else? Well, here are some ideas:

Legacy applications. "Recode or die" is often the message. Now you can at least mechanise (I would never claim automate) the recoding process. Redo the Gui in raw C#, convert the guts of the system from the original APL.

Bottleneck functions. Sometimes you just can't escape iteration. Simulation is typical, genetic algorithms would be another example. Now you can compile these, and (with Dyalog 10.1) just use them as if they were a normal namespace. Factors of around 20 seem readily achievable. Following on from Morten's 2nd talk, I think I should add 'database calls' to this list!

Memory hogs. If you want to stay in a 20M workspace most of the time, but just have to access a huge array once in a while, kick it out into C# and let Windows do the garbage-collection.

All-new APL libraries. Suppose you have a great little utility-set for actuarial work or logistics, or whatever. You are never going to sell this with a runtime Dyalog attached, but if it weighs in at 200K as a single DLL, you have a very good chance. As a bonus, you get all the right XML stuff generated from a few additional APL header comments, so all the hints and tips in Visual Studio work as they should.

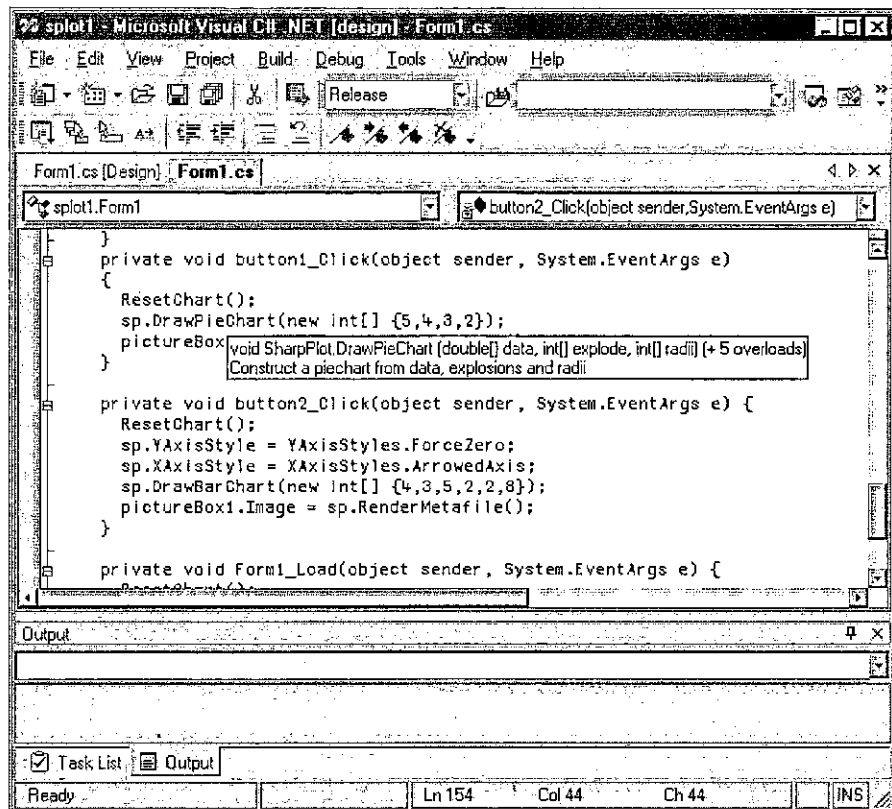
Gui stuff. I have a wild dream that come Longhorn and XAML, I am going to be able to compile *CPro* dialogue definitions and convert whole applications, not just libraries. Please don't place any bets on this!

Ticking all the right boxes

One of the (many) good things about the .Net environment is that most of the tools are either free (like the Microsoft compilers, and the SharpDevelop visual development workbench) or quite affordable (like Microsoft Visual C#). It is also really easy to integrate new components into any of these tools, using an accompanying XML file (with the same name as the DLL).

The examples which follow come from SharpPlot and were automatically generated by the code-conversion process, but there is no reason why you should not hand-author these XML files to accompany libraries created with APLScript and John Daintree's compiler. It is all very straightforward stuff.

Here is a simple example, showing a tooltip which appears in Visual C# when you mouse-over any of the major function names:



This is a trivial Gui application with two buttons to draw a piechart and a barchart. As an aside, this would be a *really* useful addition to Dyalog 10.2. Add a new control-structure such as :Summary and have this show as the tip for any function – much more use than trying to show the whole function in one tip!

Anyway, where did that fragment of informative text come from, and how did it make it from the APL original into the Visual C# documentation? We start with a simple 'special comment' in the original APL:

```
DrawPieChart arg;DATA;data;explode;radii
A Constructs a piechart from an array of data values
AAs Construct a piechart from data, explosions and radii
```

My convention is that the AAs entry offers a simple one-line summary of the function. Many of these have been created from the existing 'first comment' line in the old RainPro code. In addition you can have lots of AAr lines which are 'remarks' and also build into the XML. They are used by tools such as *NDoc* which will give you a first-cut CHM-style help file from the XML output.

The XML file which accompanies the SharpPlot DLL gets a section like:

```
<member name="M:Causeway.SharpPlot.DrawPieChart
(System.Double[],System.Int32[],System.Int32[])">
  <summary>
    Construct a piechart from data, explosions and radii
  </summary>
</member>
```

There is a similar section for each overload (variation in argument type) of each public method in the library. Enumerations (defined lists of allowed values) are documented in just the same way, for example if I wave the mouse over:

```
sp.YAxisStyle = YAxisStyles.ForceZero;
sp.XAxisStyle = XAxisStyles.ArrowedAxis;
```

... then I get a short description telling me what the option means. This comes from another block of XML:

```
<member name="F:Causeway.YAxisStyles.ArrowedAxis">
  <summary>Arrowed axis</summary>
</member>
```

... which doesn't add a lot in this case, as I haven't been through all the styles yet, adding extra text. As far as Dyalog is concerned, an enumeration is simply a namespace containing a bunch of integers:

```
#.YAxisStyles.ArrowedAxis
16
```

Until we have some way of documenting variables (an extension to JAT maybe) I just have a 'special' variable in the namespace called `#.YAxisStyles.Δfields` to give a one-line summary for each variable, and then reformat this as XML.

Code-conversion workshop

We know that the engine can convert 'Adrian code' rather well. Just for a bit of fun, let's take a typical piece of legacy code and have a go at converting that – we will learn more that way. The *Monty Hall* simulator from the most recent Vector makes an excellent starting point, and illustrates very well the sort of problems you are likely to meet. Often, they are not at all what you expect!

Here is the original code – give it a quick look and try to guess where the trip-wires are.

```

▽ CARS←N GOATDOOR C;[]IO;DOORS;PICK;GOAT;NG
[1]  A Let's make a deal! We can choose 1 of 3 doors: 1 hides a car,
[2]  A 2 hide goats. We pick one but Monty gives us a chance to re-choose
[3]  A after he opens one of the remaining 2 doors to reveal a goat.
[4]  A How many cars do we get by
[5]  A C: 0→keeping 1st choice, 1→changing to other door?
[6]  A N: number of trials.
[7]  []IO←1 ◊ CARS←0
[8]
[9]  L_MAKEADEAL:DOORS←(73)0 0 1      A Randomize choices: 1=car
[10] PICK←(73)0 0 1                    A We pick a door: 1 is choice
[11] NG←0+.GOAT←DOORS▽PICK            A Number of unpicked goat doors
[12] GOAT[(~GOAT)/1 2 3][?NG]]+NG#1   A Show a goat at random (=0)
[13] DOORS←GOAT/DOORS ◊ PICK←GOAT/PICK A Eliminate the shown door
[14] PICK←C▽PICK                      A Change pick or don't
[15] CARS←CARS+PICK/DOORS              A Add car if we got it
[16] →(*N←N-1)ρL_MAKEADEAL
▽

```

Here are a few things that the audience suggested as potential problems:

The inner product on line 11

The embedded assignments on lines 11 and 12

The use of labels and the dreaded 'goto' on lines 9 and 16

In fact, these are all quite innocent! To get this into shape as a library call, I need to do some routine housekeeping, then let's just let the converter rip and see what dies! Two additional comment lines are essential:

```

A:Public
A: int N,C,CARS

```

These make sure the function is declared *public* (*private* is always the default) and determine the data-types of the arguments and result. All other data types can be deduced from context. Also I removed the Index-origin setting (our engine is 1-origin without the option) giving:

```

CARS←0

```

Now let's give it a try and see what errors we get back:

```
qq←try 'GOATDOOR'
TYPES - (int[],int[]) not supported by AE.Or at #.GOATDOOR[11]
TYPE CLASH - int[] ≠ bool[] at #.GOATDOOR[14]
TYPE CLASH - int ≠ int[] at #.GOATDOOR[15]
```

Three errors is a pretty good start, so now we can take this apart to see what is causing the trouble. We can have a look at the generated C# source (stored in qq) in a moment. We are clearly trying to do a boolean OR on line 11 between variables DOORS and PICK – but surely these are already logical vectors?

Unfortunately not – as I explained in the last Vector I really cannot assume that a constant vector like 1 0 0 is always boolean. There are two ways to fix this one:

declare another variable such as `bool[] mask` and assign 0 0 1 to this. If I know the type of the name to the left of an assignment, I will do my best to format constants correctly!

use a couple of global variables `true` and `false` in place of 1 and 0

I chose the second approach here, as it preserves more of the spirit of the original, so three lines are modified:

```
L_MAKEADEAL:DOORS+(?3)φfalse false true
PICK+(?3)φfalse false true
NG+false+. =GOAT+DOORS∨PICK
```

The last change is really for style only – there is no problem with the integer scalar and boolean array here!

So what can be wrong with this:

```
CARS←CARS+PICK/DOORS
```

It looks innocent enough, but we APLers have got far too tolerant of singletons behaving like scalars. The one thing you *must not do* when writing code in a conventional language is change the datatype of a local variable. Here, `CARS` is a scalar integer, and the result of the compression is a 1-element array, so adding a scalar to an array yields an array and the assignment to a scalar fails. The solution is obvious:

```
CARS←CARS+↑PICK/DOORS
```

... and now we seem to have a fully functional C# conversion:

```

public int GOATDOOR(int N,int C)
{
    int CARS; // Result
    bool[] DOORS,PICK,GOAT;
    int    NG;

    CARS = 0;
    L_MAKEADEAL: DOORS = AE.Rotate(AE.Roll(3),new bool[] {false,false,true});
    PICK = AE.Rotate(AE.Roll(3),new bool[] {false,false,true});
    NG = PlusDotEQ(false,GOAT = AE.Or(DOORS,PICK));
    GOAT[AE.Compress(AE.Not(GOAT),new int[] {1,2,3})[AE.Roll(NG)-1]-1] = NG != 1;
    DOORS = AE.Compress(GOAT,DOORS);
    PICK = AE.Compress(GOAT,PICK);
    PICK = AE.NE(C,PICK);
    CARS = AE.Plus(CARS,AE.First(AE.Compress(PICK,DOORS)));
    if (AE.Signum(N = N - 1)) goto L_MAKEADEAL;
    return CARS;
}

int PlusDotEQ(bool larg, bool[] rarg) // Inner product
{
    return AE.PlusReduce(AE.EQ(larg,rarg));
}

```

The converter preserves all the trailing comments – I have stripped them here to save space. Notice that all the local variables (DOORS etc.) have been declared for us, and that the inner product has been in-lined as a reduction. The other thing I really like about this conversion strategy is that it preserves the structure of the original, so if you do get an error in the compiled code, the .Net debugger takes you to a line of C# which you can immediately track back to the original line of APL. Anyway, let's add a couple of lines to time it and throw this at the MS compiler and see what happens:

```

□~'Started ',(⌈⌊TS), ' for ',(⌈N), ' iterations'
.....
□~'Ended ',⌈⌊TS

```

```

D:\Tools\aplc>cs bench
bench.cs(120,9): error CS0029: Cannot implicitly convert type 'int' to
'bool'

```

This is complaining about our test on the penultimate line:

```

if (AE.Signum(N = N - 1)) ...

```

Again, APL is being horribly tolerant here, and I would say that this line is really a bug in the original! It would be far better coded as:

```

→(0<N~N-1)ρL_MAKEADEAL

```

... so change it to this and regenerate the C# version:

```
if (0 < (N = N - 1)) goto L_MAKEADEAL;
```

Now it works, so time for a little test:

```
Console.WriteLine("No switch was " + test.GoatDoor(150000,0));
Console.WriteLine("Switched was " + test.GoatDoor(150000,1));

Started 2005 4 24 11 36 56 622 for 150000 iterations
Ended   2005 4 24 11 36 57 643
No switch was 49980
Started 2005 4 24 11 36 57 653 for 150000 iterations
Ended   2005 4 24 11 36 58 635
Switched was 100189
```

The numbers look very plausible, and it seems to be running at somewhere near 150,000 iterations in a second. Just for interest, let's try the converted APL original with the same numbers:

```
150000 GoatDoor 0
Started 2005 4 24 11 41 7 0 for 150000 iterations
Ended   2005 4 24 11 41 30 0
49985

150000 GoatDoor 1
Started 2005 4 24 11 41 35 0 for 150000 iterations
Ended   2005 4 24 11 41 57 0
100231
```

Different answer (just as well, really) at about 23sec for each run. This is probably fairly typical of the speedup you should expect on simple scalar code. For carefully hand-crafted APL code which really exploits arrays, the chances are it will run at almost exactly the same speed when compiled. In fact it could run slower, as I am sure there are places where John Scholes' array primitives are much cleverer than Causeway's!

That hardly does more than scratch the surface of this technology, but maybe it does enough to make you look hard at your own APL code and wonder if there are some benefits in this approach. Questions and discussion can follow over a beer in the usual way.

Day-2: Morten Kromberg on Database Performance

Morten is writing this up for the next Vector, so I will just add a few comments here and leave him to complete it. Here is one good reason for using SQAPL for a few more years:


```

z2[1;4].PropList
  MinValue MaxValue Date Day DayOfWeek DayOfYear Hour Millisecond Minu
te Month Now UtcNow Second Ticks TimeOfDay Today Year
z2[;4].ToOADate
3 4 5 6 7 8 9 10 11 12

```

This is one of those “oh no” moments. Because ADONet is returning dates as C# objects (very sensible if you are a C# or VB programmer) Dyalog is converting each individual date into a namespace. So if you query 10,000 rows from a table which happens to contain dates, you get 10,000 new namespaces. This (a) fills your workspace and (b) takes forever to run.

The solution – I think I will let Morten tell it in his own words in the next unmissable instalment. See Vector 21.3 which will be following along very soon.

Dinosoft Oy and Dyalog APL with .Net

Now these guys are doing something really good with Dyalog APL and webservises. For all I said about delivery, if you are running your APL server-side then none of the issues apply, and the native Dyalog method is fine. They have been having some great fun with the SVG gradients and using RainPro in a server-side analysis tool to make some really pretty graphics for display in the browser. Well, I was impressed!

Summary

This was the 9th Forest Seminar, and FinnAPL are beginning to wonder if it is time for a change. If they choose not to repeat it again next year, at least we went out on a high note. If they do run another one, they will have to work hard to match the venue and the weather.



Ski trails into the Sunset (Erkki Juvonen)

Bayesian Financial Dynamic Linear Modelling in APL

by Devon McCormick (devon@acm.org)

An earlier version of this article was presented as a paper at the tenth "APL as a Tool of Thought" conference held in conjunction with APL98.

Bayesian statistics is a brand-new idea that's only about 235 years old. The paper that was to immortalize the last name of the Reverend Thomas Bayes, a Nonconformist minister born in 1702, was published in 1763. Unfortunately for any fame he may have hoped to gain from what proved to be his most influential work, Bayes died some 3 years before.

From such inauspicious origins, his contribution to statistics continued inauspiciously. His ideas flourished briefly in the latter part of the 18th century, being taken up by the great mathematician Laplace, before languishing in relative obscurity until this century. Even though revived in the early part of the century, first by Ramsey, then by de Finetti, his ideas only became widespread in recent years.

So, what is this great idea that has come to us through centuries? At first glance, it may appear to be rather trivial. So trivial, in fact, we may briefly derive it here without further exposition.

Bayes examined the problem of contingent probability. We may start, simply enough, by noting that the probability of an event P , denoted here as $\text{Prob}(P)$, contingent on circumstances (or Data) D , is the product of $\text{Prob}(P)$ and $\text{Prob}(D | P)$. This latter formulation may be read "the probability of D given P ". So, since

$$\text{Prob}(PD) = \text{Prob}(P) \times \text{Prob}(D | P)$$

and, the reverse being true and equivalent, that is

$$\text{Prob}(DP) = \text{Prob}(D) \times \text{Prob}(P | D)$$

Taking these two as equal to each other leads us to the common formulation of Bayes's Theorem:

$$\text{Prob}(P | D) = \frac{\text{Prob}(D | P) \times \text{Prob}(P)}{\text{Prob}(D)}$$

There might not seem to be much here, so why should this simple theorem have provoked so much contention and fallen into such conspicuous disfavour? The key here is the interpretation of the prior Data: it includes any information we see fit to include: this makes it "subjective", hence often thought to be not fit for proper scientific and mathematical treatment. That is, the powerful effect of Bayes's Theorem is to allow us to incorporate non-traditional, perhaps inexact, data into our probability formulations.

This denigration of subjectivity, in a historical context of pride in exact, "hard" science, continues to plague Bayesian techniques even up until recent times. In fact, one reference (in Kuhn's "Readings in Game Theory") to Bayesian techniques appears to justify calling it Bayesian only because of the general use of subjective probability estimations. One much-cited early work about Bayesian techniques is called "Studies in Subjective Probability" (Kyburg, 1964): at the time it was published, this was the defining feature of Bayesian statistics.

However, as we shall see, this perceived subjectivity of the technique is misconstrued as a weakness, when it is rather a strength because it acknowledges and deals with the "outside" information underlying all statistical inference. This combining of "hard" and "soft" data is helpful in the context of the Dynamic Linear Models we will explore later on.

Another strength of Bayesian inference has to do with its approach to probability, an approach often called "the inverse probability problem" because it is backwards in relation to traditional probability. This problem is stated thusly:

Given the number of times an event with unknown likelihood has occurred or failed to occur, what is the chance that the probability of it happening in a single trial lies somewhere between 2 degrees of probability?

For example, one of the Bayesians' favourite examples of this is the urn problem: given an urn containing 2 colours of balls, say black and white, and a series of tests whereby we draw a ball, note its colour and return it to the urn, what is the likelihood of various combinations of the 2 colours? For instance, given an urn with 5 balls and the evidence that 3 draws with replacement yield only white balls, what is the probability that all 5 are white?

This is the inverse of a typical problem in probability that would be more along the lines of: given an urn with 3 white and 2 black balls, what are the odds of drawing 3 white balls, 1 at a time with replacement? Before we delve further into the Bayesian example, notice how more commonly applicable to typical finance problems is this former sort of formulation than is the latter. A classical probability problem would be something like: given the mean returns and

volatility of some assets, how are they likely to perform? However, the inverse problem would be something like: given the performance of these assets, what are their likely returns and volatilities? We can see from this that the inverse problem better matches what we often have to work with in terms of real data.

Before we look more at the urn problem, we need to extend Bayes's Theorem to multiple events. Given a series of k events P_i with associated likelihoods $\text{Prob}(P_i)$ and prior observed data D , the contingent probability of a particular event P_i given observations D is stated thusly:

$$\text{Prob}(P_i|D) = \frac{\text{Prob}(P_i) \times \text{Prob}(D|P_i)}{(\text{Prob}(D|P_1) \times \text{Prob}(P_1)) + \dots (\text{Prob}(D|P_k) \times \text{Prob}(P_k))}$$

In APL, this is shown by the function `BayesPP`.

```

▽ R-BayesPP PROBS
[1]  A Calc Bayesian posterior probability given PROBS: 2 row mat:
[2]  A [0;]P(P1), P(P2)...; [1;]P(P1|D), P(P2|D)...; i.e. [0;] isolated
[3]  A event probability, [1;] conditional probability given data D.
[4]  R←R÷+/R←×/PROBS
▽

```

In practice, this works as follows. Suppose we have an urn with 5 black and white balls in unknown proportion. We draw 3, one at a time with replacement; all are white. What is the probability that all 5 are white? Invoking the APL function

```
UrnProbAllWhite 5 3
```

gives us the numerical answer of 0.15625. How do we work this out using the above theorem? We must solve for the probability of all the balls being white given the evidence of 3 white draws ($\text{Prob}(P_i|D)$). The text of this function and a discussion follows.

```

▽ P-UrnProbAllWhite NNW;IO
[1]  A Bayesian urn problem: given NNW[0] (ostensibly black and white) balls
[2]  A and experimental evidence that NNW[1] white ones picked, with
[3]  A replacement each time, what is probability that all are white?
[4]  IO←0
[5]  P←((1+NNW[0])÷NNW[0])*NNW[1]      A 0 to N possible balls of color
[6]  A picked + chance of picking NW white ones for each proportion possible.
[7]  P←+/P×(BinomialCoeff NNW[0])÷2*NNW[0] A 2* because 2 possible colors;
[8]  A would have to use other than binomial expansion if more than 2.
[9]  A P is now all probabilities of all combinations. Applying Bayes's
[10] A theorem: (Prob(all selections white given all balls in urn are white ×
[11] A Prob(all balls in urn are white)) ÷ Sum of Prob(all combos)
[12] P←((1÷2pNNW[0])*3)×÷2*NNW[0])÷P
▽

```

The subfunction `BinomialCoeff` is defined thusly:

```

▽ R←BinomialCoeff N;IO
[1]  A Give coefficients of Nth binomial expansion.
[2]  IO←0 ⋄ R←((ιN)!)N,1
▽

```

Working 1st on the divisor of the preceding equation, calculated in line 5 of the APL function, we calculate the probability of the datum D (3 white samples) for each of the possible P_i s. These latter range from no white balls (all black) to 5 white balls, or the fractions 0/5, 1/5, 2/5, 3/5, 4/5, and 5/5. These are all the $\text{Prob}(P_i)$ s.

Line 7 of the APL function figures the likelihood of 3 white draws given each of the possible combinations of white and black balls. The denominator of each of these is 2^5 or 32 because this is the number of possible combinations of 2 things taken 5 at a time. The numerator is the possible arrangements of each combination, e.g. 1, 5, 10, 10, 5, and 1 corresponding to: 1 way to have all 5 black, 5 ways to have 1 white and 4 black, 10 ways to have 2 white and 3 black, 10 ways to have 3 white and 2 black, 5 ways to have 4 white and 1 black, and 1 way to have all white. Combining all the $\text{Prob}(P_i)$ s with the $\text{Prob}(D|P_i)$ s, multiplying them together and summing the results gives us (in maths):

$$\left[\left(\frac{0}{5}\right)^3 \times \left(\frac{1}{32}\right)\right] + \left[\left(\frac{1}{5}\right)^3 \times \left(\frac{5}{32}\right)\right] + \left[\left(\frac{2}{5}\right)^3 \times \left(\frac{10}{32}\right)\right] + \left[\left(\frac{3}{5}\right)^3 \times \left(\frac{10}{32}\right)\right] + \left[\left(\frac{4}{5}\right)^3 \times \left(\frac{5}{32}\right)\right] + \left[\left(\frac{5}{5}\right)^3 \times \left(\frac{1}{32}\right)\right]$$

or (in APL, origin 0):

```
((((ι6)+5)*3)+.×1 5 10 10 5 1÷32
```

which equals 0.2. The difference between these 2 expressions prompts me to question the reader: which of these 2 expressions looks simpler? Which do you think took about 5 minutes to enter and which took about 5 seconds?

The last line of the APL function calculates the numerator in Bayes's Theorem above, which is the probability of 3 white draws given all 5 balls being white, and divides it by the denominator to give our answer. A more general version of this is the APL function `UrnProblem` that returns all probabilities instead of just the "all-white" one. An even more interesting function would allow a third input of the number of black balls in a sample instead of just restricting the possible data to observations of all-white draws. This is left as an exercise for the reader. The text of `UrnProblem` follows.

```

▽ P=UrnProblem NNW;IO;NB;NWhite;Pk;Num;Denom
[1]  A Bayesian urn problem: given NB (ostensibly black and white) balls
[2]  A and experimental evidence that NWhite white ones picked, with
[3]  a replacement each time, what are probabilities that 0 to NB are white?
[4]  IO=0 0 NB NWhite=NNW
[5]  Pk=((1/NB+1)+NB)*NWhite           A 0 to N possible balls of color
[6]  A picked + chance of picking NW white ones for each proportion possible.
[7]  Denom+=Num+Pk*(BinomialCoeff NB):2*NB A 2* because 2 possible colors;
[8]  A binomial expansion assumes prior normal distribution for 2 colors.
[9]  A Apply Bayes's theorem:
[10] P=Num+Denom
▽

```

One point in the above exposition that deserves further mention is our gloss on the use of binomial coefficients (line 7 in `UrnProblem`). This introduces a primary facet of Bayesian inference: the use of a prior distribution. By using the binomial coefficients to weight the black and white combinations, we are in fact assuming a prior distribution that approximates the normal distribution (for discrete values). The effect of this prior distribution may be seen in the values generated by

```

UrnProblem 6 1
0 0.03125 0.15625 0.3125 0.3125 0.15625 0.03125

```

versus the result of

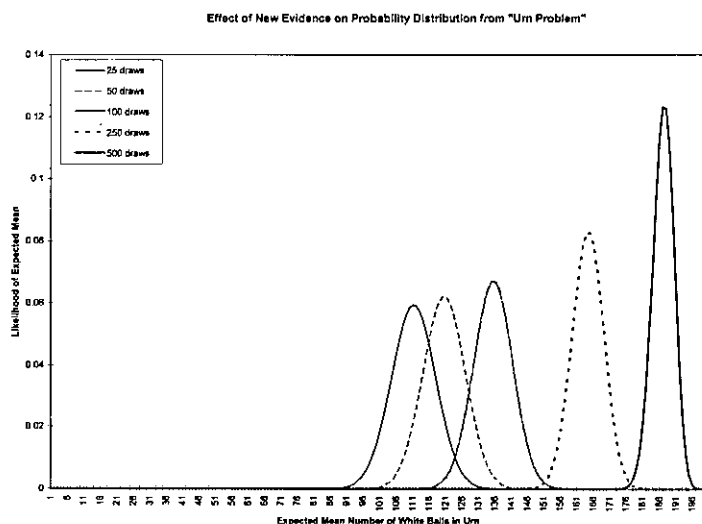
```

UrnProblem 6 6
0 0.0000275999 0.00441599 0.0670678 0.282623 0.431249 0.214617

```

Notice how, in the first case (for 1 draw from an urn containing 6 balls), the peak values are the 2 in the centre whereas, in the second case, the peak has moved to the second-to-last value. To better see this, look at the "Evidence Effect" graph: this shows the results of `UrnProblem` run for an urn containing 200 balls and 25, 50, 100, 250 and 500 draws of white balls.

Notice how the distribution shifts in the manner hinted at by our small example above. What does this mean?



In the first case, our observation of 25 white draws for 200 balls, combined with our prior (assumed) normal distribution leads us to put the most likely combinations near the centre. This shows the strong influence of the prior distribution with little additional evidence. The subsequent cases, each with more white draws, move the peak toward the all-white end of likely distributions. Of course, evidence like this might lead us to question our choice of prior distribution. In any case, we can see how this Bayesian technique allows us to combine our estimate of a distribution with subsequent evidence about that distribution.

While on the subject of prior distributions, it's worth noting that the traditional statistics, from which the "subjectivity" of Bayesian statistics has been attacked, is in fact explainable in Bayesian terms as one with a "flat" prior. That is, if we decide to study the frequency of occurrence of heads and tails when flipping a "fair" coin, we can either assume, as traditionalists do, that we have no information about the likely outcomes (that is, all probabilities are equal, hence the prior distribution is a straight line), or we might acknowledge that this assumption is just one of many possible prior distributions. Under a strict empiricist assumption, after, say 5, trials of flipping a coin where only heads came up, we might say that the likelihood of heads is 100%.

However, the more interesting and practical implication of the Bayesian method is that it inverts traditional questions. Whereas a traditionalist might assume a fair

coin and tell you that the likelihood of 5 flips all being heads is 1 in 32, a Bayesian would ask, "If a coin turns up heads for all of 5 flips, what is the likelihood that it is a fair coin?" That is, we are able to question the prior assumption itself. The problem of "A Lady Tasting Tea", from the book of the same name, provides a delightful illustration of the importance of prior information even that which is too general to quantify.

This illustration is due to the statistician L. J. Savage. Consider the following three cases. Say you have a musician friend who claims he can distinguish a musical score by Mozart from that of another composer. You test this claim and your friend passes the test 10 times out of 10. Say you have another friend, a lady who, when she tastes a cup of tea with milk, claims she can tell whether the tea was added first or the milk was added first. You test her and she passes 10 times out of 10. Now, you have a third, drunken, friend who claims to be able to predict the flip of a fair coin: you test him and he also passes 10 out of 10 times. Why would we consider these three claims to have different likelihoods when each of them has equal evidential support?

It must be that we consider data other than that uncovered by our testing: we use our prior knowledge of the world.

This influence of the prior distribution suggests a further modification to the `UrnProblem` code: we could supply a prior distribution function as one of its arguments. This change, along with the one suggested earlier that allows for more generality in observations, would give us a version of the function that starts to appear to have more practical application than the small samples of Bayesian inference we've seen so far. However, we will leave this pursuit to interested readers. Instead, we will now look at the other important technique that, combined with Bayesian inference, can give us a robust financial forecasting system: Dynamic Linear Models.

Dynamic Linear Models

The following pair of equations characterizes a dynamic linear model:

Type	Equation	Distribution Mean, Variance
Observation	$Y_t = F_t' \theta_t + v_t$	$v_t \sim N[0, V_t]$
System	$\theta_t = G\theta_{t-1} + w_t$	$w_t \sim N[0, W_t]$

The notation in the column labelled "Distribution Mean, Variance" gives the distribution of the error term for each equation; the expression $N[m, v]$ specifies a normal distribution with mean "m" and variance "v".

The variables in these equations have the following meanings:

Variable	Description
Y_t	vector of dependent variables (e.g. asset returns)
F_t'	(transposed) matrix of explanatory factors
θ_t	regression factors
v_t	observation errors
G	state evolution matrix
θ_{t-1}	previous time's θ
w_t	system errors

The ultimate aim of a Dynamic Linear Model (DLM) is to estimate the Y values in terms of a likely mean and variance. We may think of the equation that purports to explain these Y values as a linear equation with variable coefficients applied to time-varying explanatory factors. For instance, a model of bond prices might state the observation equation using factors known to influence bond prices, such as government interest rates and the spreads of corporate rates.

The system equation may be thought of as a system of relations between the observed factors. However, the dynamism of the model derives from assuming that these relations vary.

Bayesian Forecasting with Dynamic Models

The preceding system of equations is updated by successively examining predictive factors with their corresponding returns, predicting the mean and variance of expected returns based on the next set of factors, and modifying these predictions according to the actual returns. We can see how this is analogous to our earlier, simpler problem of estimating the unknown composition of balls in an urn based on the evidence of samples from the urn. In the case of the urn, we saw how the probability distribution moved from the initial assumption of normally distributed with an equal mix of black and white to a higher likelihood of all white balls based on the evidence of repeated samples of white balls. Similarly, our forecasting model starts with assumptions about means and variances of returns then modifies these assumptions based on successive samples of correlations between econometric factors and asset returns.

The West and Harrison book (*Bayesian Forecasting and Dynamic Linear Models*, Springer-Verlag, 1997) provides a series of increasingly complex sets of data to model in chapter 3. Three of these examples are implemented in APL in the functions `Theorem3p1`, `Theorem3p1_1`, and `Theorem3p1_2`. The associated data sets are contained in the variables `TBL3P1`, `TBL3P2`, and `TBL3P3`, respectively. These may be of interest for use with the book. However, we will look more at a multivariate version of the model that more closely parallels, though it is somewhat simplified, the one in use at my company for asset allocation.

The model on which we will concentrate is the one embodied in the function `MultivDLM3`. Before we look at this, though, a word about the data used in this example. To avoid problems with proprietary or purchased data, we'll be using generated datasets. These are outputs from the `GENFACRETS` function. Hence, a brief look at this function is in order.

```

▽ FR←GENFACRETS NUM;ΩIO;MAX;FACS;RETS
[1]  A Generate a set of factors and returns associated with them.
[2]  ΩIO←0 ∘ FR←(0 4p0)(0p0) ∘ MAX←1E9
[3]  A FACS←--\1 2 1 2o1 2 3 4+5 6 7 8
[4]  FACS←(5+74p100)|1+74p1E9
[5]  RETS←-/FACS
[6]
[7]  L_NEXT:FACS←1 2 1 2o0.5 1 1.5 2×FACS+2 3 4 5
[8]  FACS←FACS+0.01×+/-5+?((pFACS),20)p11 A Handful of little normal noise
[9]  RETS←-/FACS
[10] RETS←RETS+0.01×+/-5+?((pRETS),20)p11 A Handful of little normal noise
[11] →(NUM>1pp=FR+FR,[0]"FACS RETS)pL_NEXT
▽

```

Notice how the factors are generated using a "hidden" function and the returns are based on these factors. A typical use of this function might look like this:

Factors Responses+GENFACRETS 120

to assign the 2 vectors Factors and Responses, each with 120 elements as specified by the function's right argument.

The accompanying illustrations of forecasts compared to "actual" data use these generated data as inputs to the multivariate dynamic linear model. Looking now at the primary instance of this model, the APL function MultiVDLM3, there are a few important points to note. A full explanation of this model is beyond the scope of this paper; however, one of the books by West and Harrison or Pole, West, and Harrison, should provide sufficient detail (see the bibliography). Bearing this in mind, let us examine the few points on which we will concentrate to illuminate the robustness of this type of model. The text of the function follows.

```

V DMO=MultiVDLM3 FGVWYMC;IO;Wt;Vt;MC;SZ;CT;mp;Cp;F;Y;G;Ft;Yt;Rt;ft;Qt;
  At;at;et;mt;Ct;DELTA;nt;np;St;Sp;TIT
[1] R Multivariate Dynamic Linear Model, adapted from West and Harrison.
[2] R FGVWYMC: [0] Factors; [1] G (evolution matrix); [2] V (observation
[3] R variance); [3] W (system variance); [4] responses (Y values to
[4] R predict); [5] MC: prior mean and variance; [6] (optional) delta
[5] R (discount factor). E.G. for 3 factors predicting 1 return:
[6] R MultiVDLM3 (10 3p30) (3 3p4+1) (30 1p2x130) (1 1p1) (3 3p0.01)
[7] R (15 9) (0.6)
[8] R OR DMO=MultiVDLM3 F ((2p-1pF)p(1+1pF)+1) (,COVM R) (COVM F)
[9] R (((pR),1pR) ((AVG R) (ASTODEV R)) (1)
[10] R =DMO[1;]: [0] Forecast mean and [1] variance, [2] posterior system
[11] R (theta) mean and [4] variance, [5] adaptive coefficient, [6] error.
[12] R [7] Scaling factor.
[13]
[14] CT=IO=0 o TIT+ft 'Qt' 'mt' 'Ct' 'At' 'et' 'St'
[15] F G Vt Wt Y MC DELTA=7+FGVWYMC,1 o SZ=1ppF o DELTA+DELTA*0.5
[16] DMO=(pTIT)p=0 1p0 o np=Sp+1
[17] mp Cp+(cpWt)p'MC A Prior mean and variance: coerce shapes if scalar
[18] at=mp
[19]
[20] L_DO1:Ft+qF[CT;] o Yt+Y[CT;] R 1 row mats for conformability
[21] at=G+.xmp
[22] Wt=(DELTA*G+.xCp+.xqG*DELTA)-G+.xCp+.xqG A Discounting
[23] Rt=(G+.xCp+.xqG)+Wt A Prior at time t: theta variance
[24] ft=(qFt)+.xat A 1-step forecast +means
[25] Qt=Sp+(qFt)+.xRt+.xFt A and +variance for forecast Y
[26] et=Yt-ft A 1-step error forecast
[27] nt=np+1 A Start on posteriors
[28] At=Rt+.xFt+.xqQt A Adaptive coefficients,
[29] mt=at+At+.xet A Posterior thetas' means
[30] Ct=Rt-(qAt)+.xAt+.xQt A and (scale-free) variances
[31] St='pSp+((qet)+.x(qQt)+.xet)-1)*Sp+nt A Scaling factor (p. 110 (d))
[32] Ct=(St+Sp)*Ct
[33] DMO=DMO,[0]'=ft Qt mt Ct(qAt)et St
[34] Cp mp Sp np+Ct mt St nt A Current to previous for next loop
[35] -(SZ>CT+CT+1)pL_DO1 o DMO+TIT,[0.5]DMO

```

V

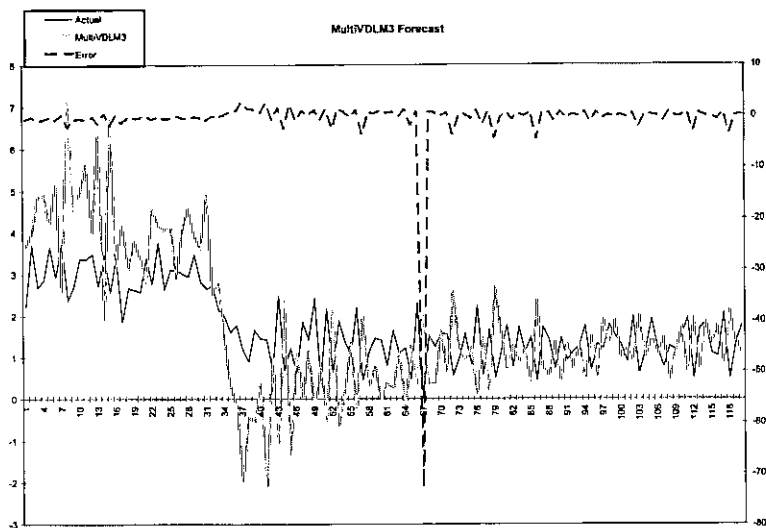
First, let's dispose of variables set to rather trivial values for this example: G and Δ . We set G to be the identity matrix: this corresponds to a random system evolution. We set Δ to be 1; this might have a value slightly less than one to more greatly discount the effects of data, which is further in the past. In addition to these 2 settings, made uninteresting for simplicity, we also take a simple default on the prior mean and variance setting (FGVWYMC [5] in the function) by using the actual mean and variance of the return series. Disposing of all these leaves us to consider only the factors and returns, which we discussed above, and the 2 variances W and V , for the system and observation series.

How to deal with these 2 inputs remains a subject of some research. We'll look mainly at V , the observation variance, to illustrate the robustness of DLMS and how well the Bayesian framework allows us to incorporate new prior information.

Many of the models (in West and Harrison, 1997) assume that this observation variance is known. In fact, it is tempting to use the same shortcut we used for the prior mean and variance values by essentially "looking into the future" and basing the value on our present data. However, this detracts from the ability of the model to adapt to new information by reinforcing the values of past data. However, notice that this is essentially what we did in our example just to get some crude numbers to look at. The important thing to notice is that the system of equations given above specifies V as varying according to time t but our APL implementation treats it as a known constant.

This limitation of our sample implementation detracts from its forecasting power. A better model would attempt to forecast the variance series as it does the return series and progressively correct these forecasts with the receipt of new data. Indeed, the flexibility of this sort of model allows us to do even better than this by incorporating anticipated future data. Since we do not have such an implementation in our simple example, let's see how and why we might do this. First, however, let's look at an example of the function's forecast on some data "made up" by the GENFACTS function as shown on page 73.

These results are shown graphically in the following chart.



Notice how the earlier attempts at forecasting, on the left, diverge widely because the model hasn't "learned" much about the relation between the factors and their associated responses. However, it does track the change in level starting at about points 31 to 43, even though it initially overshoots. Suppose that this change in level were in response to events we could anticipate to some extent: we know that the fundamentals affecting our responses are about to change but we can't quantify them exactly.

Say, for example, that one of the assets whose return and variance we are forecasting is the Hong Kong dollar. Since the value of the Hong Kong dollar has been fixed to that of the U.S. dollar for many years, our model will build a misleading picture of this currency's behaviour based on historical data. This is a very pertinent issue at the time I write this, since that colony has recently reverted to Chinese rule, Asian markets in general have become more volatile of late, and there are indications that the end of this fixed rate regime is close at hand. According to an article in Barron's (the column "The Striking Price", November 17, 1997), the American stock exchange "plans to revise its Hong Kong Option Index so that its value is calculated using a floating rate of exchange for the Hong Kong dollar rather than the fixed value that's now used."

These factors make it very likely that a Hong Kong dollar model will change behaviour drastically in the near future. In addition, this sort of deliberate change will usually be announced in advance, thus aiding the modeller using Bayesian DLMs. A more complete model than the one presented here would use a true time-varying variance and would implement it in such a way as to incorporate an external multiplier. Though one might hazard a guess as to the direction of the Hong Kong dollar when it begins to float, one is not obligated to.

Though a fuller system than the one outlined here might also accommodate external inputs to the calculations of means as well as variances, this would require a view on the market that we might not be prepared to risk. However, it does not require much of a leap of faith to predict that the volatility of the Hong Kong currency will increase relative to the U.S. dollar some time soon.

A Bayesian DLM naturally and easily incorporates such new information, even when it is rather vague and subjective as such advance information often is. We are required to assign some value to our variance multiplier, but, in doing so, we can use whatever other information we see fit (and in this example there is a strong argument for a certain lower limit.) This ability to seamlessly incorporate judgment based on new developments makes these kinds of models very attractive, especially since it minimizes or eliminates re-programming and allows vague information to be introduced in a well-controlled manner.

This document is for informational purposes only. Opinions expressed are our present opinions only, and are subject to change. In preparing this presentation, we have obtained information from sources we believe to be reliable, but do not offer any guarantees as to its accuracy or completeness.

How Wrong Was I?

Sylvia Camacho (sylvia@blueyonder.co.uk)

Dedicated to the memory of Gérard Langlet who might not have agreed with my arguments but would have greatly enjoyed the fight.

Being a third attempt to understand Bell's Inequality [0]

Other than the website [1] which Stefano Lanzavecchia was kind enough to tell me about when he published *Can Anyone Tell Me Where I'm Wrong* in October 2001, no one offered to enlighten me until Graeme Robertson sent me a letter and a book [2] in August 2003. Our subsequent exchange of e-mails provoked me to examine my perplexities yet again and energetic web-surfing brought to light some intriguing recent papers which encourage me to pursue the analysis.

I first became interested in this field when Karl Popper [3] directed my attention to a completely outrageous quotation from Werner Heisenberg who is considered one of the founders of quantum theory. He is describing an experiment with a semi-transparent mirror and assumes that the probability that light will be reflected by it is $\frac{1}{2}$. Thus the probability that light will pass through will also be $\frac{1}{2}$, and so, if the event 'reflection' is a , and the experimental arrangement b , then $p(a,b) = \frac{1}{2} = p(-a,b)$ where ' $-a$ ' stands for the event 'passing through' or 'transmitted'. Now supposing the experiment is carried out with one single photon then this 'probability wave packet', as he calls it, standing for the photon will be split yielding two 'wave packets', $p(a,b)$ and $p(-a,b)$, for which the equation $p(a,b) = \frac{1}{2} = p(-a,b)$ still holds. Heisenberg writes:

By the experiment [i.e. the measurement by which we find the reflected photon], a kind of physical action (a reduction of wave packets) is exerted from the place where the reflected half of the wave packet is found upon another place - as distant as we choose - where the other half of the packet just happens to be ... this physical action is one which spreads with super-luminal velocity. [4]

He draws this conclusion because his photon $p(a,b) = \frac{1}{2} = p(-a,b)$ has 'collapsed' into two photons for which $p(a,b) = 1$ holds here and $p(-a,b) = 0$ holds there. So if my eye is in line to receive a reflected photon it will collapse on my retina, (becoming $p(a,b) = 1$) and 'exert a kind of physical action' upon your eye should that be in line to receive the transmitted ($p(-a,b) = 0$) photon. But why should the action be this way round? Can the transmitted photon not collapse the wave function in your eye and 'exert a kind of physical action' upon my eye, so causing

me to observe the reflected photon? This would be acceptable in probability calculus, where a null observation can certainly change the odds, but how does your eye know when it has gambled and lost? I apologise for the frivolity of this last question but am delighted (with thanks to Graeme for the reference) to find my irritation shared by a real mathematician who says:

... it is not at all unusual, when it comes to quantum philosophy, to find the very best physicists and mathematicians making sharp emphatic claims, almost of a mathematical character, that are trivially false and profoundly ignorant. [5]

This tendency to objectify mathematical expressions may not be confined to quantum theorists but it is certainly very commonly found in their field. It was familiar to Richard Feynman who realised that his students were in danger of confusing the notation they used to quantify probabilities connected with the particles they were modelling with the particles themselves. He had devised a diagrammatic notation involving arrows of differing lengths, representing amplitudes, which are rotated according to the relationship between distance and wavelength and represent possible interactions between photons and test apparatus. He draws such a diagram and says:

In this example, arrows were multiplied and then added to produce a final arrow (the amplitude for the event), whose square is the probability of the event. It is to be emphasised that no matter how many arrows we draw, add or multiply, our objective is to calculate a *single final arrow for the event*. Mistakes are often made by physics students at first because they do not keep this important point in mind. They work for so long analyzing events involving a single photon that they begin to think the arrow is somehow associated with the photon. But these arrows are probability amplitudes, that give, when squared, the *probability* of a complete event.

Feynman adds a footnote:

Keeping this principle in mind should help the student avoid being confused by things such as the "reduction of the wave packet" and similar magic. [6]

Feynman is explaining to his audience what experiment has established about partial reflection by glass. First is that the percentage reflected is affected by the thickness of the glass and the colour of the light. For red light with very thin glass or glass of a particular thickness no light is reflected but otherwise the reflection varies between 4% and 16% as the thickness is increased. He emphasises that the photons are interacting with electrons within the glass although the word used to describe the effect is 'reflection'. The relationship between the wavelength of the light and the thickness of the glass is the critical factor although the clicking photomultiplier is apparently detecting photons and the result will be expressed

as a simple percentage. Elsewhere Feynman describes the increased diffraction seen when light is directed through a narrow gap between blocks as follows:

So when you try to squeeze the light too much to make sure it's going only in a straight line, it refuses to cooperate and begins to spread out.

and then he adds a footnote:

This is an example of the "uncertainty principle": there is a kind of "complementarity" between knowledge of where the light goes between the blocks and where it goes afterwards – precise knowledge of both is impossible. I would like to put the uncertainty principle in its historical place: . . . If you get rid of all the old-fashioned ideas and instead use the ideas that I am explaining in these lectures – adding arrows for all the ways an event can happen – there is no need for an uncertainty principle! [7]

Unfortunately despite Feynman's robust realism Heisenberg's malign influence is still with us.

Reading the explanation of an implementation of Bell's thought experiment in reference [1] it is not clear to me what outcome would be expected from what is described as the 'deterministic point of view'. The statistical argument analysed suggests that there will be no correlation but the inequality, if satisfied, is compatible with a particular linear correlation and this is what is illustrated in some other texts. This is perhaps because there is experimental justification for correlation = 1.0 for identical detector settings and for correlation = 0.5 (i.e. no correlation) at settings differing by 45 degrees. Bell's 'inequality' is an algebraic expression which cross-correlates the proportions of matching pairs over experiments run with different detector orientations. Bell says that this will decide for or against Einstein's claim that Bohr's quantum theory is 'incomplete'. The two postulated outcomes, where y is the fraction of matched pairs and x lies between zero and 90 degrees, are:

$$y=1 - \Phi + 90 \text{ or } y=\cos^2 \Phi, \text{ where } \Phi \text{ is the relative angle between the detectors.}$$

Both cases could be expressed in words by: 'as the angle between the detectors increases the match between the outcomes decreases.' The second outcome looks more plausible because it has family resemblances with formulae describing diffraction and interference and other such trademarks of light waves. It is reported that this is what is found when Bell's test is realised. So I looked again at Stef's reference [1] and this is what I wrote in an e-mail to Graeme:

I gather that the proof offered by the experimental design is in the form of a *reductio ad absurdum*. It is assumed that if the equality is violated by the measured correlations then 'the hidden variable theory' will have been shown to be untenable and this will constitute a proof of the quantum alternative. This form of proof is completely persuasive when used, for instance, to prove the unique factorisation theorem but it depends upon there being exclusive alternatives: either this theorem is true or it is not; assume it is not and show that this has absurd consequences. In the Bell case I do not see what these exclusive alternatives are. I do not see what part 'hidden variables' might play and why it is so important to exclude them and, for that matter, how the design does exclude such elusive factors. Specifically I do not understand why it should be surprising to find that correlations between tests on wave type phenomena can be expressed in terms of sines and cosines. [8]

Pursuing a web reference I found I was not alone in my opinion: [9 10]

First of all we point out that there is a subtle point in the EPR reasoning that, as far as we know, has been overlooked by everybody: "The Einstein Podolsky Rosen reasoning is a reasoning ex absurdum". What Einstein Podolsky and Rosen prove is that 'if quantum mechanics is a correct and complete theory' then 'it is an incomplete theory'. This is so because in their reasoning they use quantum mechanics to describe the situation of the two separated quantum entities, entity 1 and entity 2, flying apart after having interacted. In using this quantum description as part of their reasoning, they explicitly use the fact that quantum mechanics is correct and complete in the sense that it describes well the situation. [10]

In other words they 'beg the question' which is the essence of the ad absurdum technique but it must be used with a care for its consequences:

Now of course, even if the reasoning is a reasoning ex absurdum, it is possible to draw a conclusion when a contradiction is reached. The conclusion is that one of the premises of the reasoning is false. So from the proof of the implication ('quantum mechanics is correct and complete' implies 'quantum mechanics is incomplete') can be concluded that ('quantum mechanics is incorrect' or 'quantum mechanics is incomplete'). [10]

So if we hold to the hypothesis that quantum mechanics is a correct theory, the EPR reasoning proves that quantum mechanics is incomplete:

But the status of this proof is the one of a proof *ex absurdum*. That makes a big difference. We know for example very well that the steps encountered during such a proof *ex absurdum* have no truth value. This means that one of the intermediate steps in the EPR reasoning, namely that quantum mechanics has to be supplemented by additional variables, has no truth value at all. So it is incorrect to say that quantum mechanics is incomplete and has to be supplemented with additional variables to solve this incompleteness. The EPR reasoning does not give us any hint at all to what would be the nature of the incompleteness that it reveals. [10]

Or, as my tutor in logic taught me, 'any proposition whatsoever is implied by a contradiction'.

This is all good knockabout fun but we should remember that Bell's thought experiment has been converted into a real experiment, with the expenditure of considerable trouble and expense which should not go to waste. There are several such experimental realisations, some more and some less close to Bell's proposals and all of which have been attacked on various grounds connected with their design or the reliability of their results [11,12]. I am no physicist and not qualified to judge them on these grounds so I propose to accept that they do as they claim to do and that the predictions of the quantum formalism are confirmed.

So what was the Null hypothesis? None is mentioned but we can speculate that it should be that 'pairs of non-entangled photons tested by the same apparatus will give significantly different results'. Or as Aerts expresses it:

The experiments have shown that in this typical situation of quantum entities having interacted and then flying apart it is 'possible', making a big effort, to have the quantum entities flying apart while still remaining non-separated entities. This quantum effect has meanwhile been called nonlocality. So the experiments prove that quantum nonlocality (or entanglement) can be retained for quantum entities being distances apart that are of a macroscopic nature, and were(sic) 'at first sight' one would not expect this to happen. . . . But it has very little to do with the EPR reasoning. Einstein, Podolsky and Rosen, to be able to make their reasoning, need the situation of quantum entities that fly apart, and do get separated after a while. . . . They have never been made consciously, because the experimentators involved did not understand that these would be the situations leading to the Einstein Podolsky Rosen paradox. But obviously most of the badly performed EPR experiments will separate the flying apart quantum entities as Alain Aspect once told me when I informed about this, of course being amazed why this information even would interest me. [10]

Let us assume that experimental results such as described above, supporting the expectation $y = \cos^2 \Phi$, have been shown to be statistically robust for sets of photons generated by the 'entanglement' procedure. For these photons, at least, a non-local attribute can be claimed and this is defined by Bell as:

... a mechanism whereby the setting of one measuring device can influence the reading of another instrument, however remote. Moreover, the signal involved must propagate instantaneously ... [12]

What 'mechanism' or 'signal' might this be? It seems to me that there is an obvious candidate: a pair of photons. What we are told about them is that they can be considered identical for the purposes of this experiment and that when they interact with differently oriented detection equipment a predictable, statistically quantifiable, result is obtained. Now in other contexts the finding of a correlation spurs most of us to look for a causal connection. In practice we are so ready to make this assumption and statistical procedures are so much abused that the vital step of explaining the correlation is often left out. But in this simple case why is the presumed identity of the photons not accepted as the common factor explaining the correlation? It seems that, in the absence of a model for the interactions of the photons with the detectors (dare I call them 'hidden variables?'), the mathematical formalism has been accepted as complete in itself and then, as in the case of Heisenberg's half-silvered mirror, misinterpreted. This defeatism did not include Richard Feynman, de Broglie or David Bohm and the latter's alternative model was apparently fully accepted by Bell:

Is it not clear from the smallness of the scintillation on the screen that we have to do with a particle? And is it not clear, from the diffraction and interference patterns, that the motion of the particle is directed by a wave? De Broglie showed in detail how the motion of a particle, passing through just one of two holes in screen, could be influenced by waves propagating through both holes. And so influenced that the particle does not go where the waves cancel out, but is attracted to where they cooperate. This idea seems to me so natural and simple, to resolve the wave-particle dilemma in such a clear and ordinary way, that it is a great mystery to me that it was so generally ignored. [5]

So much for the wave/particle dilemma but Bell still thinks there is a problem:

The difficulty is not created by any such picture or any such terminology. It is created by the predictions about the correlations in the visible outputs of certain conceivable experimental set-ups. [5]

So his problem, like Heisenberg's, is still with probability. Identical photons measured at identical angles give identical results: a 0% mismatch. If detector A is set to 0 degrees and B to 45 degrees the mismatch is 50% i.e. there is no correlation. With A at 45 degrees and B at 22.5 degrees the mismatch is 14.6%. With A set to 45 degrees and B set to 67.5 degrees the mismatch is 14.6%. If A is set to 0 degrees and B to 22.5 degrees the mismatch is 14.6 %. If A is set to 0 degrees and B is set to 67.5 degrees we get a mismatch of 85.4% (or a match of 14.6%). Now we have to look at the graph of our results and make a guess at the

function which expresses the correlation. It is not too difficult. Despite this Bell decided that an application to these results of tests for a linear correlation, if the tests failed as given this evidence they must, would be sufficient to close down all further scientific speculation in the quantum field. The 'spooky action at a distance' was to be explained by the 'collapse of the wave function' describing both photons. All we computer buffs know functions can collapse pretty damn quick but in the real world we do not expect to stub our toes against their shards. So let us reason together.

'Collapse' : Collective Noun for Quantum Mechanical Functions

All correlation of material objects which are space-like separated are in a sense "non-locally" interrelated. [13]

... all that is needed to demolish Bell's logic is the observation that the correlations are invested in the past light cones of both detection events, ... [13]

There is no such phenomenon as the collapse of a function. Functions are abstract, mathematical concepts which may be better or worse models but they do not collapse, particularly not with causal effect. If they model a real world situation which suffers an abrupt change, for instance an interaction with detection equipment, this may correspond to a particular solution of the function describing it. It is misleading to describe this as a 'collapse'. The set of possible solutions, perhaps infinite, of a probability function is also abstract. It is not a set the members of which have a property of 'objective indefiniteness' whether super- or sub-posed. It may moreover have mathematically acceptable solutions that cannot be realised empirically.

The nature of the entanglement procedure combined with the design of the Bell experiment is such that the polarity of the pair of photons, although dissimilar at creation is registered as matching if put through polarisers oriented in parallel and facing each other. It is therefore convenient to refer to such photon pairs as 'matching' because they give identical outputs at the calibration angle. This causes less confusion than trying to correlate using $+/ -$ or $1/ -1$ nomenclature and is therefore used throughout this analysis. Moreover it is implicit in the Bell thought experiment that the result of passing a stream of photons through a polariser can, at least in principle, be recorded for each individual photon. Therefore it should be possible using two entangled beams to bring the opposed polarisation meters into coincidence and record two sets of results showing for each which of the upper or lower photomultipliers detected each photon. The resulting set of matched up/down alternatives is then an example of the ideal raw statistical data to be correlated.

If one of the meters is rotated with respect to the other, it is found that the recorded coincidence no longer holds. This is a clear demonstration that the up/down outcome is a relational property of the event detected. If we assume that the two photon streams are identical in all relevant respects this allows us to conclude that the output path of a particular photon has been changed by the re-orientation of the meter with which it has interacted. A change of path is all that we can speak for; we have no warrant for claiming that there has been any fundamental change in the characteristics of the photon itself.

There is only one re-orientation which changes the path of every photon in a stream. This is when the B meter is rotated 90 degrees with respect to the A meter. In all other circumstances only a proportion of the stream is diverted and this proportion can be predicted from the relative angle between the meters calculated with respect to an orientation at which the outputs were coincident. At such a calibration orientation (parallel polarisers) the recorded outputs will show no mismatched photon pairs. If meter B is then turned through 45 degrees the resulting mismatch will be 50%. One can speculate that 1) at least half of these photon pairs have not retained their created polarity or, 2) that this expected pairing is only manifest when the meters are identically oriented and every possible orientation manifests a distinct and not necessarily compatible pairing of polarity or, 3) that for some photons and not all, the change in meter angle causes them to register at a photomultiplier which does not match that encountered by their twin. Whatever the explanation the resulting correlation demonstrates that measured polarity is mutable. Bell does not offer any reasoning to support his contention that these results imply:

there must be a mechanism whereby the setting of one measuring device can influence the reading of another instrument, however remote. Moreover the signal involved must propagate instantaneously, so that such a theory could not be Lorenz invariant. [13]

What is not spelled out is the respect in which the reading at the remote location is influenced but, as we are concerned with only two possibilities, the choice is limited to a switch from 'up' to 'down' or vice versa. Is there any way in which this could be incontrovertibly established? Bell seems to think this is shown because the gross statistics for correlations over four angles show that the relationship is not linear but if there is an argument showing the rationale of this I have not found it. The most convincing demonstration would be to obtain detailed results for several streams of pairs of entangled photons created and tested simultaneously. One would then look for examples of individual photons showing discrepancies in behaviour between a photon and its twin. This is not a practical experiment but it can be simulated to model the precise circumstances for which non-local instantaneous influence could be claimed.

The graph which represents Bell's expected outcome shows a balance point at 45 degrees. The proportions of matching pairs between relative angles of zero and 45 degrees are balanced by the proportions of pairs which record as mismatches between 45 and 90 degrees. The appearance on the curve of this fulcrum at 45 degrees is easily misinterpreted to suggest that the correlations for an angle defined between a nominal 22.5 degrees and 45 degrees should differ from an angle defined between a nominal 45 degrees and 67.5 degrees. It must be remembered however that the labels '22.5', '45', '67.5' and so on are not significant except as an aid for the laboratory assistant: only relative angles, however set up, affect outcomes. The axes of the graph show the correlation for any relative angle each of which can be defined by any pair of nominal settings. Thus the expected outcomes for 0/22.5, 22.5/45 and 45/67.5 must be read from the same point on the curve: that is the y reading for a relative angle of 22.5 degrees. It is not, of course, the same point as the y reading for 67.5 degrees. The Bell inequality is thus seen to be an (inappropriate) test for linearity, nothing more. The choice of three pairs of labels for contiguous angles which together add to 67.5 is window dressing and without significance.

In keeping with one of the published realisations of the Bell experiment the angles chosen for the simulated comparisons here are between zero and 22.5 degrees, zero and 67.5 degrees, 45 degrees and 22.5 degrees and 45 degrees and 67.5 degrees. The population simulated is 8x1024 in 4 pairings. The table shows which of the Up or Down photomultipliers responded for each of the first 30 pairs in each stream. All outputs are compatible with the A and C populations. The B, E and H outputs are identical as are F and G. A detailed breakdown of the correlations for each angle is shown below. A is the random alternative from which the others are derived by application of $\cos^2 \Phi$ where Φ is the angular difference shown for each stream.

Simulated simultaneous readings (random link constant)

Stream 1:angle 22.5

A 0 D D D U U D D U D D U D D D U D D D U D D D U U U D U U U D D U

B 22.5 D D D U U D U U D U D D U U U D U D D D U D U U U U U D D U

Stream 2:angle 67.5

C 0 D D D U U D D U D D U D D U D D D U D D D U U U D U U U D D U

D 67.5 U U U D D U U D U U D U D U U U D D U U D D D U D D D D U U

Stream 3:angle 22.5

E 22.5 D D D U U D U U D U D D U U U D U D D D U D U U U U D D U

F 45 D D D U U D D U D D U D U D D D U D D D U U U D U U U D D U

Stream 4:angle 22.5

G 45 D D D U U D D U D D U D U D D D U D D D U U U D U U U D D U

H 67.5 D D D U U D U U D U D D U U U D U D D D U D U U U U D D U

A compare B

matching up 0.4121

matching down 0.4424

mismatch up/down 0.0762

mismatch down/up 0.0693

correlation 0.8545

C compare D

matching up 0.0723

matching down 0.0742

mismatch up/down 0.4160

mismatch down/up 0.4375

correlation 0.1465

E compare F

matching up 0.4121

matching down 0.4424

mismatch up/down 0.0693

mismatch down/up 0.0762

correlation 0.8545

G compare H

matching up 0.4121

matching down 0.4424

mismatch up/down 0.0762

mismatch down/up 0.0693

correlation 0.8545

The overall correlation conforms well with the Abner Shimony figures [2] but the UP/DOWN split is uneven, probably because 1024 is a very small population.

The simulation has, of course, to use the pseudo random number generator which requires a 'seed' value known as the random link which normally changes whenever it is invoked so that no two selections are the same. However the random link can be held constant and I have used this facility above. Careful scrutiny of the first-30-outputs table will show that the random link has been held constant for all applications of $\cos^2 \Phi$. This leads to a correlation between individual photon events. If a photon is selected at an angle of 22.5 it will also be selected at angles 45 and 67.5. It behaves as if, for specific photons, the tendency to change path is progressive over increasing angles. It is as though a given photon has a critical angle at which it will change path. This cannot be considered compatible with quantum theory because it could suggest a hidden variable and so it is only included here as an interesting speculation.

If the random link is released to vary with every use the split between UP and DOWN is different over the four tests and still rather far from the theoretically ideal 50/50 split, as can be seen below.

A compare B

matching up 0.4150

C compare D

matching up 0.0664

matching down 0.4395
 mismatch up/down 0.0732
 mismatch down/up 0.0723
 correlation 0.8545

E compare F
 matching up 0.4229
 matching down 0.4316
 mismatch up/down 0.0645
 mismatch down/up 0.0811
 correlation 0.8545

matching down 0.0801
 mismatch up/down 0.4219
 mismatch down/up 0.4316
 correlation 0.1465

G compare H
 matching up 0.4277
 matching down 0.4268
 mismatch up/down 0.0762
 mismatch down/up 0.0693
 correlation 0.8545

Simulated simultaneous readings (random link varying)
 Of course, in this case the tendency to change path is not progressive.

Stream 1: angle 22.5

A 0 D D D U D U D U D D D D U D D D U D U D D D D D U U U D D D

B 22.5 D D U U U U U U D D D D U D D D U D U D U D D D U U D D D

Stream 2: angle 67.5

C 0 D U U D D D U D D U D U U U U U D D D D D U U D U D U U U

D 67.5 U U U U U U U U U U U D D D U D U D U U U U U D D D U U U

Stream 3: angle 22.5 # # # # # # #

E 22.5 D D U U U U U D D D D U D D D U D U D U D D D U U D D D

F 45 D D U D U U U U D D U D D D D U U U U D U D D D U D D D U

Stream 4: angle 22.5

G 45 D D U D U U U U D D U D D D D U U U U D U D D D U D D D U

H 67.5 D D U U U U U U D D U D D D D U U D U D U D D D U D D D U

So whereabouts in this simulation should we point the accusing finger and say, "this correlation is unreasonable"? The pattern followed is the same as that for Malus' Law so one would not be inclined to say that the proportions are unexpected. What then of the behaviour of individual photons? The only candidates for anomalous behaviour are Streams 3 and 4 in the case of photons marked #. Here the behaviour of the E stream does not match that of the H stream although the F and G streams are identical and the angular difference is the same. However this difference in behaviour cannot give rise to total correlations higher or lower than would be expected by Malus' Law so where is the spook?

Why is it thought that the entangled photons are so touchy/feely that they change their hidden polarity in mid-flight to make the numbers at the counters come out right? How many times must it be said? The click in the photomultiplier is

registering the relationship between the photon and the detector on this occasion under these conditions. There is a pattern to such relationships which can be predicted probabilistically but probability is not a property of any thing not even a quantum particle. At lunch I may sit on your right, at dinner on your left but neither of us has a 'handedness' property; it applies only to the seating arrangement (lunch) or the seating arrangement (dinner). I may look down on a spinning ball and say, "it is spinning clockwise" or up at it and say, "it is spinning anti-clockwise". That does not mean the ball has a property of superposition of spin clockwise and spin anticlockwise. Correlations measure relationships and are the business of probability calculus, the functions of which have tended, wrongly, to be objectified since the earliest days of quantum theory. There is an amusing analogy drawn in reference [12]. Consider the colour of a chameleon: when it is measured on a leaf it is green but measured on a log it is brown, unless it is a mutant when! There is, I am happy to find, a much more respectable reference which presents something close to the case I have been making. It is in a site devoted to Kochen-Specker under the heading:

'Causal Contextuality':

An observable might be causally context-dependent in the sense that it is causally sensitive to how it is measured. The basic idea is that the observed value comes about as the effect of the system-apparatus interaction. Hence, measuring a system via interaction with a P-measuring apparatus might yield a value $v(g(P))$, measuring the same system via interaction with a Q-measuring apparatus a different value $v(f(Q))$, although both observables are represented by the same operator $f(Q) = g(P)$. The difference in values is explained in terms of a context-dependence of the observables: The latter are context-dependent, since the different ways to physically realize them causally influence the system in different ways and thereby change the observed values.

Note: FM = 'Faithful Measurement' i.e. what the measurement value shows was that value of a property of the quantum object before measurement.

If an interpreter wanted to defend causal contextuality, this would entail abandoning FM, at least for observables of the type $f(Q)$ (non-maximal observables): Since their values causally depend on the presence of certain measurement arrangements, these arrangements are causally necessary for the values to come about, thus the values cannot be present before the system-apparatus interaction, and FM is violated. As an advantage of causal contextualism the following might be pointed out. It does not imply that the ontological status of the physical properties involved must change, i.e. does not imply that they become relational. If the property in an object is brought about via interaction with another one, it can still be one which the object has for itself after the interaction. However, the idea of causal contextuality is sometimes discussed critically, since there is reason to think that it may be empirically inadequate (see Shimony 1984, Stairs 1992). [14]

Notice that according to this definition I am a causal contextualist subject to the caveat that the concept of Faithful Measurement is irrelevant. Kochen and Specker are still talking about 'a property in an object being brought about via interaction with another one'. I am saying that where I happen to sit at table does not change my properties but it does change the relational properties of the seating arrangement. Similarly if I say "the ball is spinning clockwise" it will be uninformative unless my point of view is taken into account.

R.I.P. Bell's Inequality?

References

- [0] Sylvia Camacho, *Bell's Inequality*, Vector Vol.4 No.1, *Can Anyone Tell Me Where I'm Wrong?*, Vol.18 No.2
- [1] <http://www.roxanne.org/epr>
- [2] Graeme Robertson, *Unity Consciousness & The Perfect Observer*, Robertson (Publishing), 1995 ISBN 09524167 0 0
- [3] Karl R. Popper, *Quantum Theory and the Schism in Physics*, Hutchinson, 1982 p76, ISBN 0 09 146170 7
- [4] W. Heisenberg, *The Physical Principles of Quantum Theory*, 1930, p39
- [5] Sheldon Goldstein, *Quantum Philosophy: The Flight from Reason in Science*, Dept. Math., Rutgers Univ., NJ 08903, 1996 – see www.arXiv:quant-ph/9601007 v1
- [6] Richard P. Feynman, *QED*, Princeton University Press, 1985, ISBN-691-08388-6, p75
- [7] *ibid.* p55, footnote
- [8] e-mail of 8 August 2003
- [9] D. Aerts, *The Einstein Podolsky Rosen paradox and its solution*
<http://www.vub.ac.be/CLEA/aerts/publications/EPR-paradox.html>
- [10] *ibid.* A solution of the Einstein Podolsky Rosen paradox
- [11] Caroline H. Thompson, *The Tangled Methods of Quantum Entanglement Experiments*, Dept. Comp. Science, Univ. Wales – <http://users.aber.ac.uk/cat/>
- [12] <http://uk.arxiv.org/list/quant-ph/recent> from which I found, *Locality and Bell's Inequality* at arXiv:quant-ph/9601007 v1 10 Jan 1996
- [13] A.F.Kracklauer, *Exclusion of correlation in the theorem of Bell*, arXiv:quant-ph/0210121 v3
- [14] www.plato.stanford.edu/entries/kochen-specker/#background

Efficient Compilation of Array Expressions

by Paul Cockshott

ABSTRACT. This article investigates some of the problems involved in generating efficient code for array expressions. It considers issues of cache pollution, effective register use, and vector instructions. It considers the impacts that such compilation efficiency issues can have of the definition of imperative array languages.

Let us consider compiled programming languages that allow generalised array expressions, for example F[5], ZPL[6] and Vector Pascal[4]. These all allow expressions of the form $x\omega y$ where ω is drawn from some set of arithmetic operators and x, y are either array variables or other array expressions. This class of languages derives its inspiration from APL which is generally implemented as an interpretive language and which has inspired a family of other interpretive languages J, K, A+.

In order to bring out some of the issues involved in compiling such expressions let us first consider how an interpretive language might handle them. Initially consider where ω is a primitive operator like addition or multiplication. If the interpreter is written in C then the array expression can be evaluated by a generalised function of the form:

```
arrayt eval(arrayt x, arrayt y, char omega)
```

where `arrayt` is a type representing a pointer to an array descriptor on the heap. When the function `eval` returns it will have allocated a new array on the heap into which it will have written the result of applying the operation `omega` to corresponding elements of `x` and `y`. If we abstract from the problem of handling arrays of different rank, and consider initially the case where `x` and `y` are vectors then this can be done with the following algorithm:

Algorithm 1 Interpretive array expression evaluation

```
Find L the minimum of the lengths of x and y.
Allocate a new array R of length L.
For i = 0 to L-1 set  $R_i = \text{primeval}(x_i, y_i, \omega)$ 
Return R
```

Here *primeval* is a function that evaluates the scalar expression $a\omega b$, for scalars a, b . We chose an interpretation of $x\omega y$ that returns an array whose length is the minimum of the lengths of x, y for simplicity of explanation.

This is clearly a very naive interpretive algorithm since it costs a function call for each primitive evaluation. Efficiency can be greatly improved by using a case statement to branch on ω prior to for loops:

```
switch(omega){
case ('+'): for(i=0; i<L; i++) R[i]=x[i]+y[i]; break;
case ('-'): for(i=0; i<L; i++) R[i]=x[i]-
y[i]; break;
---etc...
```

In this case, the efficiency of the interpretive code can be quite high, indeed it appears to tends to the limiting efficiency of compiled code as $L \rightarrow \infty$. This sort of optimisation explains why the performance of good interpretive array languages can be comparable with compiled code.

However the final run time performance of a program though crucially dependent on the number of arithmetic instructions in inner loops, the optimisation addressed above, also depends upon other factors. Among these are the efficient use of the storage hierarchy and the ability to exploit vector instructions.

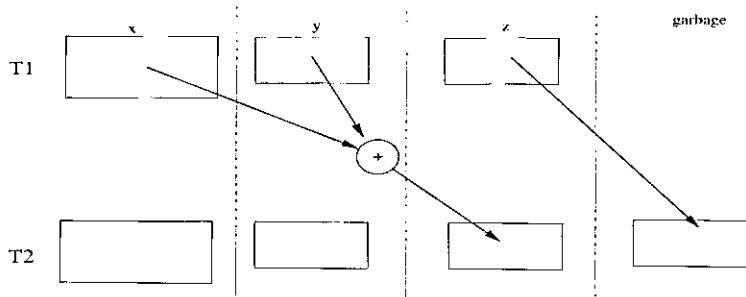


FIGURE 0.1. Data flow for $z \leftarrow x + y$.

Suppose we evaluate the assignment statement $z \leftarrow x + y$, then the data flow is as shown in Fig 0.1. At time T1 prior to the execution of the statement we have 3 buffers in memory corresponding to the arrays referenced by the three variables. We disregard the case where z has no initial value. It can be seen that the operation results in the discarding of the original buffer for z which becomes garbage. In a long running algorithm this will have to be recovered. This is storage overhead becomes more significant when we consider more complex array expressions like

$$z \leftarrow a * (b - c)$$

In this case a temporary array will be created for the result of $b - c$ prior to the array multiply. The first order cost of recovering the space will depend on the sort of storage management algorithm used, and in particular on whether the algorithm always clears any buffer that it allocates. If it is not cleared, then the allocation and recovery will be independent of the buffer size. If the buffer is cleared, then this imposes an overhead of at least one memory store for each pair of primitive operands evaluated in an array expression. By itself, this is a relatively small overhead which can be avoided if there is a non-initialising call to the buffer allocation API. But it becomes more significant once we consider the interaction of buffer allocation with the memory hierarchy.

The levels to consider are registers, cache and main memory and in very large applications virtual memory. The interpretive implementation makes no attempt to exploit registers and makes very inefficient use of the cache. Since the result of each primitive binary operation is written to memory, and since the memory block is always newly allocated, it will tend not to be present in cache. When it is written to for the first time, and in this approach buffers tend to be written to only once, the result will be stored in cache. By the time the multiplication in $a * (b - c)$ is done, the result of the subtraction is in cache, so the multiplication proceeds relatively fast. But a cost of storing the result of subtraction in cache is that an equivalent number of bytes of other data must be purged from the cache. If the arrays are sufficiently large relative to cache sizes, then the result of the subtraction could flush the array a out of the cache. Given that access to main memory can be 20 times slower than cache access such cache flushing could seriously affect performance.

A naive array expression compiler would generate code whose dynamic execution would model the dynamic execution of an interpreter. We will model the compilation process by producing equivalent C code. Clearly one route to compilation is to translate the array code

into C and leave the machine dependent part of the compilation to the code generator. We will see later that this is not always optimal. The statement

$z \leftarrow a * (b - c)$

Would map to the code shown in Algorithm 2.

Algorithm 2 Naive compiled algorithm.

```

t1=memalloc(b_min_c);
for(i=0;i<b_min_c;i++)t1[i]=b[i]-c[i];
t2=memalloc(t1_min_a);
for(i=0;i<t1_min_a;i++)t2[i]=a[i]*t1[i];
dispose(z);
z=t2;

```

For simplicity we assume that the sizes of the result arrays `b_min_c` etc, have been pre-computed either at compile time or in an earlier code fragment. This C code, whilst it has removed the interpretive overheads still retains the same cache penalties as the interpretive code and memory allocation penalties as the original interpretive version.

An alternative approach would be to decompose the array operations so that the whole expression was evaluated as a single loop equivalent to the C code shown in Algorithm 3. This approach, termed drag-through, was described by Budd[3] and by Abrams[1] and has a number of advantages.

Algorithm 3 Optimal version.

```

for(i=0;i<zlen;i++)
    z[i]=a[i]*(b[i]-c[i]);

```

In this example we have actually carried out two transformations, we have removed the redundant creation of temporary vectors, and also re-organised the code so that the final result is written into the original buffer `z`, rather than a newly created buffer `z`. This optimisation is possible so long as `z` already references a buffer of the appropriate size, something that a compiler can usually determine.

The expression `a[i]*(b[i]-c[i])` is a scalar one, and as such can be effectively computed within registers. This gives us two advantages:

- (1) Since the temporary result of `b[i]-c[i]` is in a register, it can be accessed without any store accesses when needed for the subsequent multiply.
- (2) Since no cache writes occur for the vector result of `b-c`, the elements of the array `a` are more likely to be in cache when needed.

The effects of both these optimisations are shown in Tables 1 and 2. The broad features to note are :

- As vector lengths rise, the time taken to produce each byte of the result rises, this is due to generally poorer locality of access and thus generally poorer cache utilisation for larger vectors.
- In all cases the optimal code which evaluates temporary results in registers performs better than the naive code which allocates temporaries to memory buffers. In some cases it performs twice as well.
- The degree of advantage of the optimal code varies non-linearly with vector length as a result of interaction of the vector block sizes with the levels of the cache hierarchy. With certain combinations of sizes of vector and cache, the temporaries produced by the initial subtraction do not interfere with the source data. In these cases the performance penalty for the naive implementation is less.

vector length	2 ¹⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
naive μ s	0.053	0.047	0.056	0.082	0.088	0.105
optimal μ s	0.036	0.032	0.037	0.048	0.057	0.071
naive/optimal	1.48	1.48	1.49	1.70	1.53	1.48

TABLE 1. Crusoe performance on $x:=a*(b-c)$ implemented in registers versus naively. Code generated using 16 bit numbers with MMX vectorisation enabled. Total number of integer operations were held constant as the vector length varies by altering the number of times the operation is repeated. Times are in microseconds to perform the arithmetic to produce each 16 bit result averaged over a large number of runs on a 750MHz Crusoe processor with a 64KB primary cache. Compiled using the Vector Pascal compiler. The naive version was emulated by splitting the statement into two parts thus $f:=b-c; x:=a*f$;

vector length	2 ¹⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
naive μ s	0.016	0.02	0.021	0.029	0.029	0.03
optimal μ s	0.009	0.012	0.0019	0.013	0.014	0.015
naive/optimal	1.66	1.61	1.11	2.2	2.13	2.03

TABLE 2. P4 performance on $x:=a*(b-c)$ implemented in registers versus naively. Vectorisation was enabled. Times are in microseconds to perform the arithmetic to produce each 16 bit result on a 1300MHz P4 processor with a 8KB primary cache and 256k secondary cache. Compiled as described in Table 1.

VECTORISATION OF OPERATIONS

It is desirable to make use of the vector instructions of current CPUs for array programming. Once an array expression has been re-organised by the compiler into the optimal form, it can be vectorised. Let us assume we are working with a CPU capable of performing operations on 4 numbers at a time. We can denote this in pseudo C as:

Algorithm 4 Vector version.

```
// assume vector length 4
for(i=0;i<(zlen-3);i+=4)
    z[i:4]=a[i:4]*(b[i:4]-c[i:4]);
if( zlen %4)
    for(i=zlen-(zlen % 4);i<zlen;i++)
        z[i]=a[i]*(b[i]-c[i]);
```

Here the notation $z[i:4]$ represents a sub-vector of length 4 starting at position i in z . Because we can not be sure that the vectors to be processed will all be divisible by 4 we need two loops. The first handles the vectorisable code, the second a remainder that has to be evaluated in scalar fashion. Consider the case of a vector of length 8. The first algorithm will evolve as follows

```
i=0
compute result for z[0:4]
i=4
i<5 yess
compute result for z[4:4]
i=8
```

```

i<5 no
terminate first loop, skip second

```

For a vector of length 6 the algorithm will evolve as:

```

i=0
compute result for z[0:4]
i=4
i<3 no
6%4 nonzero
i=(6-(6%4))=4
compute result for z[4]
i=5
compute result for z[5]
exit

```

If the length of the vectors is known at compile time, it is possible for the constant folding stage of the code generator to spot when the test:

```
if(zlen %4)
```

will return zero and delete both the test and the second loop from the code.

vector length	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
optimal scalar μs	0.18	0.192	0.211	0.228	0.281	0.331
optimal vector μs	0.036	0.032	0.037	0.048	0.057	0.071
scalar/vector	4.74	6.0	5.7	4.75	4.93	4.66

TABLE 3. Comparison of scalar and vector code times for the statement used in the previous examples.

Use of vectorisation brings big performance benefits as is shown by Table 3. These are substantially greater than the speedups that can be obtained by the register optimisations discussed earlier. These sorts of optimisations are discussed in more detail in [2].

Implementation Issues. In the model advanced above, an array assignment write directly into store that has already been allocated. In the statement $x:=a*(b-c)$, the variable x is assumed to stand for an array whose rank is known at compile time, and whose size is either known at compile time, or has been determined at run time prior to the execution of the statement. Thus when performing a left to right parse of the statement, the parser knows rank of the destination before parsing the source expression.

This lends itself to a simple extension of recursive descent compiling techniques. Let us assume we have a procedure *expression* to parse expressions which returns a semantic tree for the expression. After determining the rank r of the destination, the parser carries out the following steps:

- (1) Declare hidden loop iterator variables $t_0, t_1, \dots, t_{r-1}$
- (2) Create a semantic tree l made up of r nested for loops using these iterators
- (3) Let $e = \text{expression}([t_0, t_1, \dots, t_{r-1}])$ to obtain a scalar semantic tree for the array expression
- (4) Build the semantic tree of a scalar assignment of the form $x_{t_0, t_1, \dots, t_{r-1}} \leftarrow e$ and incorporate this into the loop nest l .

On encountering an array identifier the procedure *expression* attempts to reduce it to a scalar using the list of iterators passed to it. If the rank of the array is too large to allow its reduction, then a type error is signaled.

If instead of the parser being hand written, an automatically constructed bottom up parser is used in the compiler, an analogous algorithm can be performed whilst walking the syntax tree generated by the automatic parser.

DATA FLOW ISSUES

The previous section has argued against evaluating each binary array operator to produce a new array value, because it is more efficient to combine all subexpressions within a loop operating on a scalar expression. So long as one is dealing with vector or matrix expressions of the form $M \leftarrow A\omega_1 B\omega_2 C$ where ω_1, ω_2 are pairwise scalar operators, such as + or -, which are mapped over the matrices, no problems arise. The semantics of both implementation approaches are the same. When we allow other operators such as a matrix transpose or matrix multiply, problems can arise.

Suppose we transpose the matrix $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ using the assignment $A \leftarrow B^T$, then A ends up with $\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$, and if we evaluated $B \leftarrow B^T$ we would expect B to also equal $\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$. However if we have translated $X \leftarrow Y^T$ for 2 by 2 matrices X,Y, as:

```
for(i=0; i<2; i++)
  for(j=0; j<2; j++)
    X[i,j]=Y[j,i]
```

using this approach $B \leftarrow B^T$ would result in the corruption of B as the transpose was being evaluated giving $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$. A similar corruption would occur with the evaluation of some matrix products by simple nested loops: $A \leftarrow B.C$ would be safe but $A \leftarrow B.A$ would not. It is to obviate these dangers that Fortran 90 defines the semantics of array assignments as having to evaluate the entire array valued expression on the right hand side of the assignment prior to performing the assignment itself. Vector Pascal takes the opposite approach, specifying that the semantics of array expressions is defined in terms of the equivalent loop construct with the compiler signaling an error if it encounters a data flow hazard in an array expression. Since, in the presence of parameter aliasing, it is not always possible to detect such data-flow hazards at compile time, one can argue that the Fortran 90 approach is safer. The additional cost incurred by the Fortran 90 semantics can be avoided in those cases where data-flow analysis can prove that no hazard exists - for instance where the arrays are not function parameters.

ARRAY BOUNDS CHECKING

It is highly desirable that an array language have some system of array bounds checking to detect errors. The simplest way to do this is to plant explicit tests on array bounds whenever an array is indexed. Using C as an algorithmic notation this would take the general form:

```
if(i<lwb_a) boundsfault();
if(i>upb_a) boundsfault();
b=a[i];
```

Which is clearly expensive and would dominate the cost of actual array access. But this can be done with as little as one extra instruction on an Intel processor using the

```
BOUND r, pair
```

instruction where r is a register and pair is reference to a two element vector in memory. This checks whether a register r is between the values pair[0], and pair[1] generating an interrupt if it fails. Since a modern processor will be able to fetch 64 bits at time from memory, both bounds can be accessed in a single memory fetch, but the extra instruction and extra memory fetch can still represent an overhead of 50% or more on the original cost of array access. It is thus desirable to try and minimise the number of array bounds checks that are done.

Suppose we start with a loop with naive bounds checking:

```

for(i=x; i<=y; i++){
    if(i<lwb_a) boundsfault();
    if(i>upb_a) boundsfault();
    total+=a[i];
}

```

Then it is clear that $x \leq i \leq y$ throughout the loop. Thus if an array bounds fault is to occur, either $x < lwb_a$ or $y > upb_a$. We can safely move the bounds check to before the start of the loop:

```

if(x<lwb_a) boundsfault();
if(y>upb_a) boundsfault();
for(i=x; i<=y; i++)
    total+=a[i];

```

So long as $x < y$ this formulation will have a lower overhead. Indeed in many case it allow the bounds check to be totally eliminated. If the array bounds and loop bounds are known at compile time, then the tests can be evaluated during compilation. If they pass the compiler can elide the bounds checking code, if they fail, it can signal a type error at compile time.

	code	μs	ratio
optimal array bounds	vector	0.03	
naive array bounds	vector	0.036	
naive/optimal	vector		1.2
optimal array bounds	scalar	0.076	
naive array bounds	scalar	0.18	
naive/optimal	scalar		2.36

TABLE 4. Crusoe performance on $x:=a*(b-c)$ in registers with no intermediate buffer assignment comparing naive with optimised array bounds checking, otherwise same test conditions as in tables 1 and 3. Vector lengths 1024.

In an array language, most of the for loops that the compiler deals with are automatically generated ones arising from array assignments. These lend themselves very well to the optimisations described above. For statically sized arrays, which are the predominant ones in classical Fortran or Pascal programming, most array bounds checks can be performed at compile time. For dynamically sized arrays most bounds checks can precede the loop. An exception comes in scatter-gather statements such as: $b:=a[c]$, where c is an array. In this case we must plant code to check that each $c[i]$ is within the bounds of a .

As Table 4 shows, the propagation of array bounds checking outside of loops can make modest but very worthwhile improvement to performance for vectorised code, but makes a very big difference to performance on non-vectorised code. For the vectorised code, array bounds are checked for only 25% of the entries anyway, reducing the advantage of optimising the checks.

Hazards. With ordinary for loops certain hazards can arise with this sort of optimisation of array bounds checking. Consider the following Java code:

```

int [] a={1,2,4,8};
....
for (i=0; i<10; i++){
    if (i==0) a=new int[10];
    a[i]=getdata();
}

```

If we propagated the bounds check for $a[i]$ to before the for loop thus

```
if(a.length<9)throw new ArrayBoundsExcep-
tion("a");
for(i=0;i<10;i++){
    if (i==0) a=new int[10];
    a[i]=getdata();
}
```

An exception would be falsely signaled by the propagated bounds checking code. In general this hazard can arise with languages that have arrays implemented as pointers to heap data structures. In this case array bounds optimisations are only safe if data-flow analysis can prove that the array variable points to the same heap block throughout the loop. If arrays are static this problem obviously does not arise. Nor does it arise in array expressions, allowing these to be optimised more easily than standard for loops.

Condition	Truth value	Action
known at compile time	true	signal compile time error
known at compile time	false	do nothing
unknown at compile time		plant code for check

TABLE 5. Rules followed by plantcheck.

Implementation Issues. How can these bounds checking optimisations be implemented?

One simple approach is for the compiler to maintain a stack of currently active loop iterators as it process the code. Associated with each loop iterator is an upper bound list and a lower bound list. The stack and loops are manipulated as shown in algorithm 5. When an array index operation $a[i]$ is encountered algorithm 6 is invoked. The function plantcheck either follows the rules given in table 5.

Algorithm 5 to propagate bounds checks.

```
push iterator i onto iterator stack
parse the body of the loop saving the gener-
ated code in buffer B
pop the iterator i from the stack
for each element e of the i.lwbList
    plantcheck(loopstart<e)
for each element e of the i.upbList
    plantcheck(loopfinish>e)
plant code in buffer B
```

Algorithm 6 to plant array bounds checking

```
CheckIndex(Array a, Expression i)
if i is an active loop iterator
    then
        append (i.lwbList,a.lwb)
        append (i.upbList,a.upb)
    else
        plantcheck(i<a.lwb)
        plantcheck(i>a.upb)
```

LOOP UNROLLING

	code	μ s	ratio
unrolling	vector	0.018	
no unrolling	vector	0.028	
unrolling gain	vector		1.55
unrolling	scalar	0.49	
no unrolling	scalar	0.6	
unrolling gain	scalar		1.22

TABLE 6. Effects of use of loop unrolling, otherwise same test conditions as in tables 4 and 3. Vector lengths 1024.

A final optimisation that can be applied in a compiler is to unroll the loops. Suppose we have vectors that we know from type information is going to be of length 4, for example the quaternions used in graphics processing. In this case we can replace loops by the equivalent sequential code even in the absence of a vector instruction-set:

```
for(i=0; i<4; i++) z[i]=a[i]*(b[i]-c[i]);
```

gets replaced by

```
z[0]=a[0]*(b[0]-c[0]);
z[1]=a[1]*(b[1]-c[1]);
z[2]=a[2]*(b[2]-c[2]);
z[3]=a[3]*(b[3]-c[3]);
```

The same principle can be extended to other loops whose iteration count is exactly divisible by some unrolling factor. The unrolling optimisation should be applied after vectorisation to get the best effect. Thus for arrays of length 1024 and a processor vector length of 4 we might have:

```
for(i=0; i<1021; i+=4){
    z[i:4]=a[i:4]*(b[i:4]-c[i:4]); i+=4;
    z[i:4]=a[i:4]*(b[i:4]-c[i:4]); i+=4;
    z[i:4]=a[i:4]*(b[i:4]-c[i:4]); i+=4;
    z[i:4]=a[i:4]*(b[i:4]-c[i:4]);
}
```

In the original un-vectorised loop, we had a conditional branch instruction for every word of the answer. In the vectorised and unrolled version we have one conditional branch per 16 words of the answer.

AGGREGATE EFFECT

If we now consider the combined effects of all of the optimisations we can see that they speed array expressions up more than 10 times (table 7). The gains from each optimisation

	improvement contribution	cumulative improvement
registers versus temporary array results	1.48	1.48
vectorisation	4.74	7.01
optimal array bound checks	1.2	8.41
Loop unrolling	1.55	13.0

TABLE 7. The cumulative effects on performance of all the array optimisations.

combine multiplicatively. The exact figures given for each of the optimisations will depend

on the machine it is being run on, the vector lengths and the efficiency of other aspects of the compilation process. In our examples we have used 16 bit arithmetic because this vectorises on any common-denominator Pentium compatible machine. The vectorisation gains for floating point code can be less.

Several of the optimisations are equally valid for array language interpreters. If these are partly written in assembler, then they can make use of vector instructions. Whether written in C or assembler they can unroll the inner loops of their primitive array arithmetic routines. Interpreters routinely eliminate array bound tests from their inner loops. A good interpreter can be competitive with compiled code. Table 8 shows comparisons between the A+ advanced APL interpreter and compiled Vector Pascal. Timings are given per floating point result for the calculation $x \leftarrow a \times b - c$ using vectors of length 1024. As before the non-pull-through version of the compiled code was implemented as two statements

```
t ← b - c
x ← a × t
```

to get round the fact that Vector Pascal uses pull-through by default. It can be seen that on both machines the interpretive code was competitive with the worst compiled code. Applying all of the optimisations described in this paper improves the speed of the compiled code by a factor of about 10. The peak performance of the compiled code is one useful floating point operation every two clock cycles, taking into account all the overheads of addressing 3 source and 1 destination arrays.

		750MHz Crusoe		1000MHz Athlon	
		μs	MFLOPS	μs	MFLOPS
a+ version	interpreted	0.097	20	0.036	56
no-pull-through	compiled	0.077	26	0.042	48
pull-through	compiled	0.047	42	0.026	76
pull-through plus loop unrolling	compiled	0.018	110	0.008	250
pull-through, loop unroll, vectorise	compiled	n/a		0.004	500

TABLE 8. Comparison of compiled versus interpreted loops. The interpretive timings are done using the A+ advanced APL interpreter. All compiled timings are for Vector Pascal. MFLOPs are calculated on the basis that there are 2 floating point operations per result

Although for many purposes the flexibility of an interpretive solution will outweigh the speed of compiled code, for others such as graphics, speed is important. In these cases a compiled array language provides an effective solution, combining relatively concise notation with very high computational speed.

REFERENCES

- [1] Abrams, P. An APL Machine, SLAC rep 114, Stanford Univ., 1970.
- [2] Bik, A. J. C., Girkar, M., Grey, P. M., Tian, X., Automatic Intra-Register Vectorization for the Intel Architecture, International Journal of Parallel Programming, Vol 30, No. 2, 2002, pp. 65, 97.
- [3] Budd, T., An APL Compiler for a Vector Processor, ACM Transactions on Programming Languages and Systems, Vol 6, No. 3, July, 1984.
- [4] Cockshott, P., Vector Pascal Reference Manual, ACM Sigplan Notices, Vol 37, pp 59-81, June 2002.
- [5] Metcalf, M., and Reid, J., The F Programming Language, OUP, 1996.
- [6] Snyder, L., A Programmer's Guide to ZPL, MIT Press, 1999.

Rowan.net: An APL-like Interpreter for .Net

by Richard Smith (richard@redcorona.com)

Introduction

A couple of issues ago I wrote an article about R.net (*Vector* 20.2, p.136), a reverse Polish array interpreter. As I hinted in that article the target was to include a parse table similar to Hui's (*Vector* 9.4, p.85) to allow APL-like syntax, and fulfil a longstanding ambition to write my own APL interpreter.

Rowan is the result of this effort: a language which is superficially very APL-like, using single-character primitives in the main and a conventional syntax. The APLers among you should recognise this code fragment as one which you could copy-paste into another language:

```
avg+{(+/ω)÷ρω}
a+3 5 6
avg a
Result: 4.666666666666667
```

This short example shows how Rowan allows direct-definition functions, how it uses the D-style α and ω for left and right arguments (although the averaging function is a monad, so there is no α used here) and how it supports operators like $/$.

Rowan is built on the same engine as R, so it has the same array capabilities, the same connection to the main .Net framework, the same error-handling and the same namespace architecture (see the article on R if you're interested). Having an infix syntax brings in new ideas too: monadic and dyadic functions, operators and conjunctions; these are all catered for in a second level of core, which builds on the basic concepts in the R core.

Rowan supports basic arithmetic operations on extended precision numbers, represented by the *LargeInteger* class. You enter an extended precision number by appending the backquote character (```) to a number:

```
li←1e300`      A Extended precision
(li + 1) - (li - 1)
Result: 2`
d←1e300        A Normal double precision
(d + 1) - (d - 1)
Result: 0
```

As you can see, you can use arithmetic functions with 'ordinary' numbers on one side and extended-precision ones on the other; in this case the narrow operand is widened to extended-precision.

These numbers have arbitrary precision, but obviously the space they take up gets larger the more significant figures are preserved: $(1i + 1)$ takes up 128 bytes. However they are smart and when they have blank bytes at either end they shrink themselves, so $1i$ is only 4 bytes plus an integer exponent.

Character Set

As you can see, Rowan uses APL-style one character glyphs. You might recall that R is an entirely ASCII language, which dates from its days as a Java applet where no APL font was present, and I had a decision to make when I started on Rowan. I decided that since .Net has good Unicode support throughout, I would throw off the restrictions of a small character set and make use of the APL characters which are defined in the Unicode spec. I have always liked the conciseness of APL, and the Unicode character set lets me have that without defacing the accented characters and so on; it also means there are unused symbols for which I can invent a use (for example, the Customise operator $\odot$).

Rank

An important feature is the concept of *rank*. This is effectively the level of 'digging' a function does before it starts to work, and is best illustrated with an example:

```
b+[[5 6][7 8]]
avg b
Result: (6 7 )
      (avg^1) b
Result: (5.5 7.5)
```

With no digging (rank 0, the default for user-defined functions), the averaging function does $((5\ 6) + (7\ 8)) \div 2$ - running on the whole array at once - whereas with rank 1 it 'digs' one level and then runs on (5 6) and (7 8) separately. You can use rank -1 to operate on the scalar elements at the deepest level of an array.

Of course all array languages have a similar method of getting at deeply buried data within arrays, probably slightly different from mine! For the APLers among you, you can imagine that *b* is a rectangular array, and the rank of a function then corresponds to the axis to operate on.

Dyadic rank is very similar, allowing you to dig into the arguments on both sides, but with the addition of separate left and right ranks.

Scope

Rowan takes the D approach that everything is local unless declared otherwise. It's not full-blooded lexical scope, because Rowan is based on a dictionary-stack language in R and so actually uses dictionaries for scoping; how it works is that every function internally pushes a new dictionary, localising everything declared within them. For example:

```
c←12
f←{c←4}
f
Result: 4
c
Result: 12
```

(This example also shows how the last statement in a function is its return value; see below.) To affect the global environment you need to use the Global Assign function, which is represented by the Left Vane character (⌵):

```
f←{c⌵4}
f
Result: 4
c
Result: 4
```

You have to use Left Vane in a script to assign functions and variables that you want to be available once the script has finished; the normal assign arrow makes things local to the script, so would be useful for intermediate values.

While control structures in Rowan often require a block, delimited by braces, as part of their syntax, these blocks don't enclose scope because you often need to assign variables in the function they are in, and that would be impossible if they did. (You can't assign to 'super-locals', i.e. up the stack, in Rowan, only to locals and to globals.)

Functions and Statements

As the example right at the start of this article shows, you define simple functions by assigning a functional block (enclosed by braces) to a name. The same method can be used to assign longer functions; ending a line with a brace open will let you

enter the rest of the function, until you enter the final closing brace. You can also use `#ed`, as in R, which lets you edit your function in a separate window.

An important point with Rowan is that functions don't have (in any syntactic sense) lines, they have *statements*. Because everything we've seen so far has been a single statement, we've got away with ignoring this because Rowan automatically adds a statement separator at the end of every block it's asked to parse (including session entry).

If we look at a simple script for the Average example – a script is run in exactly the same way as a function – we can see what this means:

```
A Avg.rws
avg←{((+/ω)÷ρω)};
a←(3 5 6);
b←((5 6)(7 8));
c←((1 2)(3 4));
```

Each line is ended with a semicolon, which tells Rowan where to split its parsing. Note that all the assignments use the Left Vane, as explained above, as otherwise they would be local to the script and disappear once it had finished running. The new-lines are no different from other whitespace; this is intentional to allow readable definitions of large arrays:

```
A Array.rws
array←(
  (1 2 3)
  (4 5 6)
  (7 8 9)
);
parray
```

The return value of a function is that of the last statement which produces a value, the same as Perl. You can force a return by using a monadic left arrow:

```
f←{
  ←4;
  3
}
f
Result: 4
```

Connecting to .Net

As with R, the classes of the Framework are available to you with no effort at all, using a similar colon syntax. Because of not needing to put all the arguments to the left of the function, function calls in Rowan are almost identical to their C# counterparts. For example:

```
Rowan: s←300.ToString("X");
C#: String s = 300.ToString("X");

Rowan: worldString←"Hello, world!":Substring(7 5);
C#: String worldString = "Hello, world!".Substring(7, 5);

Rowan: #using "System.Drawing"; (at top of file) ...
p←#new $Point(30 40);
C#: using System.Drawing; (at top of file) ...
Point p = new Point(30, 40);
```

This means Rowan has a large advantage over non-.Net APLs, as it doesn't need any utility libraries to make it useful – Microsoft have already written one for you! More than that, it's a very good one: the Framework has utility classes for almost everything you would ever want to do. And even better, there are a large number of freely available .Net components posted by developers on the Internet, (almost) all of which you can just call from Rowan – just point Rowan at it using `#addassembly` – without a problem, in case you find something that isn't in the Framework. Of course, you can also use your own components if you happen to be an amateur C# hack too.

I say *almost* because, at the time of writing, Rowan doesn't support delegates and therefore events signalled by external components. This is due to a technical difficulty which I hope to sort out once finals are out of the way, as it is the only major stumbling block left to producing functional applications with Rowan.

The .Net bridge works both ways, too, and it's just as easy to call Rowan from your own .Net classes, written in C# (or VB.Net etc). You need to use the RowanExecutor class from RowanInterpreter.dll, which has public methods for calling the engine. For example, here is a simple program which does $2 \times \pi / 110$ direct from C#:

```

using System;
using RedCorona.Rowan.Interpreter;

namespace Tests {
    public class RowanTest {
        private static RowanExecuter rex = new RowanExecuter();

        public static void Main(String[] args){
            int[] iota10 = (int[])rex.ExecuteMonad("ι", 10);
            int sum = (int)rex.ExecuteMonad("+/", iota10);
            sum = (int)rex.ExecuteDyad(2, "x", sum);

            Console.WriteLine("Answer is "+sum);
        }
    }
}

```

Obviously the whole thing could be done with one call (`int sum = (int)rex.Execute("2x+/ι10").Value`; the `Value` is because this form of execution returns a `Symbol` structure containing type information, which we don't need here) but I've split it up to show the different ways to access the engine. To get and set variables you can use `SetVariable` and `GetVariable`:

```

rex.SaveVariable("data", new int[]{1, 2}); // save
int[] data = (int[])rex.GetVariable("data"); // retrieve

```

... and to define a function use `SetFunction`:

```

rex.SetFunction("f", "+/ιω", -1);
int[] results = rex.ExecuteMonad("f", new int[]{7, 8});

```

The numeric argument is optional and specifies the rank at which the function will run, and the default is 0. This C# expression is equivalent to the Rowan `f+{+/ιω}^-1`.

You need to compile your assembly with reference to the assemblies `RowanInterpreter.dll` (of course) and `Core.dll`, and clearly you need a full copy of Rowan present when you run it.

You can also call the base functions (Plus, Iota etc) directly – they reside in the `Main` and `Additional2` assemblies – but if you do you lose the conciseness of the language and some functionality (the evaluation of rank, for example, is done by the engine), and you need to construct the `Symbol` structure yourself each time. On the other hand, you then don't need to distribute a full interpreter, just the appropriate assemblies plus `Core.dll` and `RowanCore.dll`.

Architecture

I know this is a vector-languages journal and not a C# one so I won't go into any great detail of how Rowan works, but it is probably instructive to see how simple a vector interpreter actually is.

R core

The little rubber feet of Rowan are the classes and structures of the R core: the Symbol structure, which represents data and does a lot of work to manipulate it; the Kernel class, which provides universal array (rank, type-matching, length-matching etc.) capabilities to functions; the SymbolTable class; the tokeniser; and the BaseEngine class which lets Rowan call other .Net functions, gives it stack, function stack and other basic features.

Rowan core

The Rowan core introduces concepts not needed in R: structures and delegates to deal with arguments, monadic and dyadic functions and operators, conjunctions, system variables, and to help with the parse table.

The Rowan Interpreter

The interpreter is actually quite simple. An expression is first tokenised, then the token stack is moved through matching patterns against the parse table until it is reduced to a single value. (An error occurs if an expression doesn't eventually reduce to a single value type - noun, verb, operator or conjunction at present - or an empty token stack.) The parse table is the vital ingredient, but (as with Hui's) is simply a list of 4-element patterns to match to, and an action to perform in each case.

Plugins

The actual functions are added to the interpreter at load time from plugin assemblies. The interpreter itself provides parsing, tokenising and syntax resolution, so the functions manipulate the data in its native state. This separation of framework and content makes it very easy to add more features.

Environment

So far nothing I've mentioned lets a user pass expressions to the engine! This is the outermost layer, providing a window, input, output and a (customisable) keyboard handler and making the language usable. There's more information about the environment below.

Flexibility

Rowan was originally written as a left-to-right language, because of what I thought would be a major problem parsing invocations (colon syntax, allowing calls to .Net methods outside the interpreter). I was eventually persuaded that this wasn't going to be popular, and the problem was actually not a problem, so I added right-to-left parsing. Because of the simple architecture this was simply a case of adding a new parse table, and now Rowan can parse either way! At startup its default is now right-to-left, but you can still change it with the system variable #r21.

Extensibility

Again like R, Rowan uses plugins to provide its functionality. Any assembly which contains a class implementing the *IRowanPlugin* interface (defined in RowanCore.dll) is loaded at start-time, giving it the opportunity to add more primitives and #-functions to Rowan. This potentially lets one create powerful array-based access to databases, websites and so on, simply by writing the C# to retrieve (and replace) data in these environments.

Environment

Because Rowan uses Unicode characters which are difficult to enter in most places, and because you will always want a session to try things out in as well as a script executer (this is one of the huge advantages APL has in rapid development), Rowan needs an environment.

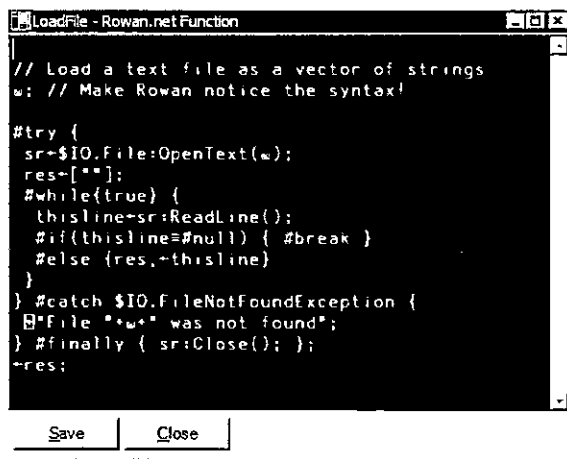
It is very simple:



The Rowan environment

The large edit field is obviously the session, where you type expressions and Rowan gives you the answer. It has a keyboard handler which lets you type the characters you need, which is customisable via a simple text file. I decided not to do what Dyalog have done for their APLScript and create a Windows keyboard layout for Rowan, because they are not customisable by the user; this means it is not possible to type Rowan anywhere except in the environment, but it can be copied and pasted into other Unicode-supporting applications (such as the Windows XP Notepad) so you can still create scripts easily.

The treeview on the left shows all the functions and variables you have defined, either by directly assigning them or by running a script which does so. You can double-click on a name in the tree, and if it is a function you will be able to edit it using the function editor:



```
// Load a text file as a vector of strings
w: // Make Rowan notice the syntax!

#try {
  sr~$IO.File:OpenText(w);
  res~[""];
  #while{true} {
    thisline~sr:ReadLine();
    #if{thisline=#null} { #break }
    #else {res,~thisline}
  }
} #catch $IO.FileNotFoundException {
  ⚠File "w" was not found";
} #finally { sr:Close(); };
~res;
```

The function editor

... which uses the same keyboard handler as the session, naturally. If you double-click anything else the value is displayed in the right-hand status field.

Conclusions

I think Rowan is potentially a very useful tool. It has certainly been a very instructive experience learning how an APL interpreter works, and I have to admit to being surprised at how easy it was to write one! By using a plugin-based system Rowan makes it possible to use its array capabilities in arenas I haven't even considered. I hope someone will find it helpful!

As I mentioned in the previous article, it is good to have a lightweight .Net vector language which makes it easy to talk to the rest of the .Net framework. Rowan has all the same 'call-out' capabilities as R, but because of its syntax it is much easier to use, and with its 'call-in' capability it is just as easy to use Rowan for little bits of array code within a standard .Net scenario.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Hack My Exe	Michael Baas	112
Technical Correspondence	Nakano, et al	116
Regular Expressions	Dan Baronet	126
APL to Unicode Transcoding in Java	Mark Wickens	137

Hackers' Corner: Hack My Exe

by Michael Baas (michael@mbaas.de)

Introduction

Dyalog's V10-feature to 'compile' a DWS into an EXE is really useful, as EXEs are easier to share with others (not requiring shortcuts to the runtime etc.). Even if it was possible before to share programs, it is just more fun now to write small utilities and have them available as EXE. (Now if only the accompanying DLL could be smaller – but that's another issue). The only issue I had with Dyalog's EXEs was that Dyalog did not give me control over the program- and version-information which shows up in the file-properties. So I started to research ways of changing that information. This article describes my findings.

When first hearing that V10.1 would include a possibility to control these, I thought this article would be redundant, but it isn't, as you get additional benefit from features not (yet?) available in Dyalog's tools: the tools described here allow for a fully automated build-process, so all properties can be set under program-control, and it is also possible to put additional resources into the EXE, like, for example, additional icons, so that your users have a choice.

Ingredients

Dyalog's EXEs are created in the Win32 PE-Format. PE stands for "Portable Executable" and is the standard-format for programs in the Win32-World (actually also for Win64, although with some modifications). More about the format can be found on the web, one comprehensive article is [1]. The important point is that since this is a standardized format, we can use any tool to edit those structures, there is no 'hidden APL-magic' preventing that.

There are various tools available on the internet to edit EXE-files, one tool that proved very suitable for the job was Resource Hacker [2] which has the advantage of being available as freeware.

With Resource Hacker it is possible to edit the version-information in the file and add icons to it – interactively. The program also has a command-line mode and supports scripting, but in that mode it requires pre-compiled resource files which it then inserts into the EXE.

Compiling – again

So, to automate the whole process, it is necessary to create a resource-file (.RC), compile it with a resource-compiler, then use Resource Hacker to update the resource-info in the EXE with the compiled .RES-file.

Fortunately that all is not as difficult as it may sound. A resource-compiler is included with Visual Studio (Common\Tools\Bin\RC.EXE) and if you don't have VS, you can download GoRC.EXE for free [3]. I had an issue when using GoRC to compile the resources, as finally the EXE was still showing the old version-info, yet I had no errors in the whole process. It finally turned out that the initial RC (created by using ResourceHacker's info) had no language-information in it and so GoRC used Windows' regional settings to determine a default-language (which, in my case, was wrong: I used German windows, but needed English language-info). Surprisingly Microsoft's RC did not have these issues, as it seems to use a hardcoded default ENGLISH,ENGLISH_US which was exactly right in this case. Anyway, the moral of the whole story: correct (English) language-info needs to be included in the .RC

Putting it all together

I use a function Build to increment version numbers, save my dws, write an appropriate RC, compile the RC, compile the DWS and finally launch Resource Hacker to update the resources in the EXE.

```
[0] Build;this;nu;txt
[1] this←{(φ^{\φω≠'\')/ω}WSID
[2] :If save
[3]     write_rc
[4]
    [+CMD [+c:\dyalog10\dotnet\aplc.exe /v /t:nativeexe
    ',this,'.dws',(FileX this,'.ico')/' /icon:',this,'.ico'
[5]
    A [+CMD "C:\Programme\Microsoft Visual Studio .NET 2
    003\Common7\Tools\Bin\RC.Exe" /r /v ',this,'.rc'
[6] [+CMD [+c:\tools\gorc\gorc /r ',this,'.rc'
[7]     :If FileX this,'.rh_script'
[8]
    [+CMD [+c:\tools\reshack\reshacker.exe -script '
    ',this,'.rh_script'
[9]     :Else
[10]
    [+CMD [+c:\tools\reshack\reshacker.exe -addoverwrite
    "',WSID,'.exe"',",WSID,'.exe"',",WSID,'.res",versioninfo,
    1,1033'
[11]     :EndIf
```

```

[12]    nu+'c:\tools\reshack\reshacker.log' NTIE 0
[13]    txt+ NREAD nu,82, NSIZE nu  NUNTIE nu
[14]    {{ }+1+ }'({= TC[2])= } TC[2],txt~ TC[3]  A wrap ev
every single line in output
[15] :EndIf

```

The function save handles version-info and the actual SAVE of the DWS, write_rc uses a template-variable, updates the version-info (simply by trying to execute anything enclosed in % and replace it with the results) in it and writes it to disk:

```

write_rc;txt;z;apl;file;nu
txt+myArc  A get resource-template
txt+txt,"= TC[3 2]
txt+qENLIST txt
A scan for any APL-Commands enclosed in '%'
z+('%'<txt)~{wv0,-1+ }'%'<txt
:While v/z
  apl+1+(1=+z)/txt
  :Trap 0  apl+~apl  :EndTrap  A execute the APL
  txt+((~1+z+1)ttxt),apl,(1~2=+z)ttxt  A repl. with res
  z+('%'<txt)~{wv0,-1+ }'%'<txt
:EndWhile
z+'%'<txt  txt+ (~z)/txt  A replace '%' with simple '%'
A and write the file
:Trap 0  A erase, if exists
  nu+(file+ WSID,'.rc') NTIE 0  file  NERASE nu
:EndTrap
nu+file  NCREATE 0
txt  NAPPEND nu
NUNTIE nu

```

And the variable my_rc contains this:

```

LANGUAGE LANG_ENGLISH,SUBLANG_ENGLISH_US
1 VERSIONINFO
FILEVERSION %GetVersion 1%,%GetVersion 2%,%GetVersion 3%,%Ge
tVersion 4%
PRODUCTVERSION %GetVersion 1%,%GetVersion 2%,%GetVersion 3%,
%GetVersion 4%
FILESOS 0x4
FILETYPE 0x2
{
BLOCK "StringFileInfo"
{
  BLOCK "040904B0"
  {
    VALUE "CompanyName", "Me and myself"
    VALUE "FileDescription", "Great app"

```

```
        VALUE "FileVersion", "%GetVersion 1%,%GetVersion 2%,%GetVersion 3%,%GetVersion 4%"
        VALUE "InternalName", "GreatApp"
        VALUE "LegalCopyright", "Copyright (C) 2004 Me"
        VALUE "OriginalFilename", "GreatApp.EXE"
        VALUE "ProductName", "GreatApp"

        VALUE "ProductVersion", "%GetVersion 1%,%GetVersion 2%,%GetVersion 3%,%GetVersion 4%"
        VALUE "Website", http://www.GreatSite.com
        VALUE "Support", support@GreatSite.com
    }
}
BLOCK "VarFileInfo"
{
    VALUE "Translation", 0x0409 0x04B0
}
}
```

As far as I know, there are no rules about the contents of the StringFileInfo-block, so you could add your own parameters there. `GetVersion` is a function which returns the current version-number (or selected parts of it, depending on the argument).

If you are interested in the `save`-function and others not shown here, or in the scripts, grab a copy of the DWS from the VectorLib [4] (or from my website [5], once it is reconstructed...)

References:

- [1] <http://msdn.microsoft.com/msdnmag/issues/02/02/PE/default.aspx>
- [2] <http://www.users.on.net/johnson/resourcehacker/>
- [3] <http://www.godevtool.com/>
- [4] <http://www.vectorlib.org.uk>
- [5] <http://www.mbaas.de>

CORRESPONDENCE

from Japan APL Association

Y. Nakano, K. Yamashita, T. Nishikawa

In J the primitive verb `p.` is for Polynomial. The monadic case of `p.` is for finding the roots of algebraic equations. This is implemented in J after Release 3 or in versions 3, 4 and 5.

For example, the solution to the cubic equation

$$5625x^3 + 59826x^2 + 339444x - 120366432 = 0$$

is coded in J as

```
p. |. 5625 59826 339444 _120366432
```

The three roots are given as the second item in the boxed notation

```
+-----+
| 5625 | 24 _17.3179j24.3248 _17.3179j_24.3248 |
+-----+
```

5625, the first item in the box, is the multiplier or the leading coefficient of the equation. That is very useful! But the use of `p.` seems to be not so popular for J users.

As far as we know, official instruction appeared only twice in the texts *J Exploring Math* (Iverson Software Inc., 1996) and *J Introduction and Dictionary* (K.E. Iverson, 1996). We suppose Dr Iverson considered that finding roots is simply an inverse of polynomials.

We, the members of the Japan APL Association and great fans of math, use the `p.` verb a great deal, because we can use it to solve algebraic equations of up to the 23rd degree.

For the cyclotomic polynomials $x^n - 1$, we have already solved them for values of n up to 143.

But recently, we found that the verb `p.` leads at times to fatal limit errors. For example, solving a simple cubic equation such as

$$x^3 - 3x^2 + 6x - 4 = 0$$

by `J` notation

```
p. |. y3=.1 _3 6 _4
```

causes a limit error.

To search for such examples more easily, we defined here a verb

```
sol=:>@({:@(p. @|.))
```

instead of using the `p.` directly.

For example, `sol 1 _3 6 _4` gives a limit error.

Similar results obtain in the following examples:

- in the Cubic (1 _3 9 _9), (1 _6 6 _2), (1 _6 18 _18), (1 _9 9 _3), (1 _9 18 _12), (1 _12 6 _1), (1 _12 12 _4), (1 _12 18 _9), (1 _15 15 _5), (1 _15 30 _20);
- in the Quartic (1 _10 27 _30 12);
- the Quintic (1 _2 _2 4 1 _2), (1 2 1 _1 _2 _1);
- the Sextic (1 _3 0 6 _3 _3 2);
- the 8th (1 0 1 _10 _33 _40 88 210 175);

and many others from polynomials derived from them.

To escape instantly from this limit error, we propose here some trials.

1. A very, very small shift in the next leading coefficient of the equation: e.g. `y3s=.1 _3.00001 6 _4`. And we solve this with `y3s` instead of `y3` by `p.` The smaller the shift the better; even a shift as small as `1e_9` works.
2. Shift coordinate ($x - r$) or $f(x-r)$ so as to reduce the next leading coefficient to zero; we made a verb coded in `J` `znc`. Then,

```
znc y3 y3z=.1 0 3 0
```

We solve the new equation y^3z using $p.$, and afterwards return r to recover roots for the old equation y^3 . The final roots are $1 \ 1j \ 1.73205 \ 1j \ 1.73205$. This procedure is expressed as the verb `sol0r`. Our verbs are shown below. The names are suggestive of the roles.

3. Of course, a combination of the above two methods might be more useful. If the next leading coefficient of the equation is already zero, as above, in the 8th $(1 \ 0 \ 1 \ 10 \ 33 \ 40 \ 88 \ 210 \ 175)$, we must adopt the first method i.e. the item 1.
4. We have already written several more trial verbs to replace the J primitive $p.$, and should like to contribute their details gradually in the coming journals.

J program of an example verb `sol1` among the `sol0r` is displayed below.

```
sol1 =: 3 : 0
r =. -0.1*(1{y)%
```

In the verb `sol1`, r is a shift in the coordinate variables x , and the z means $f(x+r)$. Accompanying verb `horn` is omitted here, as it is a simple Horner method.

Y. Nakano, nakano@mta.biglobe.ne.jp

K. Yamashita

T. Nishikawa, Toshio.Nishikawa@kiu.ne.jp

Japan APL Association

http://homepage3.nifty.com/asagaya_avenue/apl/eng_index.html

Arithmetic Boundary Checking

by Stephen M. Mansour (steve@carlislegroup.com)

Abstract

A point partitions a line into three disjoint subsets: the set of points to its left, the point itself and the set of points to its right. Relational functions produce two results 0 and 1, effectively partitioning a line into only two subsets. A technique, known as arithmetic boundary checking, uses arithmetic functions instead of relational and Boolean functions to determine where a point lies in a line or a plane.

Relational Functions

Relational functions ask the question: is the statement true or false? If the statement $\alpha < \omega$ is true, we have all the information we need, but if it is false, we don't know whether $\alpha = \omega$ or $\alpha > \omega$. To get a complete picture, we need to perform a second test. The dynamic function `bp` performs two tests to produce a Boolean pair¹:

```
bp←{⌈(α<ω)(α>ω)} A Boolean pair (2-vector)
2 3 4 bp 3        A α<ω↔1 0 ∅ α=ω↔0 0 ∅ α>ω↔0 1
1 0 0
0 0 1
```

While this preserves information, there are two problems. First, it seems somewhat redundant to do two similar comparisons, and second, we would like a scalar result because it will allow us to work more easily with arrays of data.

Signum Difference

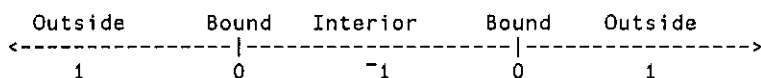
The signum function ($\times$) naturally partitions a line into three disjoint subsets, by producing three results (-1, 0, 1). We can apply signum to the difference between a value and the boundary point to produce the *signum difference*, which asks a more specific question, i.e. what is the *relationship* between α and ω ?

¹ Note: all APL expressions assume $\square IO \leftarrow 0$ and $\square ML \leftarrow 3$

Partitions

A pair of distinct points on the real line can define two types of partitions: an interval partition and a range partition.

An *interval partition* divides the real line into three disjoint subsets: the interior (between the end-points), the boundary (the two end-points themselves), and the outside (everything else).



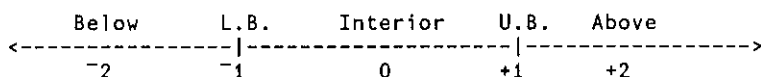
The product of the signum differences between a specified point and each of the boundary points will indicate whether the point falls in the interior, the boundary, or outside. If the point is outside the interval, the differences are both positive, or both negative so their signum product is +1. If the point is on the boundary, one of the differences is zero, thus the product is zero. And if the point is in the interior, the differences have opposite signs, so the signum product is -1.

```

xp+{*/*ω.-α}      A Signum product
 1 3 xp :5          A produces interval partition
1 0 -1 0 1

```

A *range partition* divides the real line into five disjoint regions: below, lower bound (L.B.), interior, upper bound (U.B.) and above:



	A+P xd LB	B+P xd UB	xp:A×B	xs: A+B
Below	-1	-1	1	-2
Lower bound	0	-1	0	-1
Interior	1	-1	-1	0
Upper bound	1	0	0	1
Above	1	1	1	2

Table 1: Signum Product and Signum Sum

The sum of the signum differences will produce five distinct values ranging from -2 to +2 and corresponding to each of the 5 regions in a range partition.

```

xs+{+/xw°. -α}    A Signum sum
 1 3 xs 15         A produces range partition
-2 -1 0 1 2

```

Two-Dimensional Region Checking

A blog entry (http://www.dapl.blogspot.com/2002_10_01_daparchive.htm) expresses the frustration of using standard Boolean logic in two dimensions:

=====

Friday, October 04, 2002

ONE AFTERNOON LAST WEEK while programming I wrote the following:

```

A←(R>0>UL)∧R<0>BR
A←A∧(C>1>UL)∧C<1>BR

```

It was so bad I had to break it into two lines.

Isn't it interesting that it is immediately apparent that this code is less than optimal, just from the look of it, even if you have no knowledge of the application? As it turns out, I was adding a drag and drop feature between two ListView objects, and I needed to determine whether or not a point was in an area. In other words, given a point (R,C), and the upper left (UL) and lower right (BR) coordinates of the target, was the mouse over the target ListView?

Knowing I could not live with this in my workspace, and knowing my own limitations as a programmer, I asked Brother Steve to come to the rescue. He immediately produced the following gem:

```

A←(R C){0∧. >×/▷α-+@▷w}UL BR

```

which works in N-space too! And a little tweak tells you whether you are "in" the area, "on" the border, or outside in a particular direction.

POSTED BY PAUL MANSOUR | [12:37 AM](#)

=====

The proposed solution in the blog exploited signum difference with respect to a rectangular region. We now formally extend the concept of arithmetic boundary

checking to two dimensions. You may want to know whether a point is in the interior of a rectangle, on the boundary or outside the rectangle.

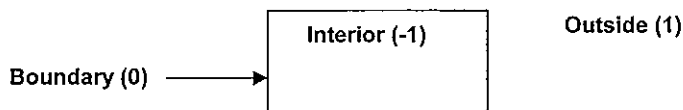


Figure 1: Regions in the plane defined by a rectangle

In the following example, we have six points in various positions relative to a rectangular region in the plane. The coordinates of the upper left and lower right corners of the rectangle are (3,2) and (5,6) respectively. The signum products of each coordinate must be calculated separately.

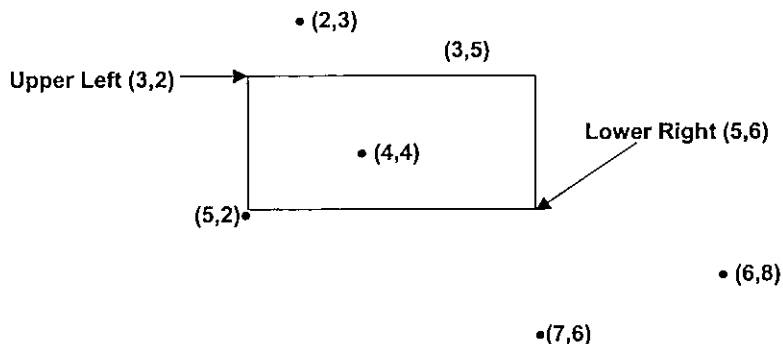


Figure 2: Points in various positions relative to a rectangle in the plane.

If we take the signum product of the first coordinate with respect to the row coordinates and the signum product of the second coordinate with respect to the column coordinates, we obtain a ordered pair indicating the position in each direction.

Table 2 on the next page shows this for the points in Figure 2 above.

In order to be in the interior, both coordinates of the point must be in the interior of their respective intervals in each direction. If either coordinate is outside the range, then the point is outside. If both coordinates are on their respective boundaries or one is on the boundary and the other is an interior point, then the point is on the boundary. Otherwise, the point is outside. In general, the region is the larger of the row and column signum products.

Point	Description	Row x_p	Column x_p	Region (r)
(6,8)	Outside	1	1	1
(5,2)	Corner	0	0	0
(4,4)	Interior	-1	-1	-1
(7,6)	Below	1	0	1
(3,5)	Edge	-1	0	0
(2,3)	Above	1	-1	1

Table 2: Various points in the plane with respect to a defined rectangular region.

```

xm←{f/>x/xα,.-qω}      A Max Signum
b→(3 2)(5 6)           A Upper Left and Lower Right Corners
b xm→(6 8)(5 2)(4 4)(7 6)(3 5)(2 3) A Various Points
1 0 -1 1 0 1

```

The left argument of the dynamic function xm is a 2×2 matrix containing the coordinates of the upper-left and lower-right corners of a rectangular region. The right argument is a 2-element vector or 2-column matrix containing the coordinates of various points in the plane.

Differentiating Border Regions

Sometimes it is necessary to divide the borders of a dialog box into specific regions. For example when the cursor is over an edge, it will resize the window in one direction. If it is over a corner, it will resize the window in two directions. The particular edge or corner selected also determines the position of the resized window.

A rectangular region naturally subdivides the x - y plane into 25 regions, 5 in the x direction by 5 in the y direction³. The following regions are shown in Figure 3: 1 interior region, four corners, four edges, and 16 outside regions.⁴

³ This is based on the fact that there are 5 disjoint subsets in a range partition on the real line.

⁴ It is unimportant to distinguish these from each other

Outside (8)	Outside (8)	Outside (8)	Outside (8)	Outside (8)
Outside (8)	Upper Left Corner (-4)	Upper Edge (-3)	Upper Right Corner (-2)	Outside (8)
Outside (8)	Left Edge (-1)	Interior (0)	Right Edge (1)	Outside (8)
Outside (8)	Lower Left Corner (2)	Bottom Edge (3)	Lower Right Corner (4)	Outside (8)
Outside (8)	Outside (8)	Outside (8)	Outside (8)	Outside (8)

Figure 3: 25 Regions defined by a rectangle

The signum sum applied to each coordinate generates an ordered pair. If we wish to identify all 25 regions uniquely, we can use base value with a root of 5 to generate the integers -12 to +12 with 0 representing the interior. More likely, we only wish to identify the 9 relevant regions, disregarding the 14 outside regions. We can use base value with a root of 3. This will produce the values -4 to +4 with 0 representing the interior. We will assign the value 8 to the outside regions. We can identify a point in the outside region if the ordered pair of its signum sums contains the value 2 or -2. The function xr generates the appropriate value.

$$xr = \{d + \phi + /x\omega, . - \phi\alpha \diamond ((2v, = |d|)/d) + 2 \diamond 3|d\}$$

$$b \quad xr = (2 \ 3)(3 \ 5)(4 \ 4)(5 \ 2)(6 \ 8)(7 \ 6)$$

$$8 \quad -3 \ 0 \ 2 \ 8 \ 8$$

Theoretical Considerations and Conclusion

Arithmetic boundary checking is based upon the topological properties of the real line and the Euclidean 2-space $R^2[1]$. The boundaries of an interval or a rectangle correspond to limit points in topology. An open interval corresponds to the interior set, whereas a closed interval corresponds to the union of the boundary set and the interior set. A k -cell is defined as an closed interval on the real line where $k=1$ or a closed rectangle in the plane, where $k=2$. [2]

While Boolean functions are useful if binary results are adequate, they do not work as well when more information is required. Location on the real line or in the plane can be determined arithmetically without the use of Booleans. The ternary nature of signum is exploited to handle boundary conditions.

References

- [1] Jerrold Marsden, "Elementary Classical Analysis" W.H. Freeman & Co. 1974, Chapter 2, Topology of R^n , pp 32-44.
- [2] Walter Rudin, "Principles of Mathematical Analysis, Third Edition", McGraw-Hill 1976, pp. 31-32.

Tools, Part 4. Regular Expressions

by Dan Baronet (dbaronet@milinta.com)

In the following article I use quotes and angle brackets to help determine the type of the object I am referring to. 'Quotes' denote a variable or workspace and <angle brackets> refer to a function or operator or file. Often, the context is sufficient to remove ambiguities. Italicized words have a special meaning and often precede their definition. Optional items are shown within [brackets]. Monospace fonts are used for examples.

Introduction

In this article I present regular expressions (RX), and how to make some use of them through `NA` and `SAX` Intrinsic Functions (IFs). RX are not new and they're useful. *nix users have known this for quite some time now. For example, to list all the lines in a file where words with a specific pattern are located or find all the files that start with a specific prefix. A pattern consists of characters, some of them having a special meaning. For example, dot (.) can mean any character but newline, and star (*) means 0 or more times. A pattern denoting all strings starting with the letter 'A' and ending with the letter 'C' would be written *A.*C* meaning *the letter A followed by any number of characters and ending with the letter C*. There are rules too and one of them says that the longest match is returned. There is also a standard. Here a few examples.

Examples

```
e1-' matches any char, ab%c shows this:
'ab.c' showfirst e1
. matches any char, ab%c shows this:
    ^^^^

** ? means 0 or 1, + means 1 or more, * means 0 or more
'ab?.c' showfirst e1 = "a", 0 or 1 "b", any char then "c"
. matches any char, ab%c shows this:
    ^^^^

** \ escapes following character; \" means "."
'ab\\.c' showfirst 'not on ab%c but works on ab.c'
not on ab%c but works on ab.c
    ^^^^

** sets may be specified within [brackets]
'ab[eculc] showfirst 'not abxc but OK with aboc!'
not abxc but OK with aboc!
    ^^^^

** predefined sets like [:digits:] can be used
'ab[[:digit:]]c' showfirst 'with ab3c and abzc!'
with ab3c and abzc!
```

```

****
** sets may also be specified as ranges
'ab[0-9]c' showfirst 'same with ab3c and abzc!'
same with ab3c and abzc!
****

** a set of 2 sets:
'ab[:,space:][:alpha:]c' showfirst 'alpha or spaces matches ab c!!!'
alpha or spaces matches ab c!!!
****

** 2 separate sets:
'ab[:,space:][:alpha:]c' showfirst 'alpha or spaces matches ab zc!!!'
alpha or spaces matches ab zc!!!
****

** exclusion is made with ^:
'ab[^0-9]c' showfirst 'not ab4c but ab5c!!!'
not ab4c but ab5c!!!
****

** subexpressions are specified with parentheses. Here, <rxmatch> returns the
offset and length of the whole pattern and of each subexpression:
'((a*)a)' rxmatch 'zzaaaa'
2 4
2 3
2 2
'a(b[aeiou])c' rxmatch 'x abec x'
2 4
3 2
'a(b[aeiou])c' showfirst 'x abec x'
x abec x
****

** first match wins
'ab*c' showfirst 'xxabbbbbbc abbcabc'
xxabbbbbbc abbcabc
*****
'wee|week)(knights|nights)' rxmatch 'weeknights'
0 10
0 3
3 7

** patterns may be repeated. Here whatever sub expression found after "a" must
also be found after "c" (\1)
'a(b[aeo])c\1' showfirst 'not abecbi but abecbe.'
not abecbi but abecbe.
*****

** the number of times a character must be repeated can be specified {range}
'ab{2,4}c' showfirst 'abbbbbbc(5bs) abbbc(3) abc'
abbbbbbc(5bs) abbbc(3) abc
*****

** Here's an example of pattern to recognise an Identifier:
NameDef+'[:alpha:][:alnum:]*'
NameDef showfirst '3+dsa/7'
3+dsa/7
***
NameDef showfirst '3+dsa_7-31'
3+dsa_7-31
*****

** here are the patterns for a word and word followed by punctuation:
Word: [:alpha:]+

```

```
pat1: ([[alpha:]]+)[[:punct:]]
```

**** <rxmatch> returns the whole match and the subexpression (word):**

```
pat1 rxmatch 'first one, then another.'
6 4
6 3
```

**** This function finds ALL matches:**

```
pat1 rxmatches 'first one, then another, then the last.'
6 4 16 8 34 5
6 3 16 7 34 4
```

**** here are more patterns:**

```
simplestring: [' ']+ a any series of char not a space or quote
```

```
quotedstring: '(['^'])+*
```

(A quoted string is a quote followed by a series of (double quotes OR a char not a quote) followed by a final quote)

```
token: '('|'|'^')*|[' ']+
```

(a token, here, is either a simplestring or a quotedstring)

**** find all the tokens in 's1'**

```
s1: age = 42 name= 'somkinda' djo'
token rxmatches s1
0 3 4 1 6 2 10 5 16 16
30 1
```

NOTE: the first 4 matches are *simplestrings* which have no subexpressions in them. A *quotedstring* with a subexpression was found at the end and the result contains 2 rows there.

**** find all lowercase letters starting a sentence, that is, here, a lowercase letter after a dot, ? or !: (note that the first 'b' does not fall in that category because there is no such character before)**

```
e1: blah blah. blah blah? Blah blah! blah! Ah!
```

```
'[.\?!] *([a-z])' rxmatches e1
9 3 31 3
11 1 33 3
```

**** we can show all the matches in CR delimited text.**

Show all 3 letter words in this 3 lines text:

```
"Now is the time for all good men to
learn more about APL and show they too can make
the world a better place to live in."
```

```
(' ([^ ]{3}) ' 1) showall text
Now is the time for all good men to
    ^^^ ^^^ ^^^
learn more about APL and show they too can make
    ^^^ ^^^ ^^^
```

The APL world

Having RX in APL is also useful.

The bad news is, trying to program them from scratch is not for the faint of heart. Some serious (and successful) attempts have been made using techniques like finite state machines to program a subset of RX but for the most part it's not easy. Plus, it would probably be different from any "standard".

The good news is, someone's already been through this. Regular expressions source is out there. DLLs have already been compiled if you're not into doing it yourself. All you have to do is search the net⁵, download it, and [N]A into it. This is, BTW, what J has done using 15!0 instead of [N]A.

RX definition

There are 4 functions that compose the GNU "standard". Their definition follows; to the left we have the routine name and its arguments definition (> means output, < means input, S=C=character, I=integer). To the right we have their C definition.

APL	C (return type & arguments)
RXcomp: >*C, <*C, I	int-(S*, char*, int)
RXexec: <*C, <*C, I, >*I, I	int-(S*, char*, int, int*, int)
RXerror: <*C, I, >*C	int-(int, int, char*)
RXfree: <*C	void-(char *)

The way this works is like this: to use regular expressions we must first "compile" the pattern we're looking for before using it on (possibly) several strings. Routine **RXcomp** does that. It takes 3 arguments: the 1st one is a returned structure whose detailed description isn't necessary here. Suffices to know that less than 32 bytes are used⁶. The 2nd, the pattern, is a NULL terminated string of what we're looking for. The 3rd and last argument is a behaviour code, 1+2 if case insensitive+4 if CRs are to be excluded from the search (usually the case). It returns an error code if the pattern is not proper. **RXerror** is used to interpret this error code if needed.

RXexec is the routine that does the work. You give it 5 arguments: the compiled structure provided by **RXcomp**, the string to search, the number of subexpressions

⁵ There is one at www.milinta.com/english/regex.dll

⁶ in the examples I often use a pointer to a character string instead of the exact structure

start-end positions to return, room for those and a behaviour code (0 for us). It returns 0 if it finds something and 1 if it finds none.

When finished with the compiled pattern we should free memory by using `RXfree` providing it as argument the structure that was returned by `RXcomp`.

This is pretty much the way it is done in APL too. To use those routines we use `⌈NA` or some such to link to each one of the routines.

⌈NA

As usual, `⌈NA` syntax varies from one APL vendor to another.

For example, with APL+Win you have to supply the NULL to NULL terminated strings. With Dyalog you tell `⌈NA` if it's a NULL terminated string (an added benefit for Unicode 16 bits handling) but you must remember to translate (to ASCII) since Dyalog's `⌈AV` isn't based on ASCII like some other vendors. And the returned results may be different! SAX uses Intrinsic Functions (IFs) instead which can be packed to file. I'll describe all of them.

Let's assume the DLL file is named `<regex.dll>` and that the routines in it are called `RXcomp`, `RXexec`, `RXerror` and `RXfree` as above.

APL+Win (tested on V3.5)

For each RX routine we provide an unused APL name as right argument to `⌈NA` and a descriptor of the associated routine as left argument. `⌈NA` will return 1 if the function was successfully created in the workspace.

The left argument should contain 'DLL ' followed by

- the type of the return (B=boolean, I=int, C=character, F=float, G=general, U=unsigned) and the number of bytes
- +
- the name of the DLL file (without the extension)
- a dot
- the routine name
- the arguments' type and length in parentheses. Like this:

```
'DLL I4+regex.RXcomp(*C1, *C1, I4)' ⌈NA 'comp'
```

We use these functions like this:

```
S←32/' ' ⋄ errorcode←comp S (pattern,NULL) 5
```

If a 0 is returned then all is well. 'S' has been modified and contains a structure used to search a string, like this:

```
'DLL I4←regex.RXexec(*C1,*C1,*I4,I4)' ⌈NA 'find'
se←1 1 ⋄ found←0←find S (string,NULL) 1 se 0
```

If found then <se> contains the offset of the first pattern found and the offset of the first char following it, effectively providing its length by doing | - / on it. The NULL is necessary for the pattern and strings to search as specified by the specs.

Once defined all these functions can be saved in a workspace.

Dyalog APL (tested on V8.2 & 10.0)

Things are a bit different here.

For each RX routine we provide an unused APL name as LEFT argument to ⌈NA and a descriptor of the associated routine as RIGHT argument (this is the reversed of APL+Win). Note that if you fail to provide a name ⌈NA will use the same name as the name of the routine in the DLL. ⌈NA does not return a result and will signal a DOMAIN error if anything goes wrong.

The right argument should contain

- Optional result type (I=int, U=unsigned, C=character, T=translated character, F=float, A=APL array) and length
- the name of the DLL file
- | (ASCII | or APL modulo)
- the routine name
- the arguments. Like this:

```
⌈NA 'I regex|RXcomp =C[] <0T I'
```

Because Dyalog's ⌈AV isn't based on ASCII we must tell ⌈NA to translate them where appropriate by using T instead of C. We can also tell ⌈NA which strings are NULL terminated in its definition by prepending a '0'. Thus '0T' tells ⌈NA that this is a NULL terminated Translated string.

Dyalog also differs in the way it returns results. You can specify whether an argument is a pointer and will be used as input, output or both by prepending the characters '<', '>' and '=' to the argument descriptors. Thus, an output list of

integer would be written '>I[]'. A returned argument is part of the result and not modified "by reference" as with APL+Win. The only real difference between '>' and '=' lies in the argument itself: with '>' you tell how much memory to allocate. With '=' you supply an array whose copy will be returned, modified. For example, assuming routine <try> returns an integer and returns 16 bytes by reference it would be described as

```
⌵NA 'I4 dllfile|try >C[16]'
```

and called like this:

```
try 16
```

but if it takes a specific array of 16 bytes and returns them it would be

```
⌵NA 'I4 dllfile|try =C[16]
```

and called like this (16=pdata):

```
try data
```

The result will be a 2 elements nested array that includes the 16 bytes as second element in both cases.

In the RX context we would have

```
'comp' ⌵NA 'I regex|RXcomp >C[] <OT I'
```

and do

```
(errcode S)+2+comp (32 pattern 5)
```

If errcode is 0 then all is well and 'S' contains a structure used to search a string, like this:

```
'find' ⌵NA 'I regex|RXexec <C[] <OT I >I[] I'
(notfound se)+find (S string 1 se 0)
```

If found then <se> contains the offset of the first pattern found and the offset of the first char following it, just like in APL+Win.

Functions cannot be saved in the workspace. They have to be redefined each time the workspace is loaded.

APLX (tested on V2.0.2 beta)

Looks like Dyalog, but again, different.

For each RX routine we provide an APL name as LEFT argument to `⌈NA` and a descriptor of the associated routine as RIGHT argument. If you fail to provide name `⌈NA` will use the same name as the name of the routine in the DLL. But be careful! It will redefine the name even if it is already in use. `⌈NA` returns 0 0p ' ' and will signal a DOMAIN error if anything goes wrong.

The right argument should contain

- Optional result type (I=int, U=unsigned, C=translated character, P=translated Pascal string) and length
- the full name of the DLL file
- |
- the routine name
- the arguments. Like this:

```
⌈NA 'I4 regex.dll|RXcomp =I[8] <CT[*] I4'
```

Because APLX's `⌈AV` isn't based on ASCII all character strings are translated by default or by using T after the C. If untranslated strings are required a U should follow the C. We don't need to tell `⌈NA` which strings are NULL terminated. All 'C' strings are padded with one.

Because we don't want APLX to add a NULL to the first argument of `RXcomp` we define it as a pointer to a series of 8 integers (8x4=32 bytes) instead.

APLX is like Dyalog in the way it returns results. You can specify whether an argument will be used as input, output or both by prepending the characters '<', '>' and '=' to the argument descriptors. Thus, an output list of integer would be written '>I []'. A returned argument is part of the result and not modified "by reference" as with APL+Win. In both cases you supply an array whose copy will be returned, modified (see the Dyalog examples above for the RX context).

Unlike Dyalog, the functions do not need to be redefined at each load and can even be saved in the workspace and overlays.

Note that since APLX is identical in behaviour under Linux the above will also work (at least under Red Hat 7.0 and Mandrake 9.1) if you change the location of the RX routines. For example:

```
⌈NA 'I4 /lib/libc.so.6|RXcomp =I[8] <CT[*] I4'
```

would be the statement to use for `RXcomp` under Linux.

APLX also works under MacOS X. According to Richard Nabavi of microAPL: "Under MacOS X, the regular expression stuff is again built-in to the OS libraries. (It's not available under MacOS 9, as far as I know). The library name to use in the JNA definition is 'System.framework'. However, there's a slight gotcha: under MacOS X the offsets in the returned structure are defined as 64-bit integers, not 32-bit integers as in all the other systems. So you need to double the size of the array passed as the fourth argument of 'regexexec', and pick up the results from the 2nd, 4th... elements: a match from position 4 to 7 comes out as: 0 4 0 7."

SAX

This is Linux/Unix world. JNA, here, is a different thing altogether. To achieve the same result we can use intrinsic functions (IFs) instead. An added benefit being to allow packing, like APLX. The procedure is like this:

Create an IF group that will contain the 4 functions. In file `~/saxif` add the lines that define the groups and how the functions work:

```
regex dynamic    ~/regex.si
comp Cfunction int  RXcomp(char*,char*,int)
exec Cfunction int  RXexec(char*,char*,int,int *,int)
error Cfunction int  RXerror(int,char*,char*, int)
free Cfunction void RXfree(char *)
```

Then do 'ifgen `~/saxif regex`'. This will produce a `<regex.stub>` file that contains precompiled definitions for the compilation to come.

We now have to write the C code. All the RX code should already be all there and all we really have to do is put an '#include' statement to reference it. We put the following 3 lines in file `<regex.c>`:

```
#include <sys/types.h> // standard types
#include <regex.h>      // the regex code
#include "regex.stub"   // the definitions above
```

We are now ready to compile. We do so by issuing

```
ifcc regex.si regex.stub regex.c
```

which produces file `<regex.si>` that SAX will use to produce the binds.

When we start SAX we can verify that the group 'regex' is in the list of intrinsic functions by doing `jiflist` and that all 4 functions are in it by doing `jiflist regex`. To associate an APL name with one of them we do, for ex:

```
'regex.comp' ⍷bind 'rxcomp'
```

and voilà! <rxcomp> is now a full-fledged IF that is used like this:

```
code+rxcomp S>pattern>5 → S+32/' '
```

S now contains the compiled pattern (unless code is not 0).

Note for *Solaris* users: the RX definition is slightly different and even though the ⍷bind will work, the APL functions won't work as is. You need to triple the size of the array passed as the fourth argument of 'regexcomp', and pick up the results from the 3rd and 4th elements.

Cover functions

Now, all these functions are cute but not very useful by themselves. We've seen how to interface with the system but that is not enough. We must use them like building blocks. We need a function that, given a string and a pattern, compiles the pattern, runs it through RXexec, frees memory and returns the desired result, possibly ALL the matches, not just the first one.

I wrote these functions:

RXmatch: find the first match

RXmatches: find ALL matches

Showfirst: show the first line where a match is found with carets to show the positions

Showall: same for all matches

They're only prototypes. I put them, along with an <example> function, in a workspace you can download if you wish.

Now, what about searching an entire workspace? Well, those of you who have been reading Vector carefully for the past year may remember about a 'wssearch' workspace I described in Vol 20 no.1 of July 2003 entitled 'Tools, part 2'. I had mentioned in that article that a pattern matching function would allow searching the workspace for, well, patterns.

Well, search (!) no more. You'll find, in the same workspace, along with those other functions above, function <OSSPATMATCH>, which can be copied into workspace 'wssearch' to allow searching with patterns, like this:

```
oss 'p/⍷[:a]pha:])+'
```

which will find occurrences of, in this case, all $\square$ functions and variables in the workspace (no $\square$ alone will be reported).

<OSSPATMATCH> will only be called if the 'p' option is supplied (see the article for details). It is basically $\underline{g}$ using a pattern as left argument. It uses local names to associate with IFs or DLL functions.

Final note

I don't know of any complete standalone APL implementation of RX. There was a paper published circa 1984 on RX by some I. P. Sharp people but its code was only ever included in Logos, a SAPL product. It turned out to be a powerful tool as it was geared towards APL and included some features like ω to denote a number (even complex numbers as SAM had them) and α to denote an identifier plus a lot much more.

For example, one could look in code for strings denoting APL filenames like this: ' * ω + α * ' meaning "look for quote followed by 0 or more spaces, then a number, then 1 or more spaces, then an identifier, then 0 or more spaces, then a quote". Even more, Logos included features that could never be part of the actual available RX such as line number and char position and the use of $\$$ in *replace* mode.

In the filename example above, you could have replaced ALL similar hardcoded filenames with unique variable names. Extremely powerful! Unfortunately, few people had the chance to deal with Logos.

As usual the code shown in this article and the DLL are available at www.milinta.com/english. Feel free to play with it.

APL to Unicode Transcoding in Java

by Mark Wickens (m.wickens@rhodium-consulting.com)

Summary

A method for translating between vendor-specific APL source files and Unicode files is presented. This allows APL source files to be viewed, edited, printed and shared by any Unicode-compatible application, and between APL applications.

The Problem

One of the problems with APL is that it uses a custom character set. This presents an additional hurdle for the potential user in that any APL installation must install a set of custom fonts for the APL application to use. As font-handling is still fairly operating-system dependent installation problems are common. Further, as most APL systems are 'mature' their font handling has not benefited from the standardization effort for font encoding that is Unicode. Finally, as APL source files are vendor-specific they are not compatible between different APL systems.

Various APL implementations have solved the font encoding problem in much the same way. They have taken the standard ASCII (or EBCDIC) character set and extended it to 8-bits (standard ASCII only defines the meaning of the first 7-bits) then used the additional 128 characters to encode the custom APL symbols. Whilst within the APL application environment the encoding mechanism is transparent and therefore does not present a problem. However, any attempt to view, edit, or print APL files outside the APL environment is made difficult by the fact that the viewer, editor or print utility must take into account the special font required to view the APL symbols. Sharing or publishing of APL documents is even more difficult because the recipient must have the APL font installed on their system and an application configured to use the font before they can view the APL source file contents correctly.

Unicode

A concise definition of Unicode is: Unicode provides a unique number for every character, no matter what the platform, no matter what the program, no matter what the language.

The result of Unicode is a database of characters together with their corresponding description and unique number. The unique number is 16-bits

long, so there can be a maximum of 65,535 ($2^{16}-1$) characters. The Unicode standard is realised on any particular computing platform by the integration of three key elements:

- A font-engine supporting Unicode fonts. There are two main font engines in use at the moment that support scalable fonts: TrueType and PostScript.
- A font containing glyphs (the definition of how each character should be drawn by the font-engine) for each of the Unicode characters.
- An application supporting the display of characters contained within the Unicode font, and some method of loading and storing files containing Unicode characters,

The Unicode standard includes a section that defines a unique number for each of the symbols defined within the APL language. It is therefore possible to describe any APL source file using Unicode characters⁷.

Any APL system that uses an 8-bit font encoding mechanism is not Unicode-compatible. The problem is therefore how to translate the vendor-specific APL files into Unicode-compatible files and visa-versa.

Java and Unicode

Java was designed to support the Unicode standard from the ground up. The primitive type 'char' in the java language is defined to be 16-bits and can therefore contain a unique number for each of the 65,535 possible characters that can be defined within Unicode.

Java uses a set of character transcoders to translate between the internal Unicode representation of characters and other encoding mechanisms such as ASCII. The standard Java runtime contains encoders and decoders for the following standards:

ASCII	- 7-bit encoding
Cp1252	- 8-bit extension of ASCII used on Windows computers
ISO8859_1	- 8-bit extension of ASCII used on Unix computers
MacRoman	- 8-bit extension of ASCII used on Mac computers

⁷ the fonts supplied with some APL systems may contain additional characters that are not part of the APL language, for example line-drawing characters. For the transcoders supplied, the nearest equivalent character defined within the Unicode standard is used where possible. Where there is no obvious equivalent, the character is mapped to the space character.

UTF8 - Compressed format for Unicode
Unicode - 16-bit Unicode characters

There are many more character transcoders supported within java for translating between foreign language encodings and Unicode. The standard transcoders are defined within the package `sun.io` and the classes are contained within the runtime archive `rt.jar`. For international version of the java run-time environment, an additional archive `charsets.jar` contains additional foreign language transcoders.

Java Character Encoder/Decoder Service Provider Mechanism

Java 1.4 contains a mechanism by which custom character encoders and decoders can be seamlessly integrated into the java runtime and made available to applications transparently. The mechanism is called a *Service-Provider Interface* and is defined within the java package `java.nio.charset.spi`.

Unicode Fonts Defining the APL Symbols

At present I know of only one Unicode font that defines all the APL symbols contained within the Unicode standard, together with a standard set of normal characters. This is Phil Chastney's *SiMPL* truetype font⁸. The font is described in his article *An APL Unicode Font*⁹.

Downloading, installing and using the APL Transcoders

NOTE: you must have version 1.4 or higher of the java runtime/SDK to use the transcoders.

The transcoders are contained in the java archive file `apl-transcoding.jar`. This file must be installed on the java CLASSPATH so that the java runtime has access to it. The easiest way to do this is to copy it into the java runtime library extensions directory. If you have installed the java runtime in the default location, the library extensions directory is located at `C:\Program Files\Java\jre1.4.2\lib\ext` (your specific version number may differ). If, alternatively you are using the Java System Development Kit (SDK) then the directory will be `C:\j2sdk1.4.2\jre\lib\ext`.

⁸ <http://www.vector.org.uk/resource/simpl02.zip>

⁹ <http://www.vector.org.uk/v161/phil161.htm>

Once the transcoding file is located within the library extensions directory, it will be automatically loaded whenever a java application is run. If your application supports the selection of file encodings when loading a file, you should be able to select one of the supported APL file types. Note that the transcoding mechanism does not provide a way for the application to query the available transcodings, so you are likely to have to supply a custom encoding name. The currently supported APL encodings are:

ENCODING NAME	APL SYSTEM FILE FORMAT
-----	-----
SOLITON-APL	Soliton Associates' SHARP APL for Linux

The open-source editor jEdit¹⁰ is an excellent example of an editor that utilises the flexibility of the java character transcoding mechanisms for supporting numerous native file formats.

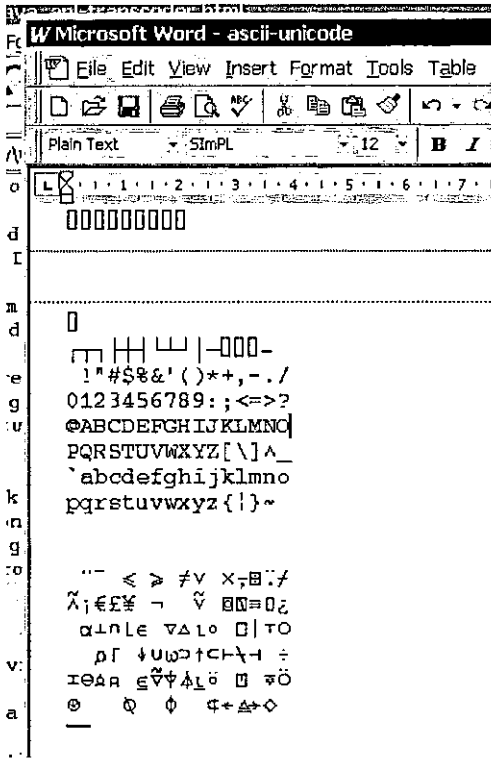
As an example, if you use jEdit then you can load a SHARP APL encoded file by:

- Selecting the File->Open menu item (Ctrl-O).
- Right clicking on the SHARP APL file and selecting the option menu item Encoding->Other...
- Entering a value of SOLITON-APL in the Character Encoding to Use text field.

Once a file has been opened, jEdit remembers the last encoding specified, so you only have to perform this operation once for each APL file. Saving the file causes it to be translated back into the SHARP APL format. Any Unicode characters that cannot be translated into a value in the 8-bit SHARP APL character map are stored as spaces.

In order to view the APL symbols correctly you must install a Unicode font that contains definitions for the APL symbols. Such a font is Phil Chastney's SImPL truetype font. This must be selected as the text editor area font within jEdit (select the Utilities->Global Options... menu item, then the jEdit-Text Area tree-option then select the Text Font option menu. The checkboxes Smooth Text and Fractional Font Metrics should be highlighted for the best quality display.

¹⁰ <http://www.jedit.org>



If you wish you can then specify that the font used should be embedded in the word document - you then have a word document containing APL source code that can be emailed to anyone to be viewed, edited or printed.

Transferring Source Code between different APL Systems

If you wish to load an APL source file in the native-format of one APL system (for example, Sharp APL) and then save it in the native-format of another system (for example, IBM APL2) then you must set the encoding option for the buffer before saving. This can be done in jEdit by loading the APL source file as described above, then selecting the Utilities->Buffer Options menu. You can

then enter the encoding format for the file to be saved as (in this case you would enter a character encoding of IBM-APL2 then save the file using a different name). The file will be encoded in the new format in the saved file. Note, however, that there is no guarantee that the source will run in an alternative system. That is entirely a different problem!

Java Source for the Transcoding Library

The source code for the transcoding library can be downloaded in the file `apl-transcoding-src.jar`¹¹.

¹¹ <http://www.rhodium-consulting.com/apl/apl-transcoding-src.jar>

Index to Advertisers

MicroAPL Ltd	2
Vector Back Numbers	4

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2004/2005 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Via Monferrato, 1 20144 - Milano, Italy
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Bob Hoekstra 01483-771028 bob.hoekstra@hoekstrasystems.ltd.uk	Dominique, Salisbury Road WOKING, Surrey GU33 7UR
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Stephen Taylor sjt@vector.org.uk	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra (01483-771028), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696780
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Dyalog Ltd
Grove House, Luyens Close,
Chineham Court, BASINGSTOKE,
Hants RG24 8AG
Tel: 01256-338461
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems Aps
Nordre Strandvej 119G
DK-3150 Hellebæk, Denmark
Tel: +45 70 26 13 26
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield, E Sussex TN22 5AA
Tel: 01825-768050
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St., No. 375
Palo Alto CA 94301 USA
Tel: +1 866 kdb fast
Email (general enquiries): info@kx.com
In Europe: Simon Garland, simon@kx.com

Optima Systems Ltd
Optima House, Mill Court, Spindle Way, Crawley,
West Sussex, RH10 1TT
Tel: 01293 562700
Email: paul@optima-systems.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850, USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Soliton Inc
44 Victoria Street, Suite 2100
Toronto, Ontario M5C 1Y2, Canada
Tel: +1 (416) 364 9355
Email: sales@soliton.com
Web: www.soliton.com

First Derivatives plc
First Derivatives House
Kilmorey Business Park, Kilmorey Street,
Newry Co. Down, N. Ireland BT34 2DH.
Tel: 028 3025 2242
Email: enquiries@firstderivatives.com

The Carlisle Group
544 Jefferson Avenue, Scranton PA, 18510
USA
Tel: +1 (570) 963-2036
Web: www.carlislegroup.com